

计算机网络

第一章作业问题讲解

R18

- ❑ 一个长度为1000字节的分组经距离为2500km的链路传播，传播速率为 $2.5 \times 10^8 \text{ m/s}$ ，传输速率为2Mbps，它需要用多长时间？
- ❑ How long does it take a packet of length 1,000 bytes to propagate over a link of distance 2,500 km, propagation speed $2.5 \cdot 10^8 \text{ m/s}$, and transmission rate 2 Mbps?
- ❑ 更为一般的，一个长度为L的分组经距离为d的链路传播，传播速率为s，传输速率为Rbps,它需要用多长时间？
- ❑ 该时间与传输速率相关吗？

a) 10 msec

b) d/s

c) 无关

R19

- 假定主机A要向主机B发送一个大文件，从主机A到主机B的路径上有三段链路，其速率分别为 $R_1=500\text{kbps}$ ， $R_2=2\text{Mbps}$ ， $R_3=1\text{Mbps}$
 - a. 假定该网络中没有其他流量，该文件传送的吞吐量是什么？
500kbps
 - b. 假定该文件大小为4MB。该文件传输到主机B大致需要多长时间？
4MB/500kbps
 - c. 重复(a)和(b)，只是这时候 R_2 减少到100kbps
100kbps, 4MB/100kbps

P25

3. 假定两台主机 A 和 B 相隔 20,000km(即 $2 \times 10^7 \text{m}$),由一条直接的 $R=2\text{Mbps}$ 的链路相连。
假定跨越该链路的传播速率是 $2.5 \times 10^8 \text{m/s}$ 。 ←

a) 计算“带宽时延”积 $R \cdot t_{\text{prop}}$ (R 指传输速率, t_{prop} 指传播时延)。 ←

答: $t_{\text{prop}} = 2 \times 10^7 \div 2.5 \times 10^8 = 0.08 \text{s}$, $R \cdot t_{\text{prop}} = 2 \text{Mb/s} \times 0.08 \text{s} = 0.16 \text{Mb}$ ←

b) 从主机 A 向 B 发送一个 800,000b(0.8Mb)的文件。假定该文件作为一个大的报文连续发送。在任何给定的时间,在链路上具有的比特数量最大值为多少? ←

答: $0.8 \text{Mb} / 2 \text{Mbps} = 0.4 \text{s} > 0.08 \text{s}$, $0.08 \text{s} \times 2 \text{Mbps} = 0.16 \text{Mb}$. 在链路上具有的比特数量最大值为 0.16M. (如果大文件都传输完—都推上链路,需要 0.4 秒,但在路上传播只需要 0.08 秒,也就是要算 0.08 秒时间内,多少 bit 在链路上 ---gm 注) ←

c) 一段链路的带宽时延积: 在这段链路上最多可能有多少比特

d) 一个比特的宽度,比足球场更长吗? $20000 \text{km} / 0.16 \text{Mb} = 125 \text{m/b}$, 一个比特宽度为 125 米 (正规足球场场地: 105 米 * 68 米) 比足球场更长

e) 传播速率 s , 带宽 R (传输速率) 和链路长度 m

一个比特宽度: $m / \text{比特数(带宽时延积)} = m / (R \cdot t_{\text{prop}}) = m / (R \cdot m/s) = s/R$

P26

对P25, 什么样的R值, 一个比特的宽度能与该链路的长度一样?

$s/R=20000\text{km}$, then $R=s/20000\text{km}=2.5*10^8/(2*10^7)=12.5\text{ bps}$

P27

现在 $R=1\text{Gbps}$

- a) 计算带宽时延积。80,000,000 bits.
- b) A到B发送800000比特的文件（作为一个大报文连续发送），链路上具有的比特数量的最大值是？

800,000 bits, this is because that the maximum number of bits that will be in the link at any given time = min(bandwidth delay product, packet size) = 800,000 bits. 注意, 文件只有800,000 bits, 所以这个值和a)不一样

- c) 该链路上一个比特的宽度是多少? 0.25 meters

P28

□ 再次考虑习题P25（假定两台主机A和B相隔20000km，由一条直接的R=2Mbps的链路相连。假定跨越该链路的传播速率是 $2.5 \times 10^8 \text{m/s}$ 。）

a.假定连续发送，发送该文件需要多长时间？

0.48s (传输0.4s、传播0.08s)

$$t_{trans} + t_{prop} = 400 \text{ ms} + 80 \text{ ms} = 480 \text{ ms}.$$

b.假定现在该文件被划分为20个分组，每个分组包含40000比特。假定每个分组被接收方确认，确认分组的传输时间可忽略不计。最后，假定前一个分组被确认后，发送方才能发送分组。发送该文件需要多长时间？

3.6s (传输0.4s、传播 $0.08\text{s} \times (20 \times 20) = 3.2\text{s}$) (注意分组是需要确认的)

$$20 * (t_{trans} + 2 t_{prop}) = 20 * (20 \text{ msec} + 2 * 80 \text{ msec}) = 3.6 \text{ sec}.$$

c.比较（a）和（b）的结果。

(a)结果更小，(b)主要是浪费在多个传播时延上了

Breaking up a file takes longer to transmit because each data packet and its corresponding acknowledgement packet add their own propagation delays.

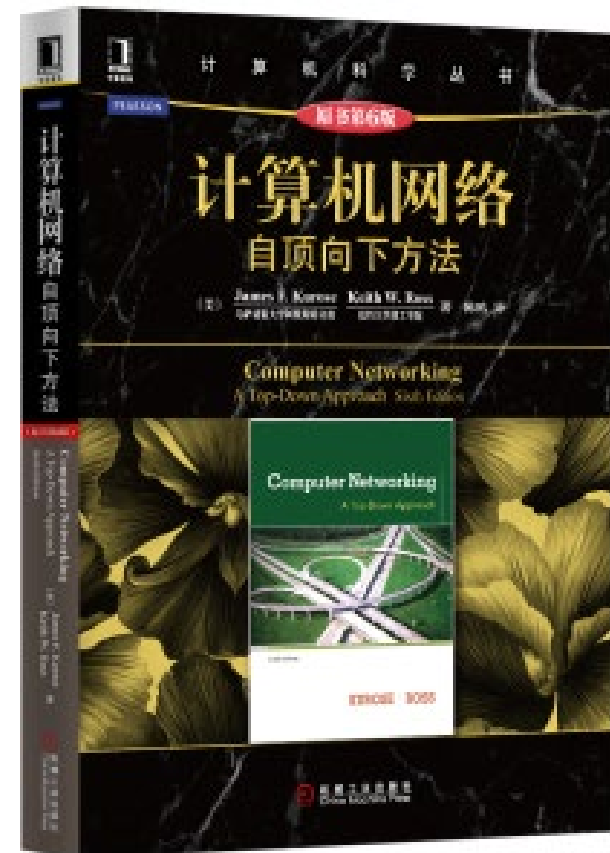
计算机网络：自顶向下方法

陈鸣 翻译，机械工业出版社

Computer Networking: A Top Down Approach

James Kurose
Keith Ross

高旻
2026年3月



Chapter 2: Application Layer

Our goals:

- ❑ conceptual, implementation aspects of network application protocols
 - ❖ transport-layer service models
 - ❖ client-server paradigm
 - ❖ peer-to-peer paradigm
- ❑ learn about protocols by examining popular application-level protocols
 - ❖ HTTP
 - ❖ SMTP / POP3 / IMAP
 - ❖ DNS
- ❑ programming network applications
- ❑ socket API

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.4 DNS
- ❑ 2.5 P2P applications
- ❑ 2.6 Video Streaming and Content Distribution Networks
- ❑ 2.7 Socket programming
- ❑ -FTP
- ❑ +Video Streaming and CDN

Some network apps

- ❑ e-mail
- ❑ web
- ❑ instant messaging
- ❑ remote login
- ❑ P2P file sharing
- ❑ multi-user network games
- ❑ streaming stored video clips
- ❑ voice over IP
- ❑ real-time video conferencing
- ❑ grid computing

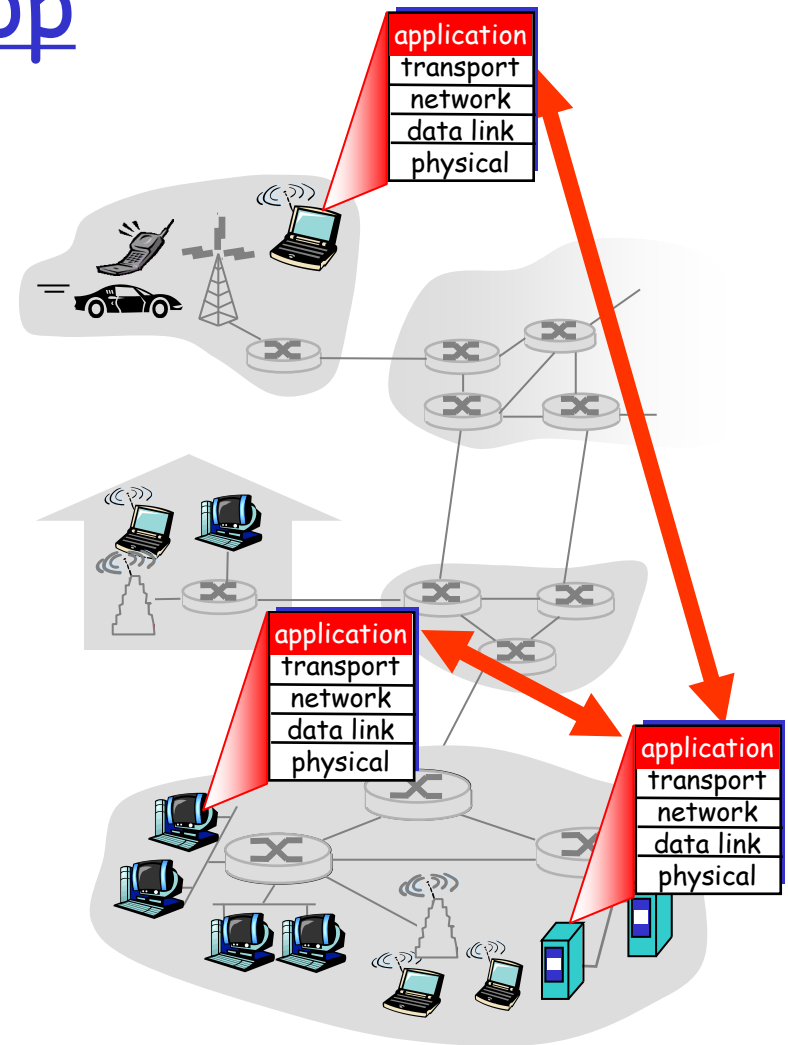
Creating a network app

write programs that

- ❖ run on (different) *end systems*
- ❖ communicate over network
- ❖ e.g., web server software communicates with browser software

No need to write software for network-core devices

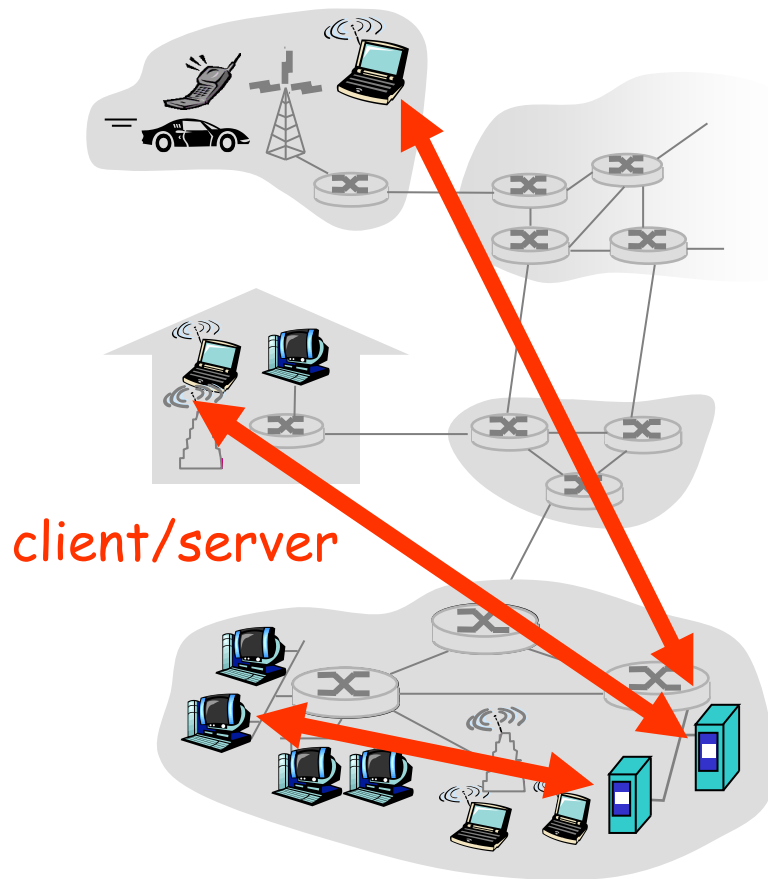
- ❖ Network-core devices do not run user applications
- ❖ applications on end systems allows for rapid app development, propagation



2.1.1 Application architectures

- ❑ Client-server
- ❑ Peer-to-peer (P2P)
- ❑ Hybrid of client-server and P2P

Client-server architecture



server:

- ❖ always-on host
- ❖ permanent IP address
- ❖ server farms for scaling

clients:

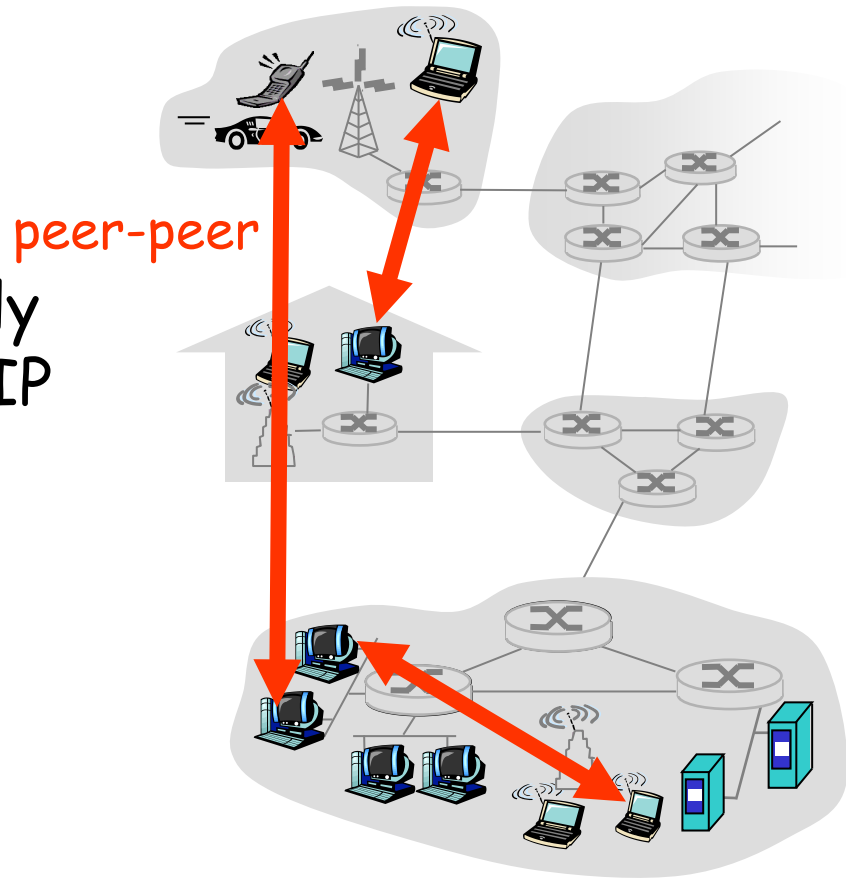
- ❖ communicate with server
- ❖ may be intermittently connected
- ❖ may have dynamic IP addresses
- ❖ do not communicate directly with each other

问题: **Web**应用是不是这种架构?

Pure P2P architecture

- ❑ no always-on server
- ❑ arbitrary end systems directly communicate
- ❑ peers are intermittently connected and change IP addresses

Highly scalable but
difficult to manage



3、客户机/服务器与P2P的混合

□ Napster: MP3文件共享应用程序

- P2P: 对等方直接交换MP3文件
- 服务器注册/定位:
 - 对等方在中心服务器上注册内容
 - 对等方查询相同的中心服务器以定位内容

□ 即时讯息:

- P2P: 两个用户直接聊天
- 服务器检测/定位:
 - 用户在线时, 向中心服务器注册其IP地址
 - 用户联系中心服务器以找到聊天伙伴的IP地址

2.1.2 进程通信 P58

□ 网络应用程序由**成对的进程组成**，并通过网络相互发送报文

✓ **客户机进程**：发起通信的进程

✓ **服务器进程**：等待其他进程联系的进程

如Web应用程序中，一个客户机浏览器进程向某个Web服务器进程发起联系，交换报文

□ **进程 (process)**：在主机上运行

□ **进程通信**

✓ **同一主机中两个进程间的通信**：由**操作系统**控制

✓ **不同主机中进程间的通信**：通过网络交换**报文**进行

◆ **发送进程**：产生报文并向网络发送

◆ **接收进程**：接收报文，并回送报文

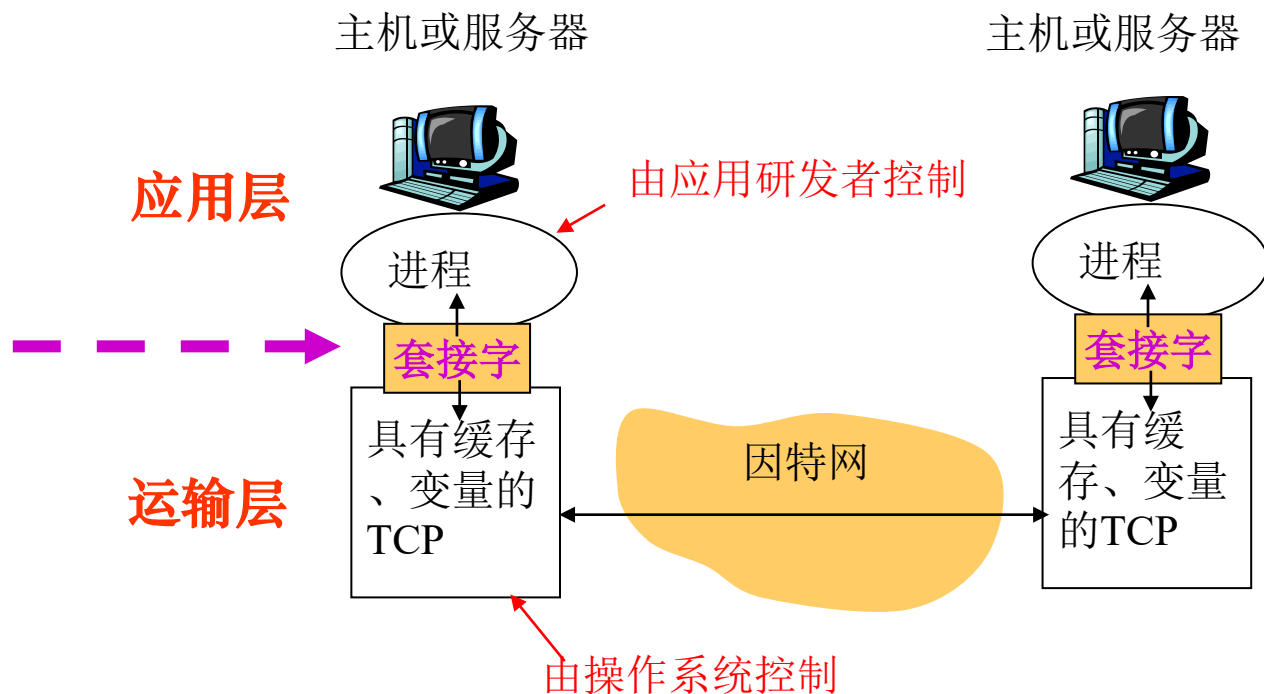
说明

- P2P结构的应用程序也可分别看成是客户机进程或服务
器进程
- 如，对等方A（**客户机**）请求对等方B（**服务器**）发送
某个文件

2 进程与计算机网络之间的接口

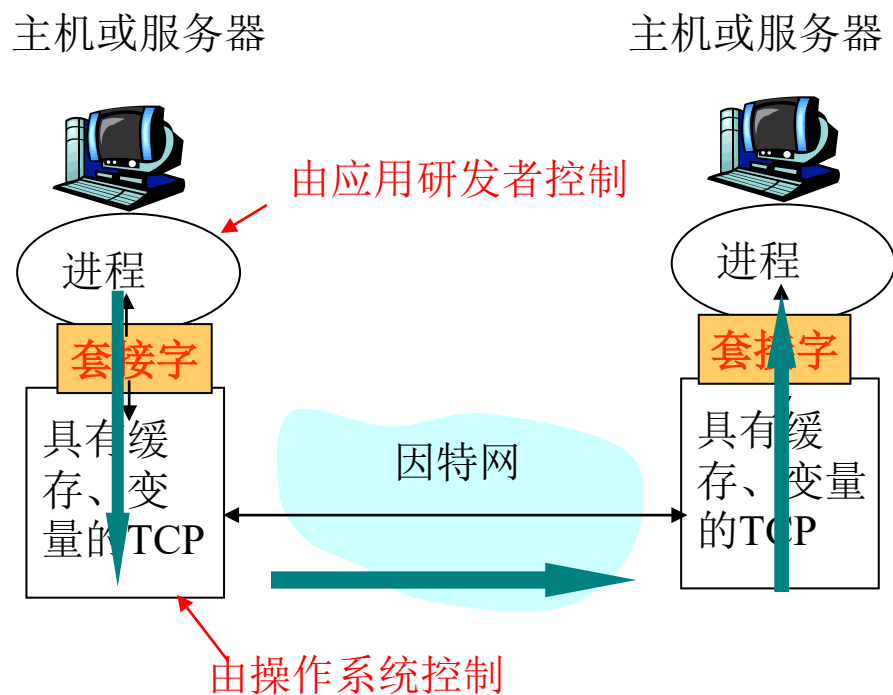
- ✓ **套接字socket**: 同一台主机内**应用层与运输层**之间的接口

也叫应用程序和网络之间的**应用程序接口API**, 是在网络上建立网络应用程序的**可编程接口**



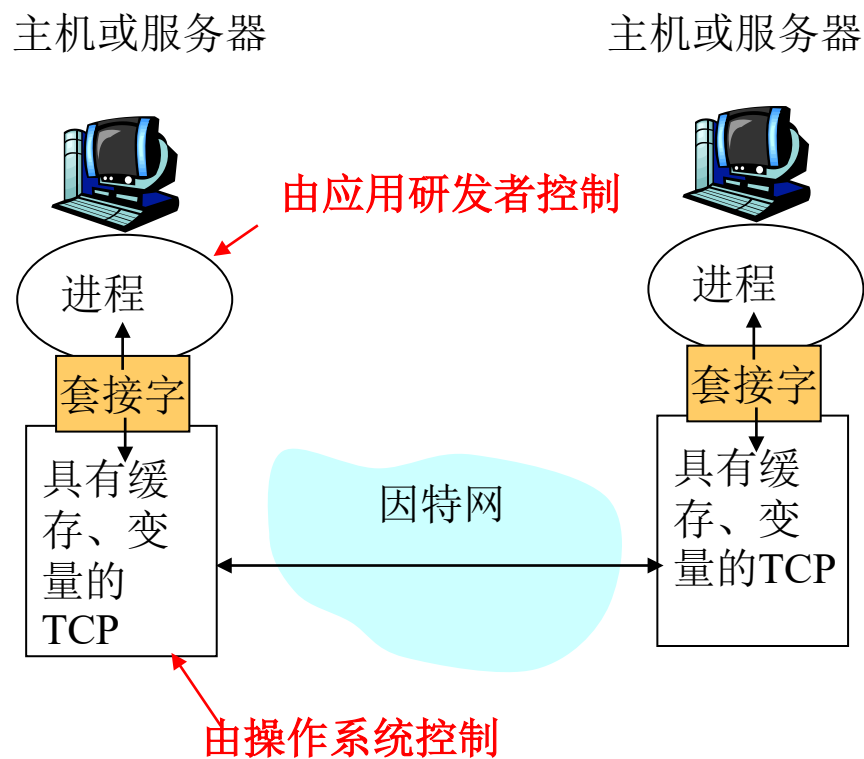
进程与套接字关系

- ✓ 进程类似**房子**，套接字是进程的**门**
- ✓ 进程通过**套接字**在网络上发送和接收报文
- ✓ **发送进程**：把报文推出门（套接字）
- ✓ **传送报文**：通过下面网络把报文传送到目的进程门口
- ✓ **接收进程**：通过其门（套接字）接收报文



说明

- ✓ 应用程序开发者**可以控制套接字应用层端的全部**
- ✓ 对套接字的**运输层端几乎不能控制**（只能选择运输层协议、设定几个运输层参数等）
- ✓ 应用程序开发者选择了一个运输层协议，则应用程序就建立在由该协议提供的运输层服务之上，如TCP协议



3、进程寻址

如何识别
进程？

- ✓ 主机上的进程可以有多个
- ✓ 网络中有多个主机，每个主机上有多个进程

□ **进程识别信息：**表示哪台主机上的哪一个进程

源主机上的进程向目的主机上的进程发送报文时，应带有接收进程的识别信息（标识）

□ **进程寻址：**根据进程识别信息找到相应进程

确定主机 → 确定进程

进程识别信息（两部分）

- **主机名称或地址：**网络中的哪一个主机

因特网中，用**IP地址**标识（32位）

- **进程的标识：**主机中的哪一个进程

因特网中，采用**端口号**标识(port number)

- ✓ 常用的应用程序被指派固定的端口号（**周知端口**）

如，Web服务进程(HTTP协议)：80

邮件服务进程(SMTP协议)：25

- ✓ 创建一个新的网络应用程序时，必须分配一个新的端口号，不重复

应用程序需求

- ❑ **可靠的数据传输（无数据丢失）**
 - ✓ **数据不能丢失的应用：**如文件传输、金融事务等
 - ✓ **能容忍数据丢失的应用：**如多媒体应用
- ❑ **带宽（数据传输率）**
 - ✓ **带宽敏感的应用：**需要特定的带宽才能正常工作
如，因特网电话、其他多媒体应用
 - ✓ **弹性应用：**使用的带宽多或少影响不大
如电子邮件、文件传输以及Web传输
- ❑ **定时（数据传输的时间限制）**
 - ✓ **交互式实时应用：**对时间敏感，要求时延小
如，因特网电话、视频会议以及多方游戏等
 - ✓ **非实时应用：**时延无限制，低更好
 - ✓ **安全性（加密解密）**

2.1.4 因特网提供的运输服务 P62

两个运输层协议

- 用户数据报协议UDP
- 传输控制协议TCP
- ✓ 每个协议为调用它们的应用程序提供不同的服务模型
- ✓ 在创建一个新的因特网应用时，要选择其中一个

1、TCP服务

- ✓ 面向连接的服务
- ✓ 可靠的传输服务
- ✓ 拥塞控制

2、UDP服务

提供**最小服务模式**运行

- ✓ **无连接**：两个进程通信前没有握手过程
- ✓ **不可靠数据传输**：不保证报文能够被接收，或收到的报文是乱序到达
- ✓ **没有拥塞控制机制**：发送进程可以任何速率发送数据
- ✓ **不提供时延保证**
- **适于实时应用**

未提供的服务

- ✓ **不确保最小传输速率：** 发送进程受拥塞控制机制制约
- ✓ **不提供时延保证：** 数据传输的时间不确定

**TCP协议能保证交付所有的数据，但并不保证这些数据
数据传输的速率以及期待的传输时延**

TCP协议不适合实时应用

典型应用的运输服务要求

应用程序	数据丢失	带宽	时间敏感
文件传输	不能丢失	弹性	不
电子邮件	不能丢失	弹性	不
Web 文档	不能丢失	弹性	不
实时音频/视频 (因特网电话/视频会议)	容忍丢失	音频: 5kbps-1Mbps 视频: 10kbps-5Mbps	是, 100 ms
存储音频/视频	容忍丢失	同上	是, 几秒
交互式游戏	容忍丢失	几kbps以上	是, 100 ms
即时讯息	不能丢失	弹性	是和不是

2.1.5 应用层协议 P64

- 定义了运行在不同端系统上的应用程序进程如何相互传递报文
- 区分网络应用与应用层协议？（问题）
 - ❖ 应用层协议只是网络应用非常重要的一部分
 - ❖ 例如Web应用组成部分
 - 文档格式的标准（如HTML HyperText Markup Language超文本标记语言）
 - Web浏览器
 - Web服务器
 - 应用层协议（HTTP HyperText Transfer Protocol）

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.4 DNS
- ❑ 2.5 P2P applications
- ❑ 2.6 Video Streaming and Content Distribution Networks
- ❑ 2.7 Socket programming
- ❑ -FTP
- ❑ +Video Streaming and CDN

本节内容

2.2.1 HTTP概况

2.2.2 HTTP连接 (非持续和持续连接)

2.2.3 HTTP报文格式

2.2.4 用户与服务器交互: Cookie

2.2.5 Web缓存

2.2.6 条件GET方法

Web

产生于1994年 Sir Timothy John "Tim" Berners-Lee

- ✓ 改变了人们与工作环境内外的交流方式
- ✓ 提升因特网地位
- ✓ 生活和工作发生变化
- ✓ 方便、快捷得到所需要的信息 (**按需操作**)
- ✓ 任何人在Web上发布信息
- ✓ 超链接和搜索引擎帮助人们浏览Web站点



- ✓ 2004年，英女皇伊丽莎白二世向伯纳斯·李颁发大英帝国爵级司令勋章
- ✓ 2009年4月，他获选为美国国家科学院外籍院士
- ✓ 在2012年夏季奥林匹克运动会开幕典礼上，他获得了“万维网发明者”的美誉

2.2.1 HTTP概况

□ HTTP (超文本传输协议)

应用层协议，Web的核心 RFC 1945 2616

□ 包括两部分

✓ 客户机程序

✓ 服务器程序

分别运行在不同的端系统中，通过交换HTTP报文进行会话

□ HTTP协议定义了报文的格式以及客户机和服务器交换报文的方式

Web常用术语

✓ **URL: Uniform Resource Locator, 统一资源定位符**

标识万维网WWW上的各种文档, 全网范围唯一

✓ **URL地址组成**

存放对象的**服务器主机名**和**对象的路径名**

如

http:// www.someSchool.edu / somedepartment / picture.gif

主机名

路径名

Web常用术语

□ **Web页(文档)**: 由若干**对象**组成

对象: **是文件**, 通过一个**URL地址**来寻址, 如HTML文件、JPEG图形文件、Java小程序等

✓ Web页含有**一个基本的HTML文件**及**几个引用对象**

例如, 一个Web页包含HTML文本和5个JPEG图形文件 (即有6个对象)

✓ 在基本的HTML文件中, **每个对象可由URL来寻址**

超文本标记语言

统一资源定位符

Web常用术语

- ❑ **浏览器（客户机）**：是Web应用的**用户代理**，用于显示所请求的Web页，提供导航功能和配置属性

实现了HTTP协议的客户端

- ❑ **Web服务器**：用于存贮Web对象（由URL寻址）

实现HTTP协议的服务器端

HTTP协议定义了Web客户机(浏览器)如何向Web站点请求Web页，以及服务器如何将Web页传送给客户机

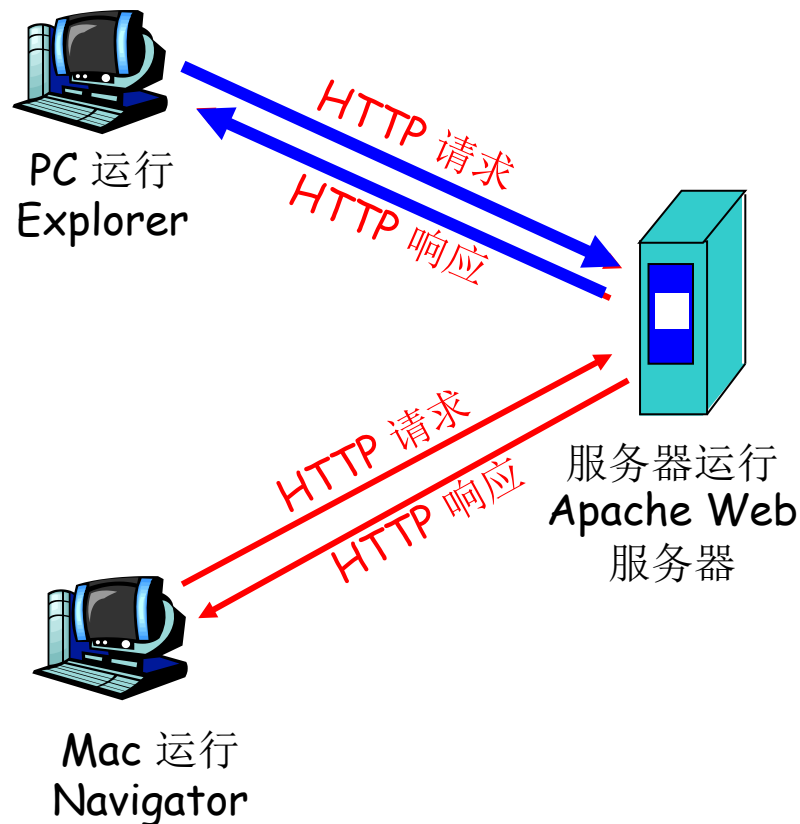
客户机和服务器之间交互过程

□ 客户机

用户请求一个Web页（如点击一个超链接），浏览器向服务器**发出**对该页所含对象的“**HTTP请求报文**”

□ 服务器

接受请求，回发包含请求对象的“**HTTP响应报文**”



说明

□ HTTP协议使用的底层运输协议是TCP

□ 工作过程

创建TCP连接→交换报文→关闭TCP连接

- ✓ 客户机先与服务器建立TCP连接，然后，浏览器和服务进程通过套接字访问TCP
- ✓ 客户机：从其套接字接口发送“HTTP请求报文”和接收“响应报文”
- ✓ 服务器：从其套接字接口接收“HTTP请求报文”和发送“响应报文”

说明

- TCP提供可靠的数据传输服务：客户机进程和服务进程发出的每个HTTP报文能完整地到达对方
- HTTP是无状态协议：服务器不保存关于客户机的任何信息

2.2.2 非持续和持续连接

非持续HTTP连接

- ✓ 每个TCP连接上只传送一个Web对象
- ✓ 只传送一个请求/响应对

持续HTTP连接

- ✓ 一个TCP连接上可以传送多个Web对象
- ✓ 传送多个请求/响应对

默认方式下使用持续连接

1、非持续连接

例，客户机向服务器请求传送一个Web页

- ✓ 含有一个基本HTML文件和10个JPEG图形，11个对象位于同一个服务器上
- ✓ HTML文件的URL为

<http://www.someSchool.edu/someDepartment/home.index>

工作过程

TCP
连接

1a. HTTP客户初始化一个与服务
器主机www.someSchool.edu中
HTTP服务器进程的TCP连接

1b. www.someSchool.edu服务器
主机中的HTTP服务器在80端
口监听来自HTTP客户的TCP
连接请求, 收到连接请求, 接
受, 建立连接, 通知客户

2. HTTP客户发送一个HTTP请
求报文 (包含URL) 到TCP
连接套接字, 报文指明客户
需要的Web对象—
someDepartment/home.index

3. HTTP服务器接收请求报文,
产生一个响应报文 (包含被
请求对象), 并发送到其TCP
连接套接字

报
文
传
输

time

4. HTTP服务器关闭TCP 连接

5. HTTP客户机接收包含HTML
文件的响应报文，显示并解
析HTML文件，发现10个引
用的 jpeg对象

6. 对10个jpeg对象重复步骤1~5

time



请求并接受一个HTML文件所需时间

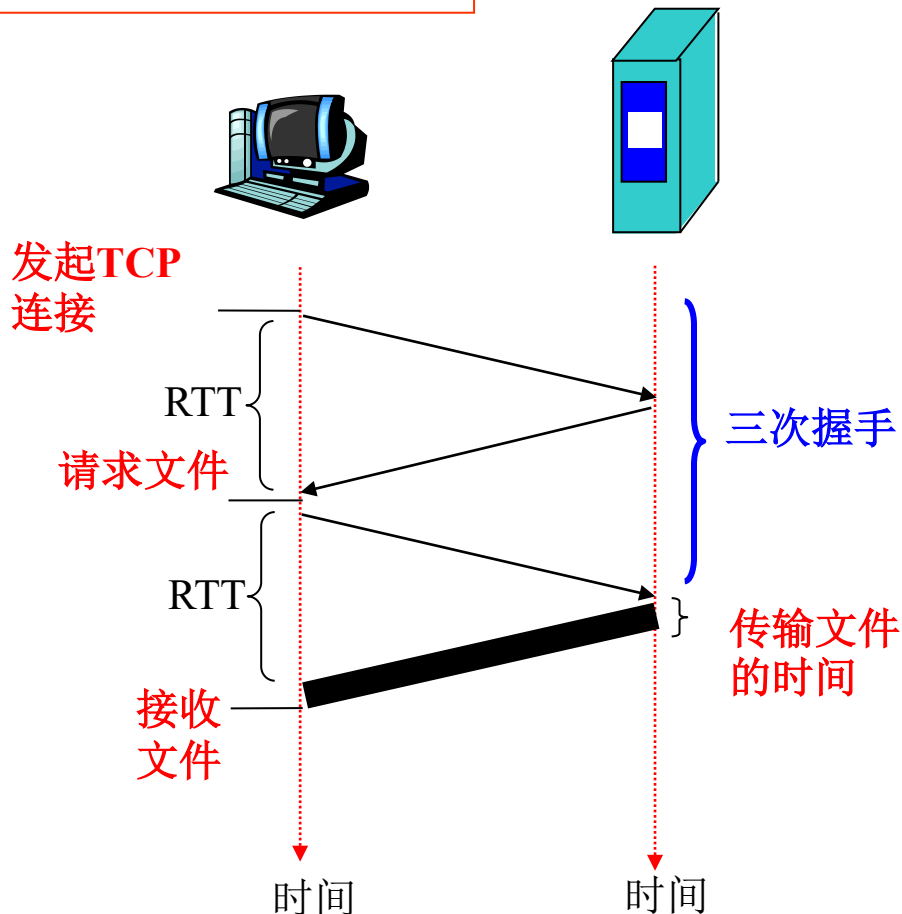
建立TCP连接 → 交换报文

□ TCP连接的“三次握手”过程

- ✓ 客户机发送一个TCP连接请求报文
- ✓ 服务器回送一个TCP确认响应报文
- ✓ 客户机向服务器发送一个包含“HTTP请求”与“TCP确认”的报文

□ 总响应时间：两个RTT时延加上服务器发送文件的时间

总计 = $2RTT + \text{文件传输时间}$
(粗略时间)



非持续连接缺点

- **服务器负担重：** 每一个请求对象建立和维护一个新的连接
- **每一个对象的传输时延长：** 包含两个RTT时延，一个用于TCP建立，一个用于请求和接收对象

2、采用持续连接的HTTP

□ 服务器在发送响应后保持该TCP连接

✓ 相同客户机与服务器之间的后续请求和响应报文通过相同的连接进行传送

如，一个Web页 的所有对象可以通过一个TCP连接传送

✓ 或同一服务器上的多个Web页也可以通过一个TCP连接传送给同一个客户机

□ 连接经一定时间间隔(超时间隔)未被使用，服务器就关闭该连接

持续连接两种方式

- **非流水线方式：**客户机只能在前一个响应接收到之后才能发出新的请求
- ✓ 客户机为每一个引用对象的请求和接收都使用一个RTT时延
- ✓ 会浪费一些服务器资源：服务器在发送完一个对象，等待下一个请求时，会出现空闲状态

持续连接两种方式

□ 流水线方式

- ✓ 客户机可一个接一个**连续产生请求**（只要有引用就产生），即在前一个请求接收到响应之前可以产生新的请求
- ✓ 服务器一个接一个**连续发送相应对象**

□ 特点

- ✓ 节省**RTT时延**，可能所有引用对象**只花费一个**
- ✓ TCP连接空闲时间很短

□ 默认方式：流水线方式的持续连接

方法（命令）——

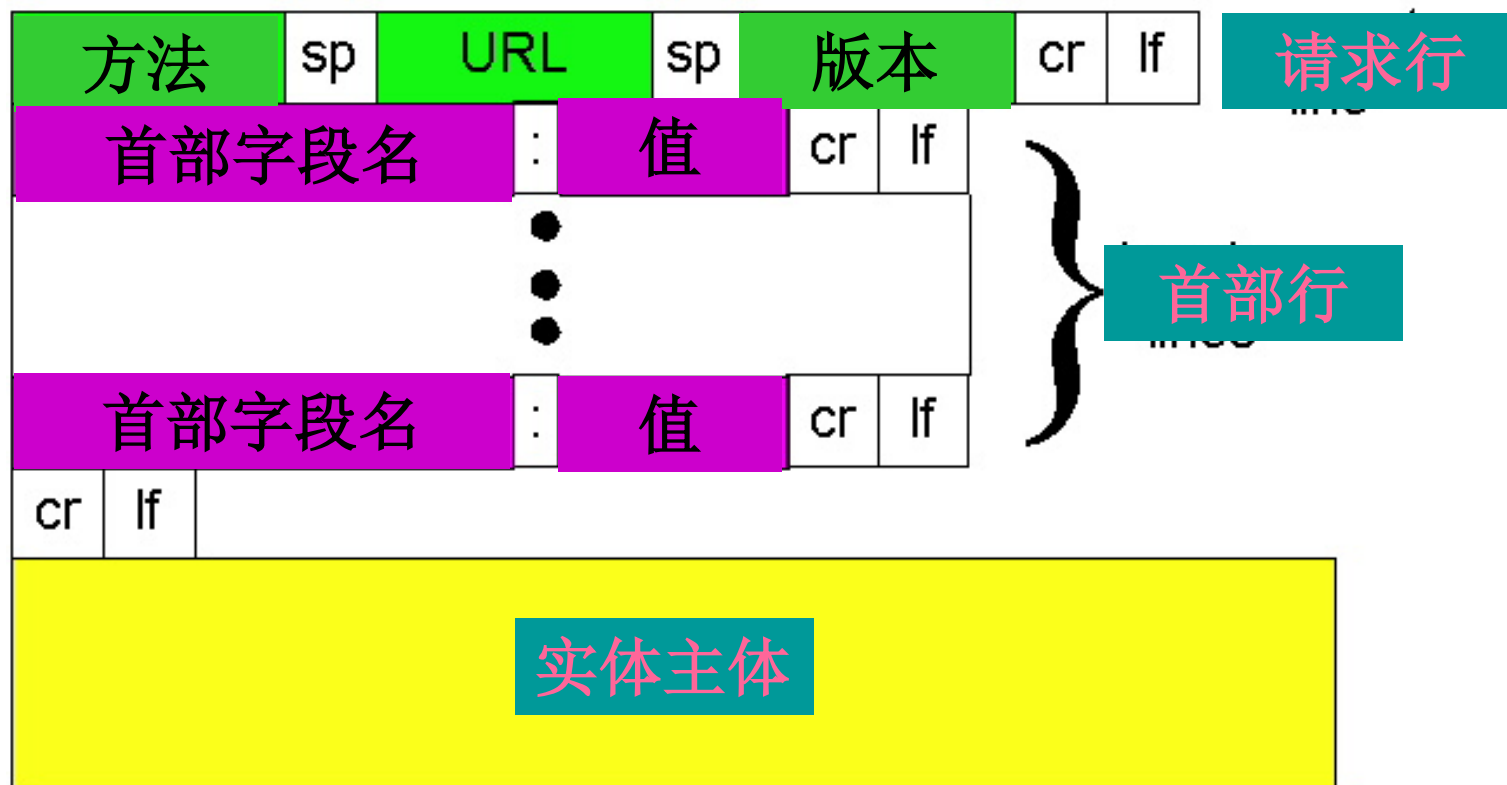
- ✓ GET：请求一个对象。
- ✓ POST：提交表单（添加信息）。
- ✓ HEAD：请求返回对象响应报文首部

报文

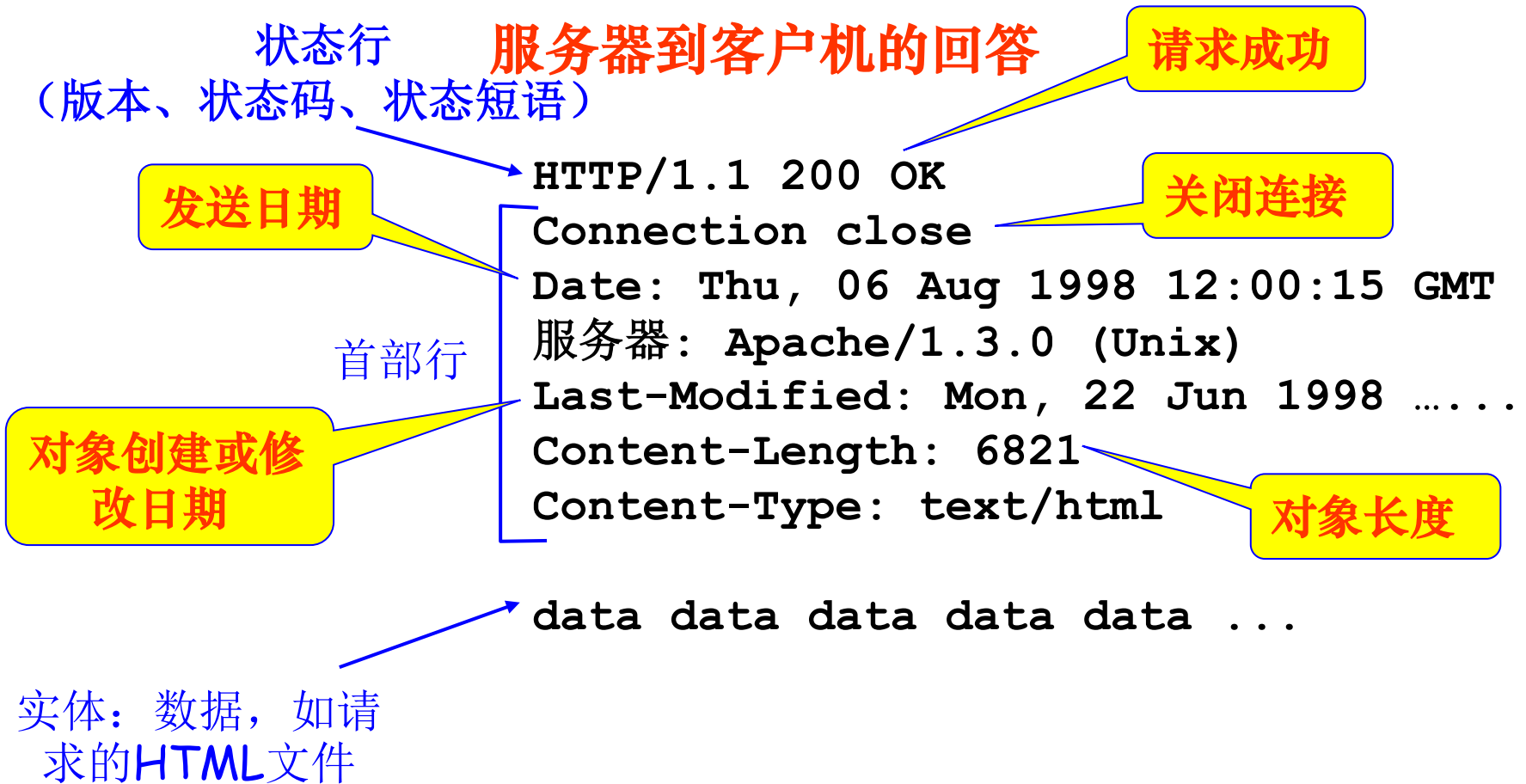
文本形式，易读。



HTTP请求报文通用格式



2、HTTP 响应报文



HTTP响应状态码

□ 在服务器到客户机响应报文中的首行。

200 OK

- 请求成功，请求的对象在这个报文后面

301 Moved Permanently

- 请求的对象已转移，新的URL在响应报文的Location:首部行中指定

400 Bad Request

- 请求报文不为服务器理解

404 Not Found

- 请求的文档没有在该服务器上发现

505 HTTP Version Not Supported

- 服务器不支持请求报文使用的HTTP版本

自行试验HTTP (客户机侧)

1. Telnet 到某个Web服务器上:

`telnet www.baidu.com 80`

打开到位于**www.baidu.com**的端口**80**(默认的**HTTP**服务器端口).
键入的任何东西将发送到位于**www.baidu.com**的**80**端口

2. 回车执行命令, 得到一个黑窗口, 按下ctrl+] 键



自行试验HTTP (客户机侧)

3. 键入一个GET HTTP请求:

GET / HTTP/1.1
Host: www.baidu.com

向HTTP服务器发送最小的GET请求



The screenshot shows a Telnet window titled 'Telnet www.baidu.com'. The prompt is 'C:\>'. The user has entered 'GET / HTTP/1.1' and 'Host:www.baidu.com' on separate lines. Below the input, yellow text provides instructions: '依次输入:' followed by 'GET / HTTP/1.1', '回车' (press Enter), 'Host:IP (或主机名)', and '连续两次回车' (press Enter twice).

```
C:\> GET / HTTP/1.1
Host:www.baidu.com
```

依次输入:
GET / HTTP/1.1
回车
Host:IP (或主机名)
连续两次回车

4. 得到由HTTP服务器发送的响应报文!

<!DOCTYPE html><html><head><meta http-equiv="Content-Type" content="text/html; charset=utf-8"><meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1"><meta content="always" name="referrer"><meta na

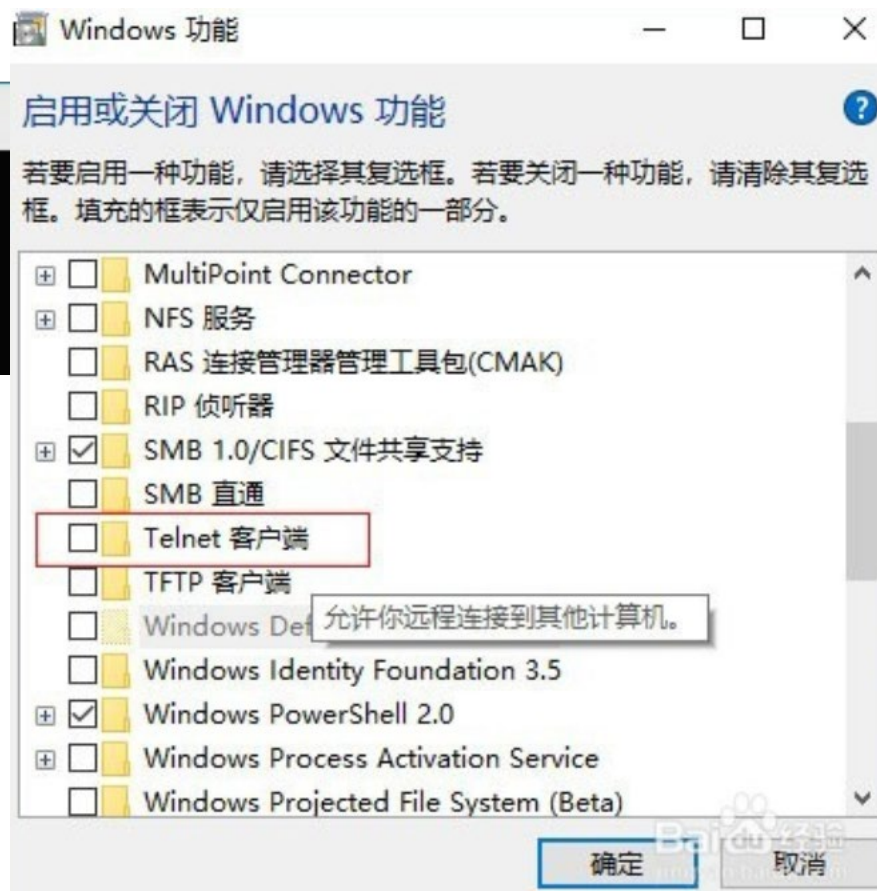
保疏涓塢杓鍤滅儲寮疊捧鉞飲噓鐸說路璁一綉妍曼滄渚拷嶠鏇拌峽鑾柊俊鎮 紆鎵傢垠鎵€€姘傘
鎷鰂悝塞據紆緋 五鏈珍相筵價相繼摺呂鐳數直絀(三)竈錫漣€2、1ink rel="choutout icon"

不能用telnet的解决方案

❑ <https://jingyan.baidu.com/article/0eb457e5e2b37903f0a90576.html>

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.17134.112]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\Administrator>telnet
'telnet' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
```



2.2.4 用户与服务器交互：Cookie

HTTP服务器是**无状态**的，不保存客户信息。

- **Cookie**: 允许Web站点跟踪、识别用户；服务器可以限制用户访问，或把内容与用户身份关联。

许多重要的Web站点使用cookies

- **包括四个部分**

- 1)在HTTP响应报文中有一个cookie 首部行
- 2)在HTTP请求报文中有一个cookie 首部行
- 3)用户主机中保留有一个 cookie 文件并由浏览器管理
- 4) Web站点的后端数据库保存cookie

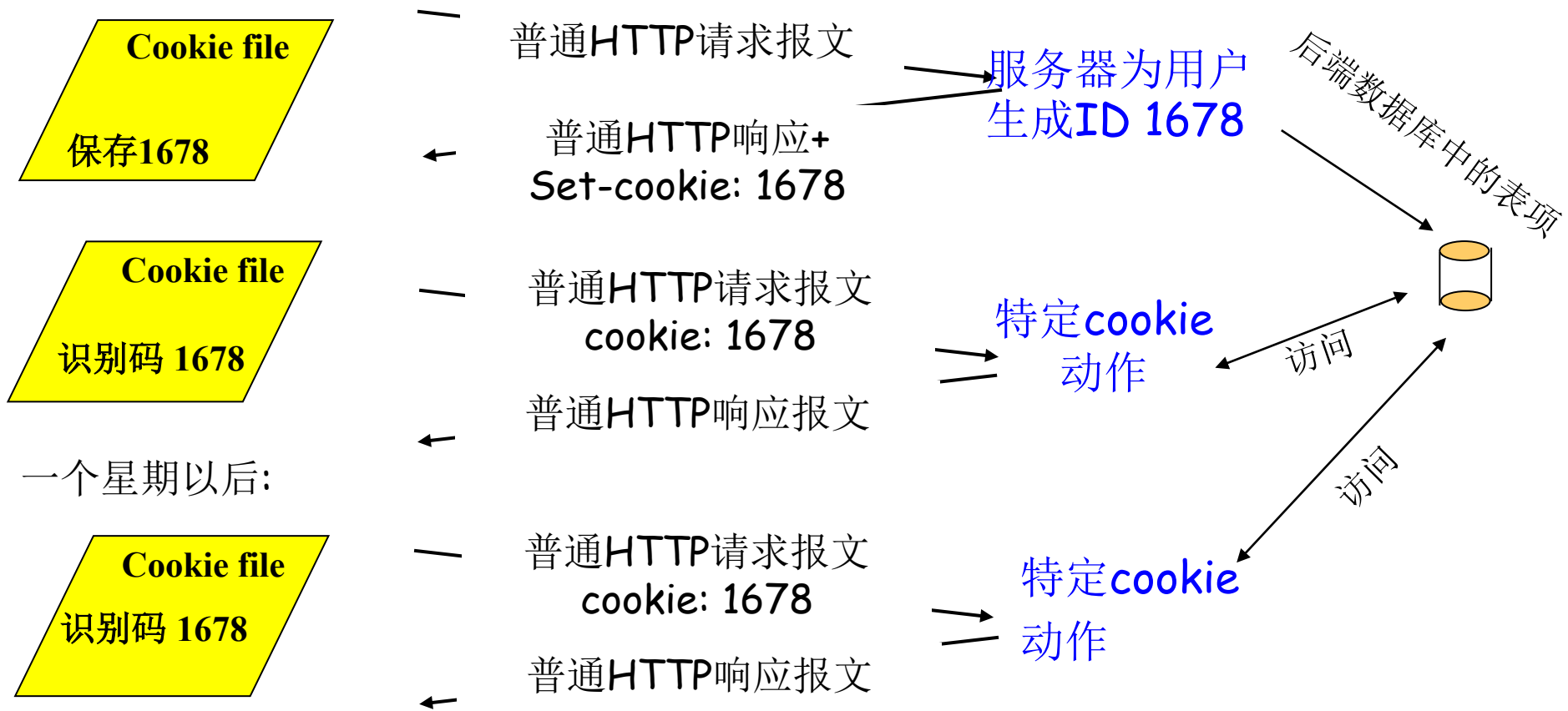
例

- Susan总是从相同的PC访问因特网
- 她首次访问一个特定的电子商务站点
- 当起始HTTP请求到达站点时，站点产生一个独特的ID，并为ID在后端数据库中生成一个表项

工作过程

客户机

服务器



❑ Cookie用途

- ✓ 身份认证
- ✓ 虚拟购物车(跟踪用户购买的物品)
- ✓ 推荐广告
- ✓ 用户会话状态 (Web e-mail)

❑ Cookie缺陷

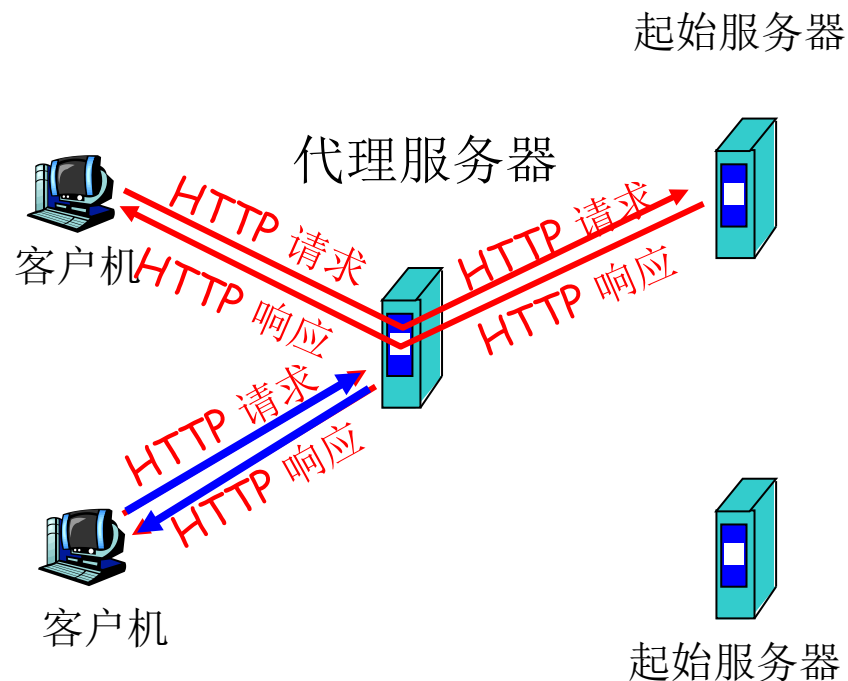
- ✓ 站点可以知道用户许多信息
- ✓ 不利于用户隐私保护

2.2.5 Web缓存

例：假设浏览器请求对象

`http://www.someschool.edu/campus.gif`

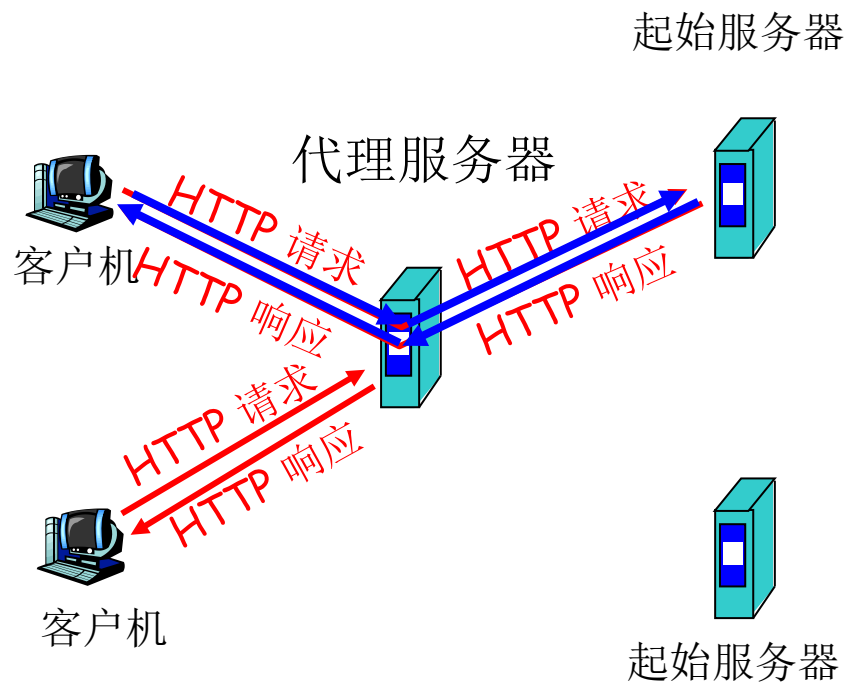
- **浏览器**：建立一个到缓存的TCP连接，并向缓存发送一个对该对象HTTP请求
- **Web缓存器**：检查本地是否有该对象的拷贝
- ✓ **有**：就用HTTP响应报文向浏览器转发该对象



✓ **没有：** 与该对象的起始服务器打开一个TCP连接

客户机 $\longleftrightarrow$ Web缓存器 $\longleftrightarrow$ 起始服务器

- ◆ 缓存在TCP连接上发送获取该对象的请求
- ◆ 起始服务器收到请求，向缓存发送该对象的HTTP响应
- ◆ 缓存接收该对象，存储一份在本地中，并通过HTTP响应报文向浏览器转发该对象（通过已经建立的TCP连接）



说明

- Web缓存器既可以是服务器也可以是客户机
- ✓ 当它接收浏览器请求并发回响应时，是服务器
- ✓ 当它向起始服务器发出请求并接收响应时，是客户机

Web缓存优点

- 减少客户机请求的响应时间

客户机 $\longleftrightarrow$ Web缓存器 $\longleftrightarrow$ 起始服务器

高速链路

- 减少机构内部网络与因特网连接链路上的通信量

降低开销，改善各种应用的性能。

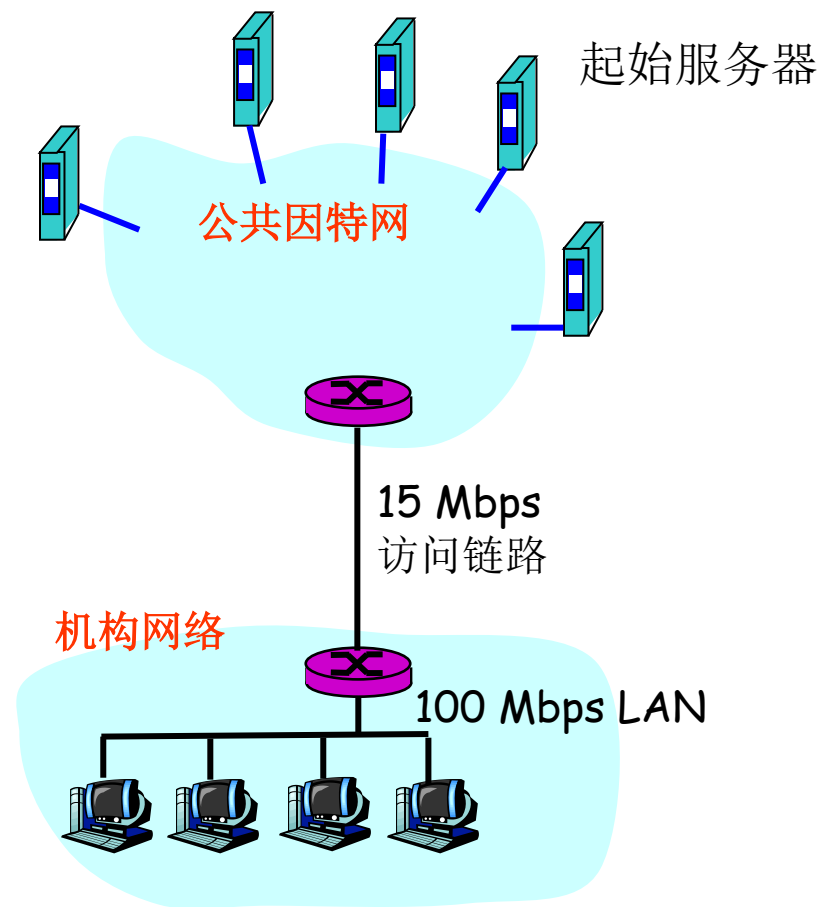
例1, 无Web缓存

□ 包括两部分：**机构的内部网络和因特网**

✓ **机构内部网络**：是一个高速的局域网100Mbps

其路由器与因特网上的路由器通过一条15Mbps的链路连接

✓ **起始服务器**：与因特网相连，遍布全球

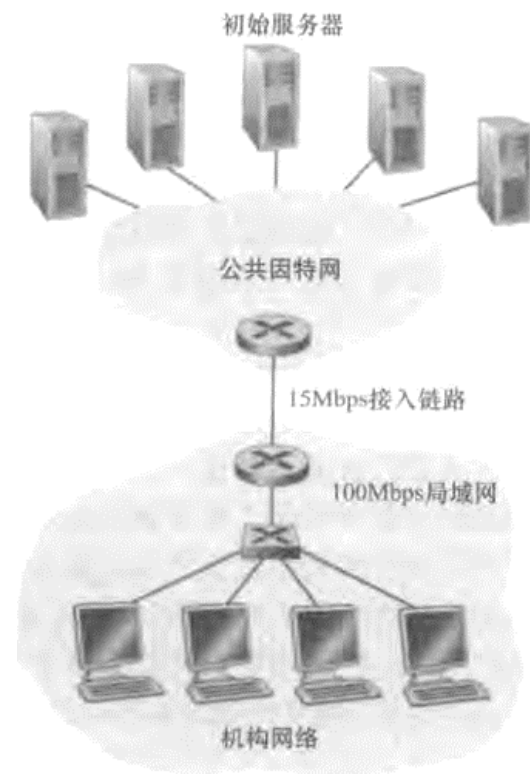


假设

- ✓ 对象平均长度为**1Mb**
- ✓ 机构内浏览器对原始服务器上对象的平均请求率=**15/sec**
- ✓ 从因特网路由器转发HTTP请求报文，到收到其响应报文的时间平均=**2s（因特网时延）**

浏览器从请求一个对象到接收到的时间：
三部分和

- ✓ **局域网时延**
- ✓ **接入链路时延(接入两个路由器间链路)**
- ✓ **因特网时延**



- ✓ **局域网时延**：与流量强度有关（**比特到达率/推出率**）

$$(15\text{请求/s}) \times (1\text{Mb/请求}) / (100\text{Mbit/s}) = 0.15$$

强度为0.15的通信量最多数十毫秒的时延，可**忽略**

- ✓ **接入链路时延**：与接入**流量强度**（路由器之间）有关

$$(15\text{请求/s}) \times (1\text{Mb/请求}) / (15\text{Mbit/s}) = 1$$

强度1，链路**时延非常大**或无限增长。

- ✓ **总响应时间** = 接入链路时延 + 因特网时延

$$= (\text{分钟} + 2 \text{ sec})$$

请求时间长，用户难接受

改进方法一

- ✓ 增加接入链路的速率

如从15Mbps增加到100Mbps，使链路上的流量强度减少到0.15，链路时延也可以忽略了

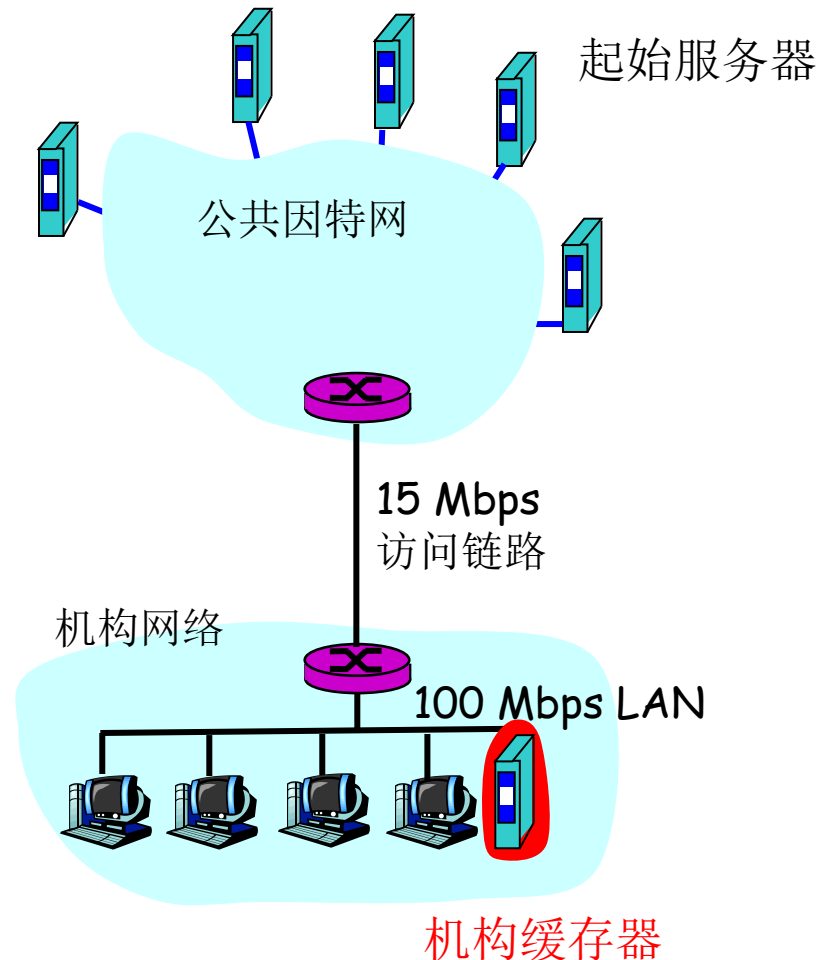
总响应时间约等于因特网时延=2秒钟

- ✓ 投资较大，成本昂贵

改进方法二

在机构网络中安装一个Web缓存器

- ✓ Web缓存器的命中率：缓存器满足请求的比率 (0.2~0.7)
- ✓ 设命中率为0.4

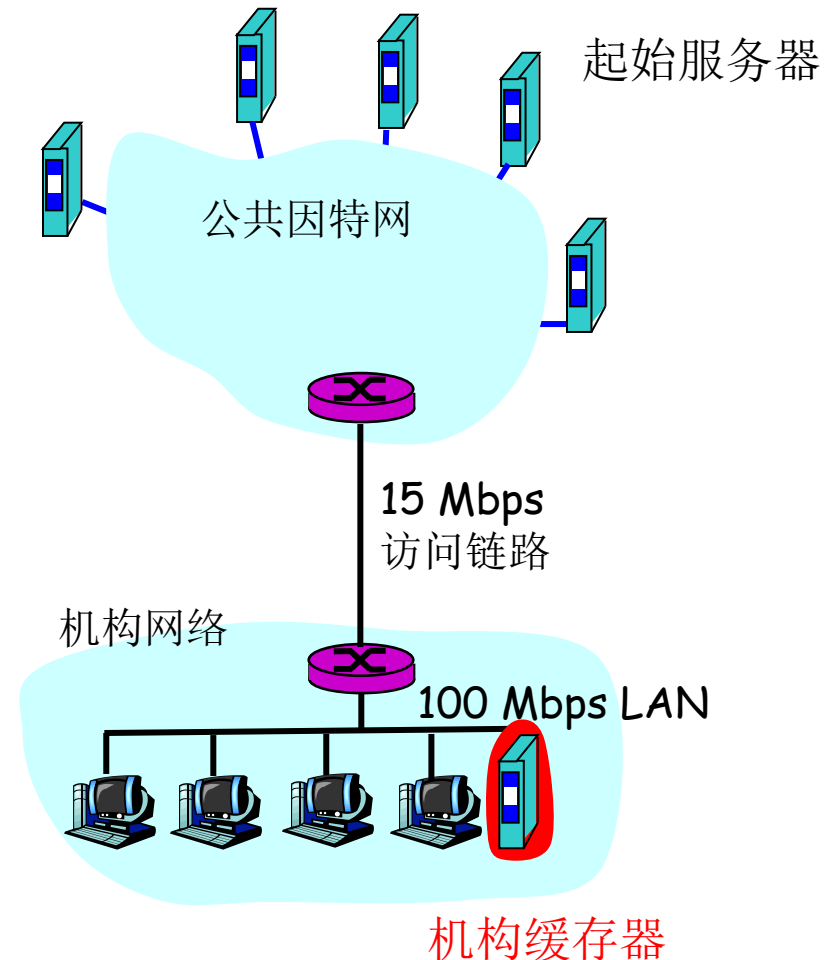


改进方法二

1Mb/100Mbps

- ✓ **局域网时延：** 客户机缓存器位于同一局域网，40%的请求几乎会立即得到响应，时延约10ms

剩下的60%请求需要通过访问起始服务器才能满足



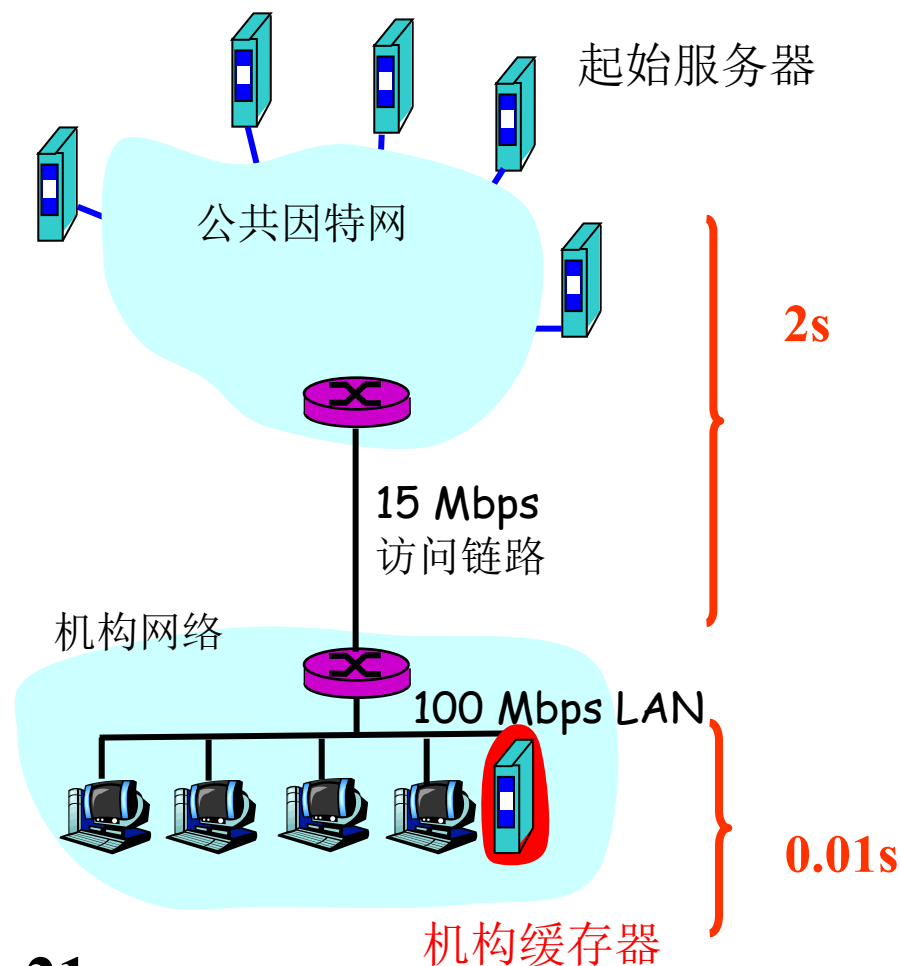
✓ 接入链路时延

只有60%的请求对象通过接入链路传送，流量强度从1.0减小到0.6

通常，在15Mbps链路上，当流量强度小于0.8时，时延很小，可忽略

✓ 平均时延约为

$$0.4 \times (0.01s) + 0.6 \times (0.01s + 2s) = 1.21s$$



Web缓存器减少响应时延，成本低

2.2.7 条件GET方法

□ 高速缓存

- ✓ 减少响应时间
- ✓ 存放在缓存中的对象拷贝可能是旧的，即保存在起始Web服务器中的对象可能已经被修改

□ 条件GET方法

- ✓ 使缓存器能够证实其保存的对象是否为最新
- ✓ 如果缓存中是最新对象版本，可继续使用，起始Web服务器就不需重新发送该对象

条件GET方法使用

- Web服务器回发**响应报文**：包括对象的最后修改时间
Last-modified: date1
- 缓存检查Web服务器中的该对象是否已被修改，发送一个**条件GET请求报文**
If-modified-since: date1
- ✓ 告诉服务器，仅当**自指定日期之后该对象被修改过**，才发送该对象
- ✓ 若Web服务器中的该对象未被修改，则响应报文含有**304 Not Modified**，并且实体为空

例

缓存器

服务器

缓存器将对象转发到浏览器，
并保存对象到本地（包括对象的
最后修改时间）。

GET /fruit/kiwi.gif HTTP/1.1
Host:www.exotiquecuisine.com

HTTP/1.0 200 OK
Last-modified: date1
<data>

一周后，用户再次请求该
对象（仍保留在缓存中），
缓存发送一个条件GET，
检查该对象是否已被修改

GET /fruit/kiwi.gif HTTP/1.1
Host:www.exotiquecuisine.com
If-modified-since: date1

304 Not Modified
实体为空

对象未修改

对象未修改，缓存可以继续使用
该对象的拷贝，并转发给用户
浏览器。

谢谢大家！

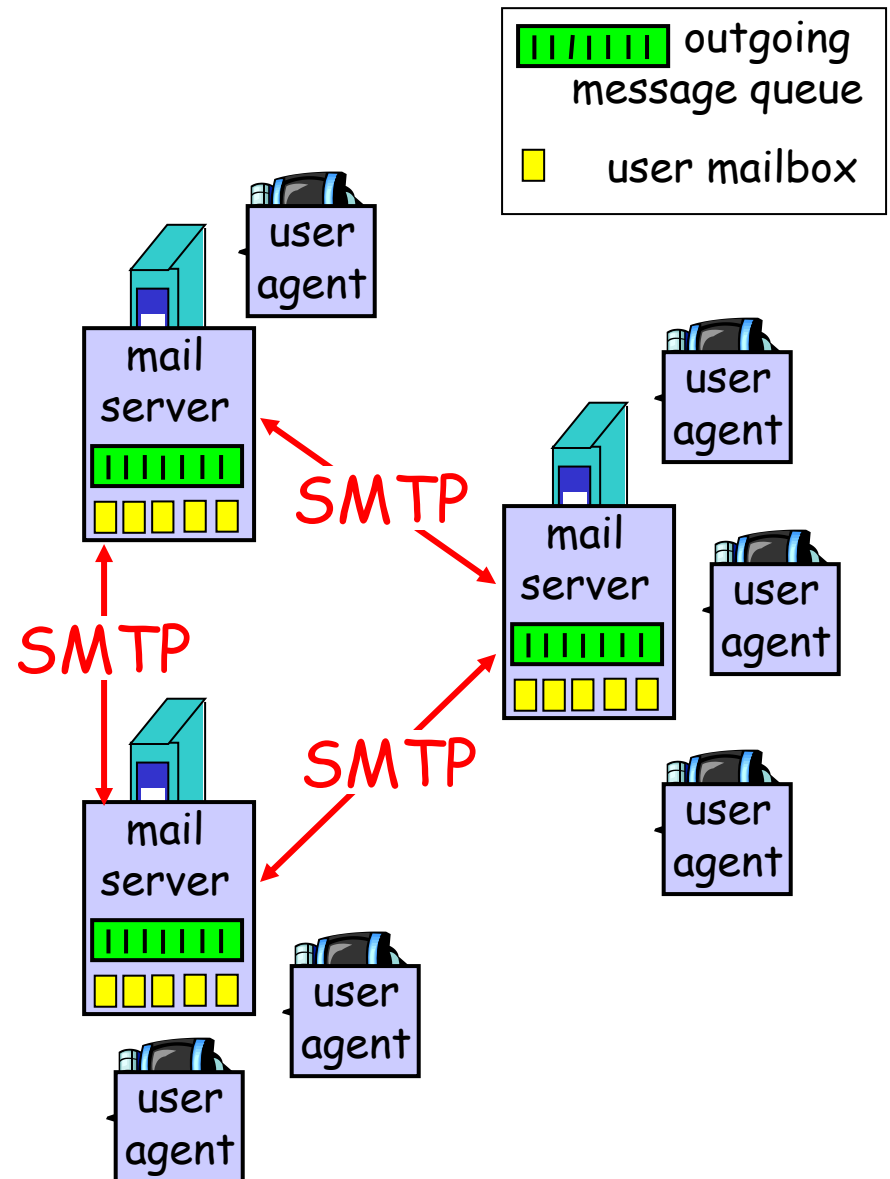
Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.4 DNS
- ❑ 2.5 P2P applications
- ❑ 2.6 Video Streaming and Content Distribution Networks
- ❑ 2.7 Socket programming
- ❑ -FTP
- ❑ +Video Streaming and CDN

Electronic Mail

Three major components:

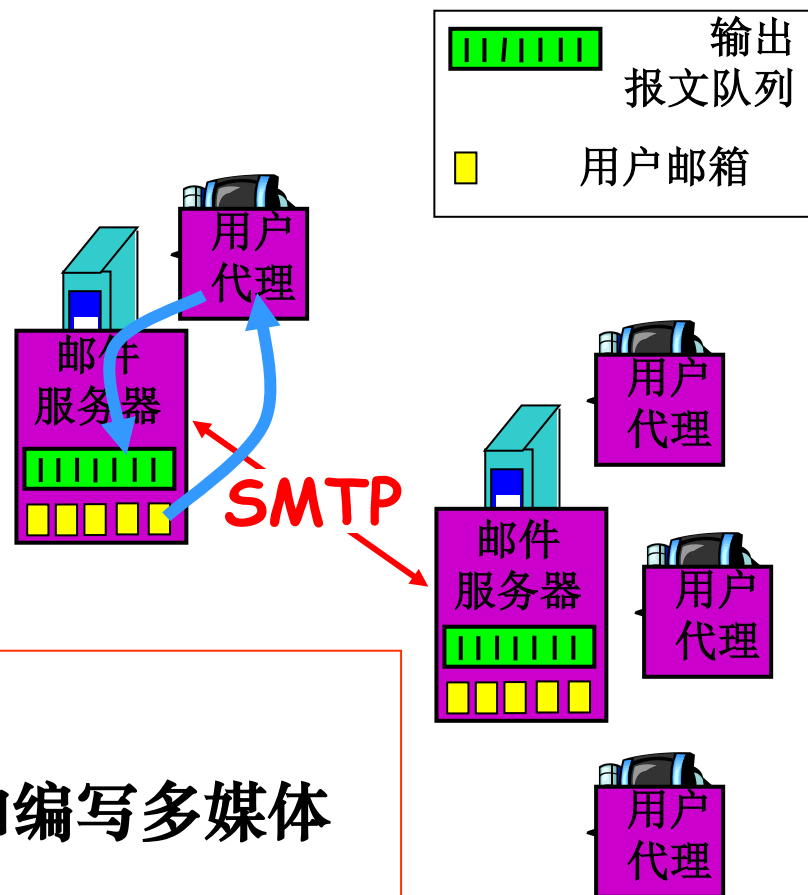
- user agents
- mail servers
- simple mail transfer protocol: SMTP



1、用户代理(user agent)

邮件阅读器，允许用户阅读、回复、发送、保存和撰写报文

- ✓ 当用户完成邮件撰写时，邮件代理向其邮件服务器**发送**邮件，并存放在发送队列中
- ✓ 当用户想读取一条报文时，邮件代理从其邮件服务器的邮箱中**获取**该报文



□ 种类

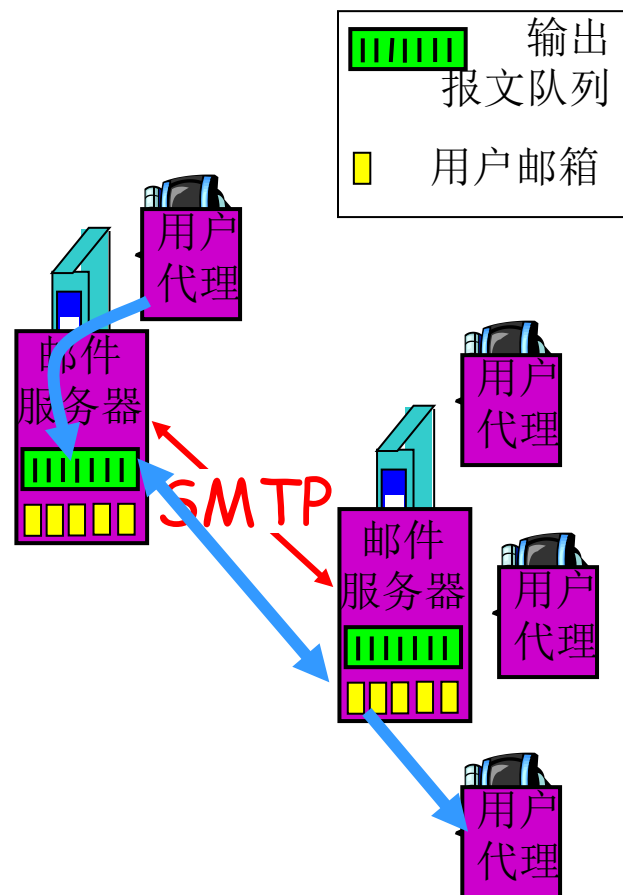
- ✓ GUI（图形用户接口）：阅读和编写多媒体邮件，如Outlook、Foxmail等

2、邮件服务器(mail server)

- ✓ **邮箱**：发送给用户的报文
- ✓ **报文队列**：用户要发出的邮件报文

□ 邮件发送主要过程

- ✓ 邮件**保存**到发送方报文队列
- ✓ 通过**SMTP协议转发**到接收方邮件服务器，保存到相应邮箱中
- 若投递失败，发送方将其保存在一个报文队列中，以后每30分钟发送一次，若几天后仍未成功，将该报文删除，并通知发送方
- 用户访问自己邮箱时，邮件服务器对其身份进行验证(用户名和口令)



3、简单邮件传送协议SMTP(2.3.1)

从发送方的邮件服务器向接收方的邮件服务器发送邮件

✓ 使用TCP可靠数据传输服务

□ 包括两部分

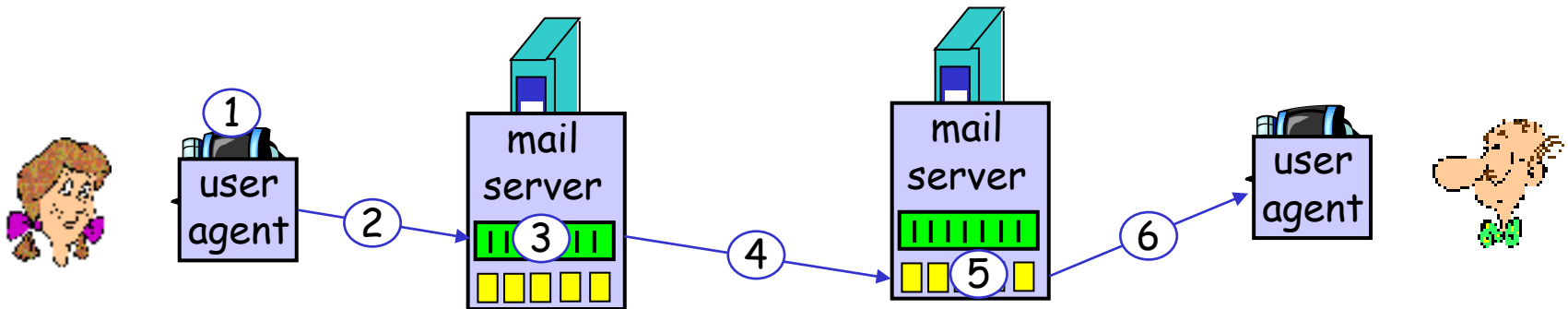
✓ 客户机端：在发送方邮件服务器上运行

✓ 服务器端：在接收方邮件服务器上运行

每个邮件服务器上都有SMTP的客户机端和服务端

Scenario: Alice sends message to Bob

- 1) Alice uses UA to compose message and "to"
`bob@someschool.edu`
- 2) Alice's UA sends message to her mail server; message placed in message queue
- 3) Client side of SMTP opens TCP connection with Bob's mail server
- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message



□ SMTP不使用中间邮件服务器发送邮件，即
TCP 连接是从发送方到接收方的直接相连

如果接收方的邮件服务器没有开机，该邮件仍保留在发送方邮件服务器上，并在以后进行再次传送，邮件不会在某个中间邮件服务器停留。

Try SMTP interaction for yourself

- ❑ `telnet servername 25`
- ❑ see 220 reply from server
- ❑ enter HELO, MAIL FROM, RCPT TO, DATA, QUIT commands

above lets you send email without using email client (reader)

练习（telnet发邮件）例子

telnet smtp.qq.com 25

开始输入

helo xiaoyu (随意一个名字)

auth login

eGlhb3l1（你信箱加密后的用户名）

eGlhb3l1（你信箱加密后的授权码）

mail from:<76126128@qq.com>

rcpt to:<8454051@qq.com>

data

From:76126128@qq.com

To:8454051@qq.com

Subject:Hello SMTP

Hello World!

.(最后这个小数点一定要加上)

通过Telnet连接QQ邮箱服务器发送电子邮件

<https://blog.csdn.net/ttwlqqj/article/details/111147421>

可以用这个网站在线加密

<http://tool.chinaz.com/Tools/base64.aspx>



```
选择C:\WINDOWS\system32\cmd.exe
220 newxmesmtplgiccsvrsc8.qq.com XMail Esmtpp QQ Mail Server.
HELO localhost250-newxmesmtplgiccsvrsc8.qq.com-9.46.31.207-33239066
250-SIZE 73400320
250 OK
AUTH LOGIN334 VXN1cm5hbWU6
bQ==334 UGFzc3dvcmQ6
bWU6JiZg==235 Authentication successful
MAIL FROM:<76126128@qq.com>250 OK.
RCPT TO:<8454051@qq.com>250 OK
DATA354 End data with <CR><LF>.<CR><LF>.
SUBJECT:Hello SMTP

Hello World!

.
250 OK: queued as.
quit
221 Bye.

遗失对主机的连接。
```

<https://blog.csdn.net/ttwlqqj>

2.3.2 HTTP和SMTP

- ✓ 都用于从一台主机向另一台主机传送文件
 - ◆ **HTTP**主要用于从Web服务器向Web客户机(浏览器)传送文件 (对象)
 - ◆ **SMTP**用于从一个邮件服务器向另一个邮件服务器传送文件(电子邮件报文)
- ✓ 持续HTTP和SMTP都使用持续连接

区别

□ **HTTP主要是拉协议：**用户使用HTTP从服务器**拉取**信息，其TCP连接是由想获取文件的机器发起

SMTP是推协议：发送邮件服务器把文件**推向接收邮件服务器**，其TCP连接是由要发送文件的机器发起。

区别

- **SMTP使用7位ASCII码格式** American Standard Code for Information Interchange 美国信息交换标准代码

对一些包含了**非7位ASCII字符**的报文或二进制数据(如图片、声音), **需要按照7位ASCII码进行编码, 再传送**

在接收方需要解码还原为原有报文

HTTP数据没有该限制

- **对含有文本和图形 (或其他媒体类型)的文档**
- ✓ HTTP把**每个对象封装**在它各自的HTTP响应报文中发送
- ✓ 电子邮件则把**所有报文对象放在一个报文中**

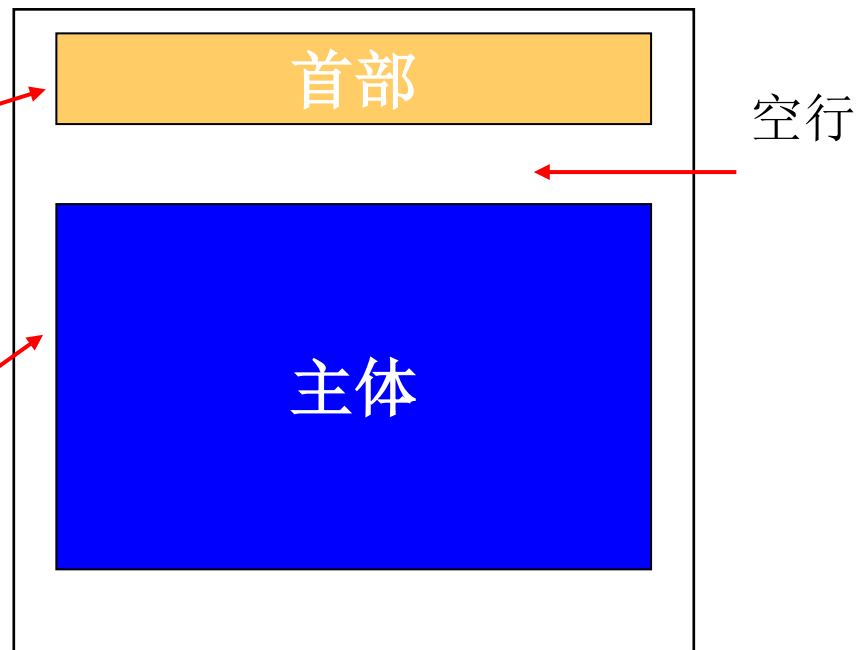
2.3.3 邮件报文格式

□ 首部行 (收发人、主题)

- From: alice@asu.edu
- To: bob@qq.com
- Subject: A test

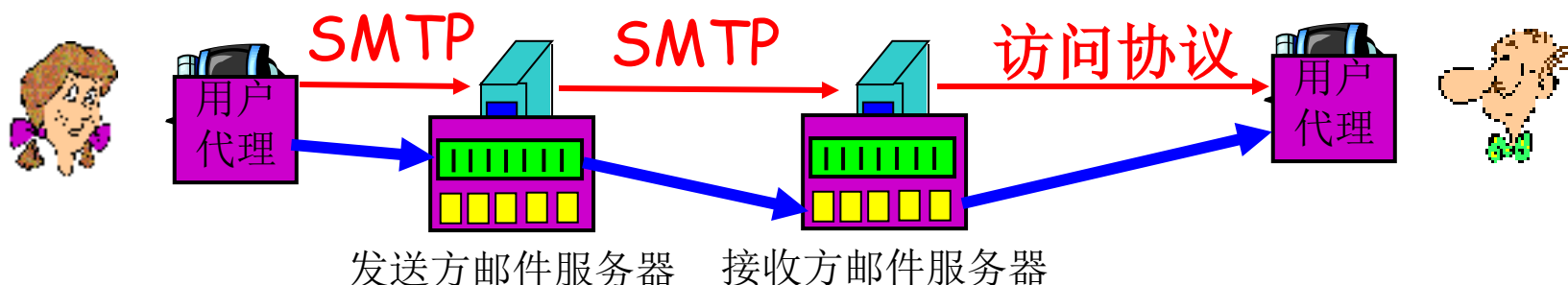
□ 主体

- “报文”, 均为ASCII字符

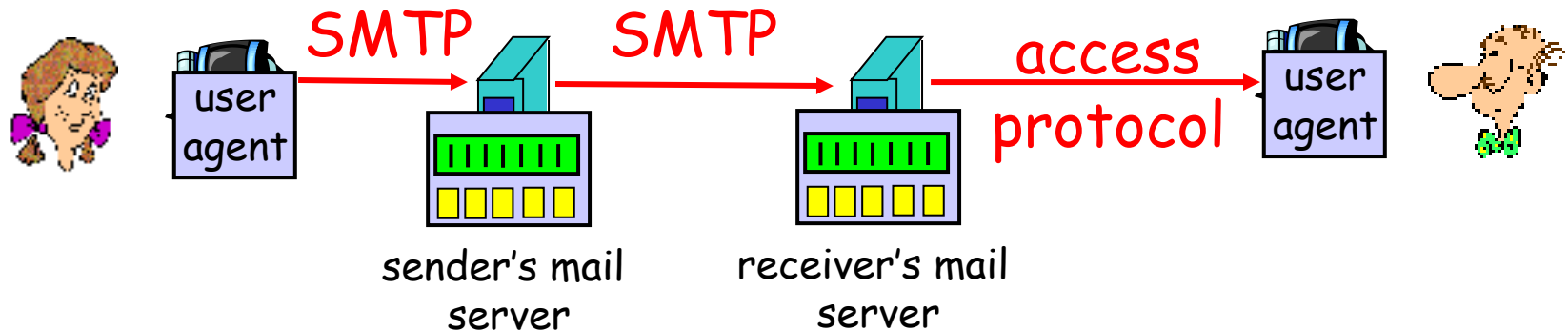


2.3.4 邮件访问协议

- **发送方：** 用户代理用**SMTP**将邮件推入其邮件服务器
→ 邮件服务器再用**SMTP**将邮件转发到接收方的邮件服务器
- **接收方：** 通过其用户代理使用一个**邮件访问协议**（不是SMTP），从其邮件服务器上取回邮件
取邮件是一个拉操作，而SMTP协议是一个推协议。



Mail access protocols



- ❑ SMTP: delivery/storage to receiver's server
- ❑ Mail access protocol: retrieval from server
 - ❖ POP: Post Office Protocol [RFC 1939]
 - authorization (agent <-->server) and download
 - ❖ IMAP: Internet Mail Access Protocol [RFC 1730]
 - more features (more complex)
 - manipulation of stored msgs on server
 - ❖ HTTP: gmail, Hotmail, Yahoo! Mail, qq, etc.

1、POP3

简单、功能有限

在用户代理打开了一个到邮件服务器(服务器)端口110上的TCP连接后，开始工作

工作步骤（三阶段）

- ✓ **特许阶段：** 用户代理以明文方式发送用户名和口令获得下载邮件的特许（**身份认证**）
- ✓ **事务处理阶段：** 用户代理取回报文，可对邮件进行某些操作
如做删除标记、取消删除标记、获取统计信息等
- ✓ **更新阶段：** 邮件服务器删除带有删除标记的报文，结束POP会话

2、IMAP

□ POP3：功能简单

□ IMAP：功能强

- ✓ 在用户的PC机上运行IMAP客户程序，然后与ISP的邮件服务器上的IMAP服务器程序建立TCP连接
- ✓ 用户在自己的PC机上就可以操纵邮件服务器的邮箱，就像在本地操纵一样，是一个联机协议；如：在服务器创建和维护文件夹
- ✓ 实现起来复杂

3、基于web的电子邮件

1995年12月 Hotmail 引入该技术，用户使用浏览器收发电子邮件

□ 用户代理是普通的浏览器，用户和其远程邮箱之间的通信通过HTTP进行

✓ 发件人使用HTTP 将电子邮件报文从其浏览器发送到其邮件服务器上

✓ 收件人使用HTTP从其邮箱中取一个报文到浏览器

□ 邮件服务器之间发送邮件时，使用SMTP

□ 用户可以在远程服务器上以层次目录方式组织报文
如，Hotmail、Yahoo、Mail等

Hotmail

- ❑ 萨比尔-巴迪亚 (Sabeer Bhatia) Hotmail的创始人，毕业于斯坦福大学。加入苹果公司后，为方便与同事联系但又防止被上司发现，巴迪亚萌发了建立一种新型的互通模式
- ❑ Hotmail全世界第一个免费的电子邮件服务商诞生了，而巴迪亚也因此在年仅31岁时就奠定了的事业基础，在当时的IT领域极具影响，个人财产达到了1亿美元
- ❑ 1996年3月，HOTMAIL公司在国际互联网上注册了网址域名
- ❑ 1996年7月开始在国际互联网上提供免费网上电子邮件服务
- ❑ 1997年12月31日，微软公司完成了与成立不足两年、员工仅有26人、主导业务仅为提供免费邮件业务的小公司--HOTMAIL公司的购并事宜（4亿美元。）

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.4 DNS
- ❑ 2.5 P2P applications
- ❑ 2.6 Video Streaming and Content Distribution Networks
- ❑ 2.7 Socket programming
- ❑ -FTP
- ❑ +Video Streaming and CDN

DNS: Domain Name System

People: many identifiers:

- ❖ SSN, name, passport #

Internet hosts, routers:

- ❖ IP address (32 bit) - used for addressing datagrams
- ❖ "name", e.g., www.yahoo.com - used by humans

Q: map between IP addresses and name ?

Domain Name System:

- ❑ *distributed database*
implemented in hierarchy of many *name servers*
- ❑ *application-layer protocol*
host, routers, name servers to communicate to *resolve* names (address/name translation)
 - ❖ note: core Internet function, implemented as application-layer protocol
 - ❖ complexity at network's "edge"

说明

- ✓ DNS协议运行在UDP之上，使用53号端口
- ✓ DNS通常由其他的应用层协议 (包括HTTP、SMTP 和FTP)使用，以将用户提供的主机名解析为IP地址
- ✓ 用户只是间接使用
- ✓ 问题：为什么用UDP？

- ❑ UDP比起TCP效率更高 但是不能保证数据的完整性
- ❑ 解析DNS用UDP 主要是考虑到降低服务器的负载 降低网络流量
- ❑ 并且DNS服务器的网络通常很稳定，万一失败重试的损失也不大

□ 例，某个用户主机上的一个浏览器访问某个Web页
www.someschool.edu/index.html

要将一个HTTP请求报文发送到Web服务器
www.someschool.edu，需先得到相应的IP地址

□ 过程

- ✓ 用户主机上运行着DNS的客户机端
- ✓ 浏览器从URL中解析出主机名，传给DNS客户机端
- ✓ DNS客户机向DNS服务器发送一个包含主机名的请求
- ✓ DNS客户机收到含有对应主机名的IP地址的回答报文
- ✓ 浏览器向该IP地址指定的HTTP服务器发起一个TCP连接

增加一定时延

DNS服务

❑ 主机名到IP地址的转换

❑ 主机别名

- 规范名和别名：通过DNS可以得到主机别名对应的规范主机名及IP地址

❑ 邮件服务器别名

❑ 负载分配

- 繁忙站点冗余地部署在多台服务器上（多个IP地址）
- 也就是一个主机名对应多个IP地址
- DNS在这些IP循环分配

```
C:\Users\Min>tracert www.yahoo.com
```

```
通过最多 30 个跃点跟踪
```

```
到 atsv2-fp-shed.wgl.b.yahoo.com [124.108.103.103] 的路由
```

```
C:\Users\Min>ipconfig /all
```

无线局域网适配器 WLAN:

```
连接特定的 DNS 后缀 . . . . . :
描述. . . . . : Intel(R) Dual Band Wireless-AC 8265
物理地址. . . . . : F4-96-34-CE-D0-29
DHCP 已启用 . . . . . : 是
自动配置已启用. . . . . : 是
本地链接 IPv6 地址. . . . . : fe80::781e:7e2a:5700:8aa8%12(首选)
IPv4 地址 . . . . . : 192.168.1.7(首选)
子网掩码 . . . . . : 255.255.255.0
获得租约的时间 . . . . . : 2020年3月3日 20:13:46
租约过期的时间 . . . . . : 2020年3月5日 9:55:16
默认网关. . . . . : fe80::1%12
                      192.168.1.1
DHCP 服务器 . . . . . : 192.168.1.1
DHCPv6 IAID . . . . . : 66360884
DHCPv6 客户端 DUID . . . . . : 00-01-00-01-21-81-E3-33-54-E1-AD-B6-C3-19
DNS 服务器 . . . . . : fe80::1%12
                      221.11.132.2
TCPIP 上的 NetBIOS . . . . . : 已启用
```

自设DNS

控制面板 > 网络和 Internet > 网络连接

禁用此网络设备 诊断这个连接 重命名

WLAN
Hi-Cable-7-
Intel(R) Dual

发出
VPN CS Support S...

WLAN 属性

网络 共享

连接时使用:

Intel(R) Dual Band Wireless-AC 8265

此连接使用下列项目(O):

- ☒ Microsoft 网络客户端
- ☒ Microsoft 网络的文件和打印机共享
- ☒ QoS 数据包计划程序
- ☒ Internet 协议版本 4 (TCP/IPv4)
- ☐ Microsoft 网络适配器多路传送器协议
- ☒ Microsoft LLDP 协议驱动程序
- ☒ Internet 协议版本 6 (TCP/IPv6)
- ☒ 链路层拓扑发现响应程序

安装(N)...

卸载(U)

描述

传输控制协议/Internet 协议。该协议是默认的用于在不同的相互连接的网络上通信。

Internet 协议版本 4 (TCP/IPv4) 属性

常规 备用配置

如果网络支持此功能，则可以获取自动指派的 IP 设置。否则，你需要从网络系统管理员处获得适当的 IP 设置。

☒ 自动获得 IP 地址(O)

☐ 使用下面的 IP 地址(S):

IP 地址(I):

子网掩码(U):

默认网关(D):

☒ 自动获得 DNS 服务器地址(B)

☐ 使用下面的 DNS 服务器地址(E):

首选 DNS 服务器(P):

备用 DNS 服务器(A):

☐ 退出时验证设置(L)

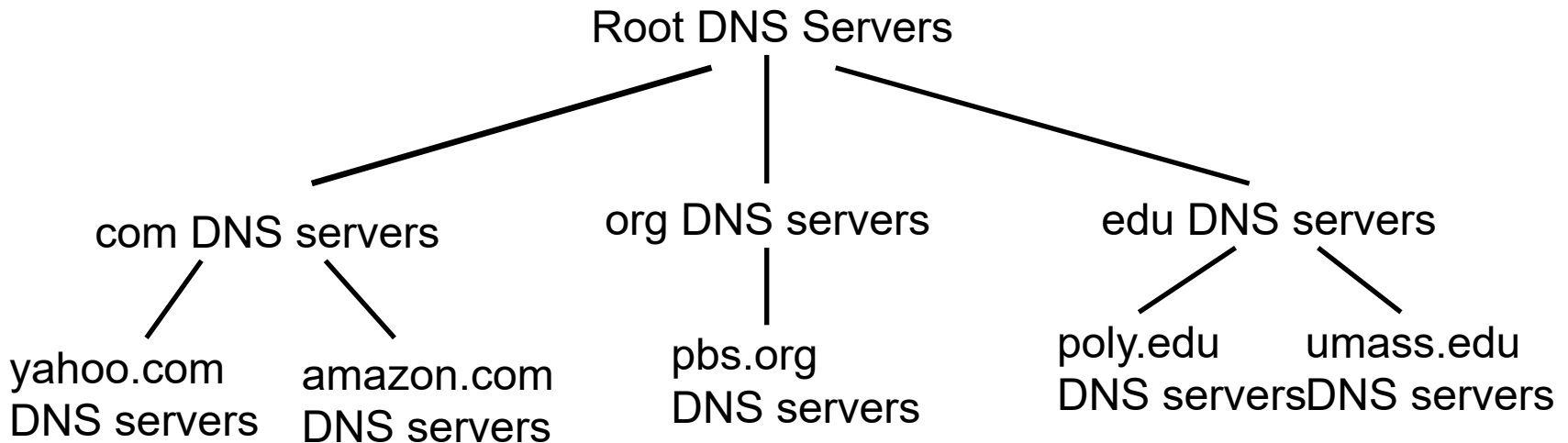
高级(V)...

确定

确定

取消

Distributed, Hierarchical Database

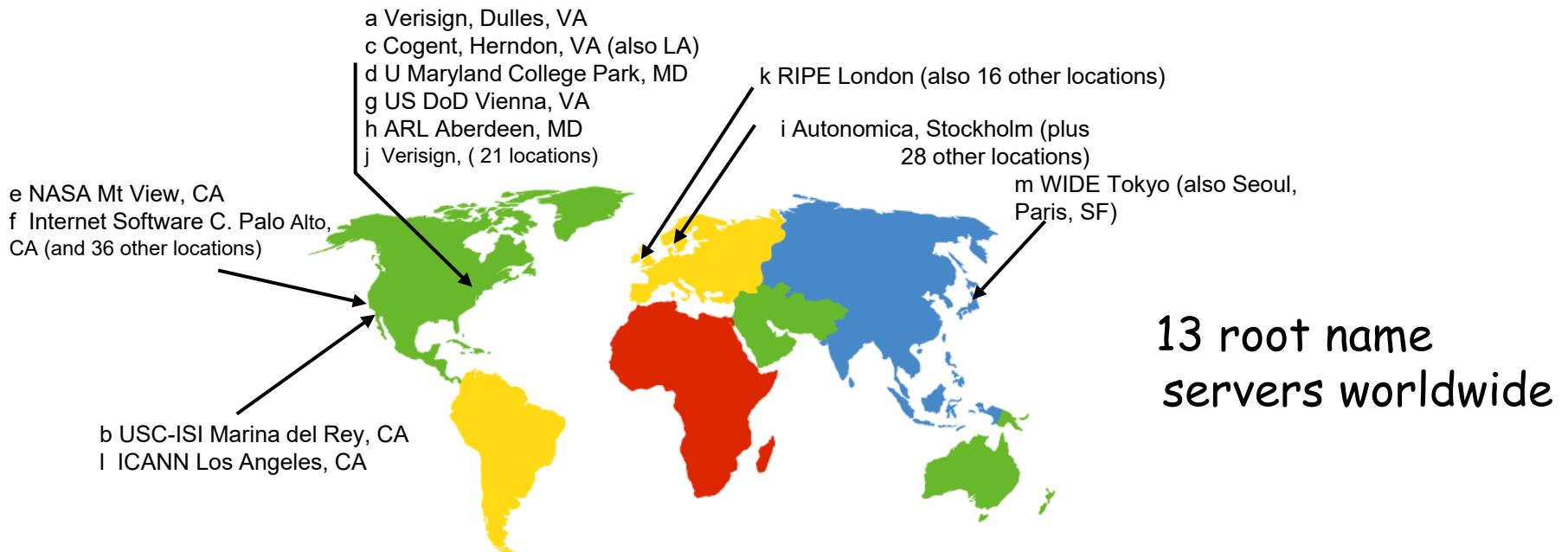


Client wants IP for www.amazon.com; 1st approx:

- ❑ client queries a root server to find com DNS server
- ❑ client queries com DNS server to get amazon.com DNS server
- ❑ client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: Root name servers

- ❑ contacted by local name server that cannot resolve name
- ❑ root name server:
 - ❖ contacts authoritative name server if name mapping not known
 - ❖ gets mapping
 - ❖ returns mapping to local name server



TLD and Authoritative Servers

□ Top-level domain (TLD) servers:

- ❖ responsible for com, org, net, edu, etc, and all top-level country domains uk, fr, ca, jp.
- ❖ Network Solutions maintains servers for com TLD
- ❖ Educause for edu TLD

□ Authoritative DNS servers:

- ❖ organization's DNS servers, providing authoritative hostname to IP mappings for organization's servers (e.g., Web, mail).
- ❖ can be maintained by organization or service provider

Local Name Server

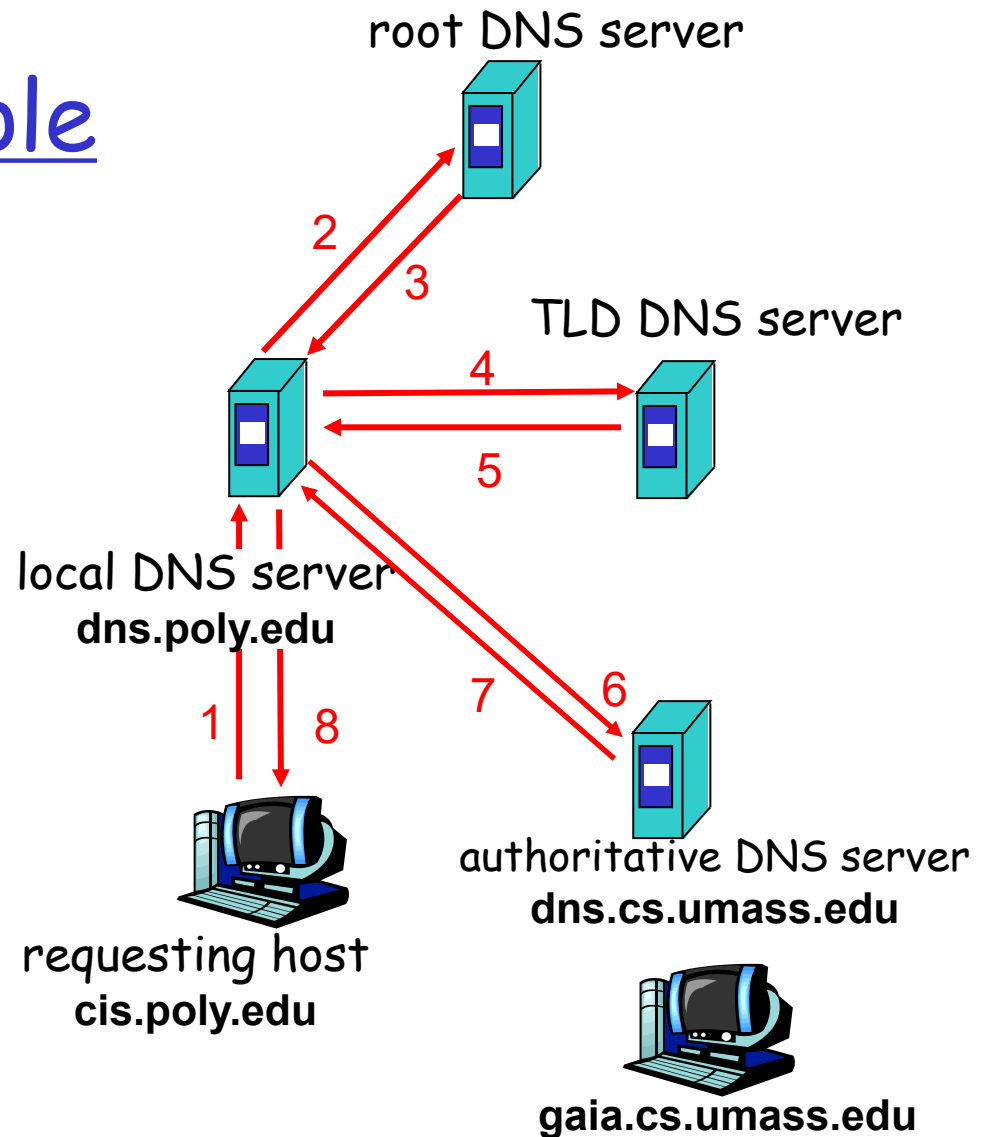
- ❑ does not strictly belong to hierarchy
- ❑ each ISP (residential ISP, company, university) has one.
 - ❖ also called "default name server"
- ❑ when host makes DNS query, query is sent to its local DNS server
 - ❖ acts as proxy, forwards query into hierarchy

DNS name resolution example

- Host at cis.poly.edu wants IP address for gaia.cs.umass.edu

Iterated迭代 query:

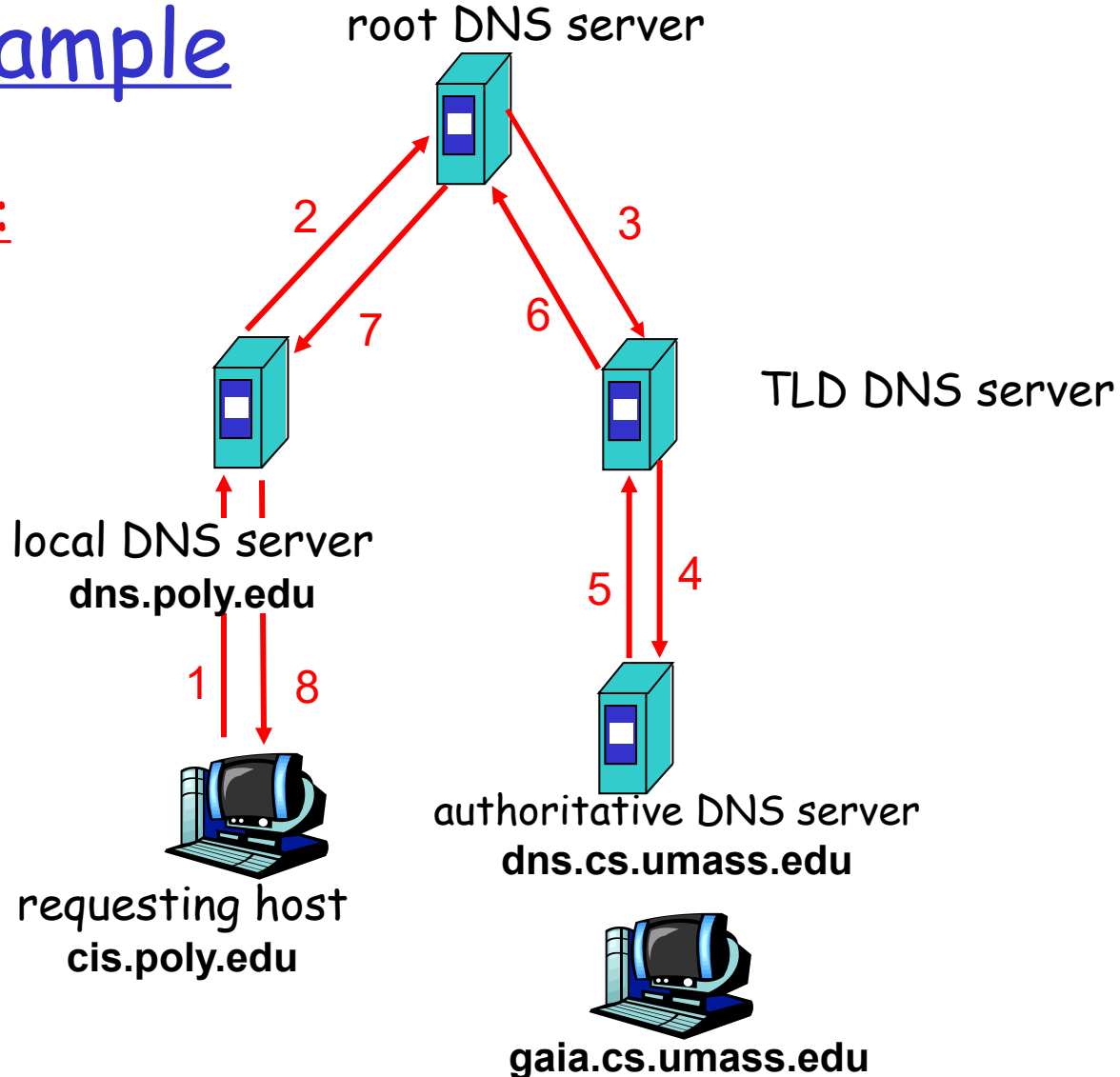
- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"



DNS name resolution example

Recursive递归 query:

- ❑ puts burden of name resolution on contacted name server
- ❑ heavy load?



DNS: caching and updating records

- once (any) name server learns mapping, it *caches* mapping
 - ❖ cache entries timeout (disappear) after some time
 - ❖ TLD servers typically cached in local name servers
 - Thus root name servers not often visited
- update/notify mechanisms under design by IETF
 - ❖ RFC 2136
 - ❖ <http://www.ietf.org/html.charters/dnsind-charter.html>

DNS records

DNS: distributed db storing resource records (RR)

RR format: (name, value, type, ttl)

□ Type=A

- ❖ name is hostname
- ❖ value is IP address

□ Type=NS

- ❖ name is domain (e.g. foo.com)
- ❖ value is hostname of authoritative name server for this domain

□ Type=CNAME

- ❖ name is alias name for some "canonical" (the real) name
www.ibm.com is really
servereast.backup2.ibm.com
- ❖ value is canonical name

□ Type=MX

- ❖ value is name of mailserver associated with name

DNS protocol, messages

DNS protocol : *query* and *reply* messages, both with same *message format*

msg header

- ❑ **identification**: 16 bit #
for query, reply to query
uses same #
- ❑ **flags**:
 - ❖ query or reply
 - ❖ recursion desired
 - ❖ recursion available
 - ❖ reply is authoritative

identification	flags
number of questions	number of answer RRs
number of authority RRs	number of additional RRs
questions (variable number of questions)	

↑
12 bytes
↓

事务ID (Transaction ID)	标志 (Flags)
问题计数 (Questions)	回答资源记录数 (Answer RRs)
权威名称服务器计数 (Authority RRs)	附加资源记录数 (Additional RRs)
查询问题区域 (Queries)	
回答问题区域 (Answers)	
权威名称服务器区域 (Authoritative nameservers)	
附加信息区域 (Additional records)	

<http://c.biancheng.net/view/6457.html>

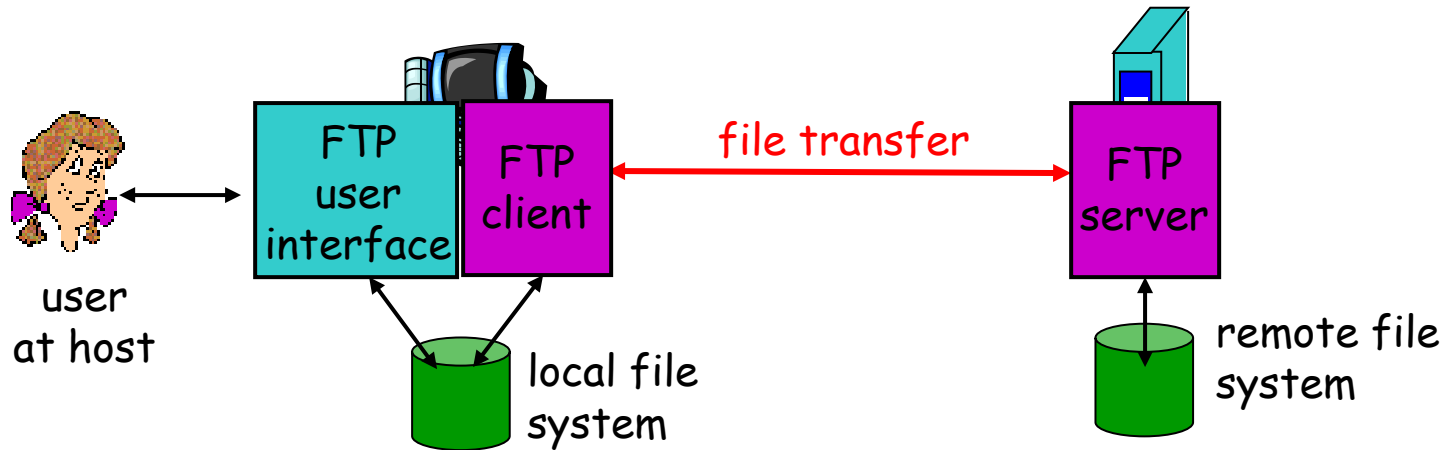
□ DNS很强壮！

□ 只遇到一次DNS攻击

Chapter 2: Application layer (补充)

□ FTP

FTP: the file transfer protocol



- ❑ transfer file to/from remote host
- ❑ client/server model
 - *client*: side that initiates transfer (either to/from remote)
 - *server*: remote host
- ❑ ftp: RFC 959
- ❑ ftp server: port 21

FTP与HTTP比较

都是文件传输协议，并运行在TCP上

✓ *FTP使用了两个并行的TCP连接*

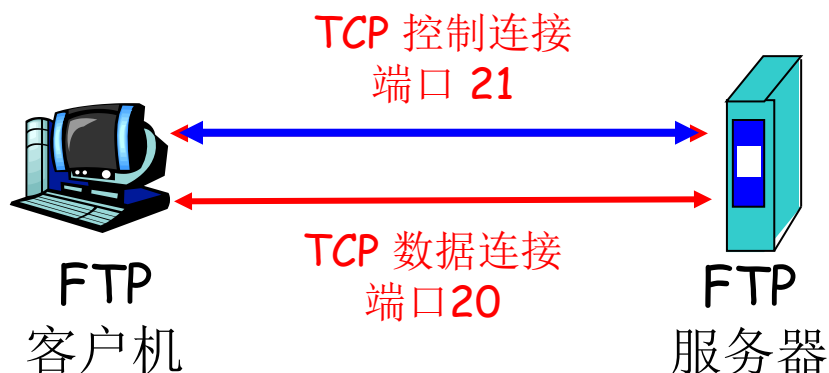
- ◆ 控制连接
- ◆ 数据连接



控制连接

用于在两主机间**传输控制信息**（如用户标识、口令等）

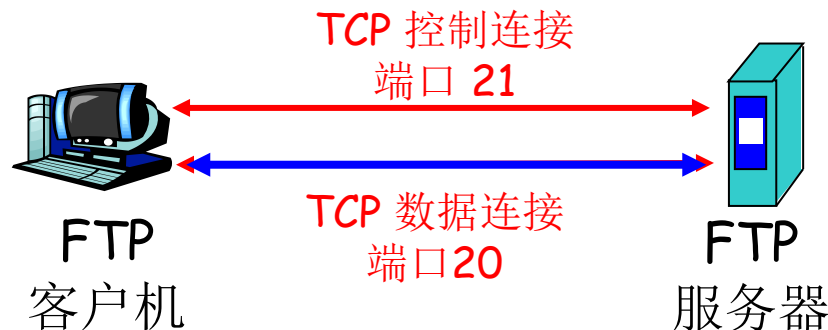
- ✓ FTP会话开始前，FTP的客户机与服务器在**21号端口**上建立
- ✓ FTP的客户机通过该连接发送用户标识和口令，或改变远程目录的命令



数据连接

用于准确传输文件

- ✓ 当服务器收到一个文件传输的命令后(从远程主机上读或写)，在20端口发起一个到客户机的数据连接
- ✓ 在该数据连接上传送一个文件并关闭连接
- 控制连接是持久的：在整个用户会话期间一直保持
- 数据连接是非持久的：会话中每进行一次文件传输，都需要建立一个新的数据连接



FTP命令, 响应

命令示例

- 经控制信道以ASCII 文本发送
- ✓ **USER *username***
- ✓ **PASS *password***
- ✓ **LIST**返回当前目录中的文件列表
- ✓ **RETR *filename***获取(get)文件
- ✓ **STOR *filename*** 存储 (puts) 文件到远程主机

返回码示例

- 状态码和短语(如在HTTP中的那样)
- ✓ **331 Username OK, password required**
- ✓ **125 data connection already open; transfer starting**
- ✓ **425 Can't open data connection**
- ✓ **452 Error writing file**

FTP



ftp上传工具 48个软件

的需求。现在比较常见FTP上传工具的有cuteftp和flashfxp。我们p小编用的就是flashfxp，比较方便而且功能不错，很智能，有很多可以设置的地方。这里小编推荐给大家一些优秀的FTP上传工具给大家，都比较简单易用，前三位的绝对可以说是精品了，觉得好用要记得分享哦！

分享到:

FlashFXP免费版
等级: ★★★★★
v5.4.0.3970 2017-07-17
6.8M 简体
推荐理由: 功能强大的 FXP/FTP 软件，融合了一些其他优秀 FTP软件的优点，如像 CuteFTP 一样可以比较文件夹，
版本: PC版

Cuteftp
等级: ★★★★★
v9.3.0.3官方 2018-07-13
19.4M 多国语言[中]
推荐理由: CuteFTP是小巧强大的FTP 工具之一，友好的用户界面，稳定的传输速度,CuteFTP最新Pro版是最好的FTP
版本: PC版 | Mac版

FileZilla(免费FTP客户端)
等级: ★★★★★
v3.47.1绿色 2020-02-26
22.5M 多国语言[中]
推荐理由: 一个免费的 FTP 客户端软件，虽然它是免费软件，可功能却一点也不含糊，比起那些共享软件来有过之而
版本: PC版 | Mac版

Bullet Proof FTP Registered to louie - ftp.buptnet.edu.cn (Default.bps)

Main Options Commands Session Toolbars Help

Host Port Login Pass

230- this then please disconnect
 230-
 230-
 230-
 230 Guest login ok, access restrictions apply.

C:\Program Files\BPFTP\ /pub

Filename	File Size	Date/Time
sites	0	99-10-18 19:14:40
VERSIONS.T...	23,292	99-10-6 10:34:42
README.TXT	1,857	99-10-6 10:38:28
BPFTP.REG	275	99-10-18 19:13:30
BPFTP.HLP	129,892	99-10-4 14:14:36
BPFTP.EXE	392,192	99-10-6 11:24:38

Filename	File Size	Date/Time	Description
solaris	512	99-10-14 14:01:00	
services	512	99-6-25 20:46:00	
security	512	99-10-1 11:01:00	
netscape	512	99-9-28 10:00:00	
linux	512	99-10-15 14:05:00	

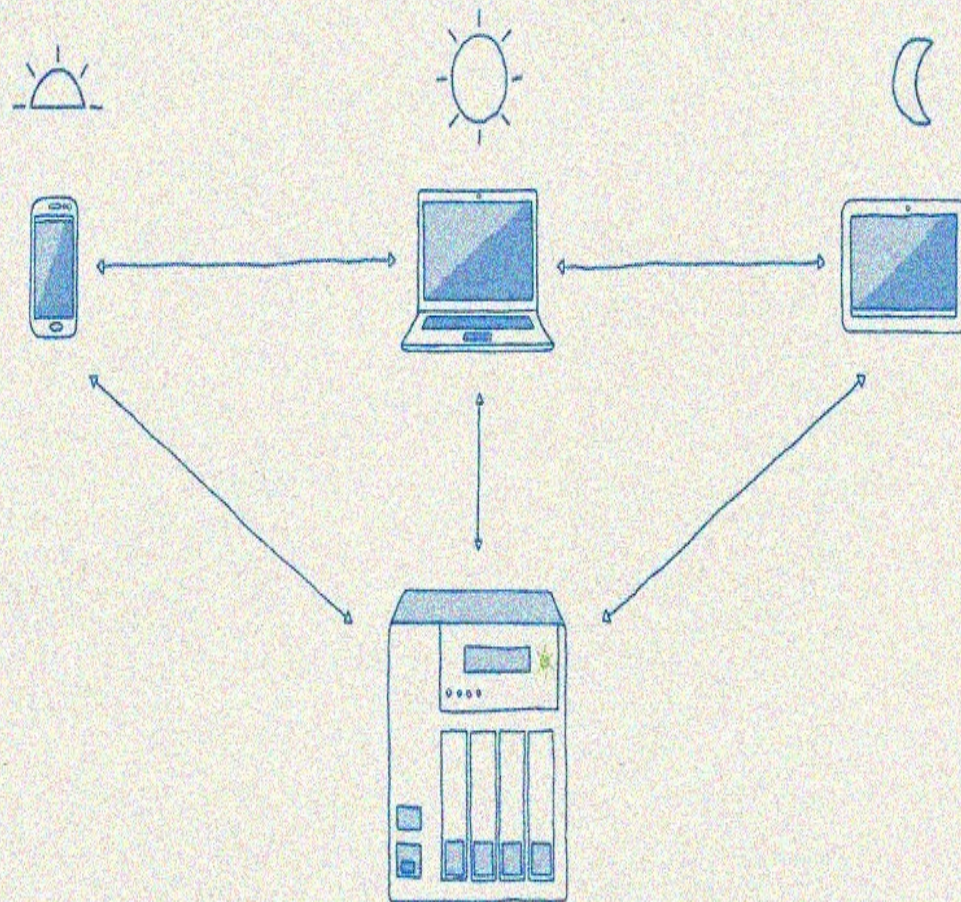
Filename	File Size	Date/Time	Destination



计算机网络

高旻

2024年3月



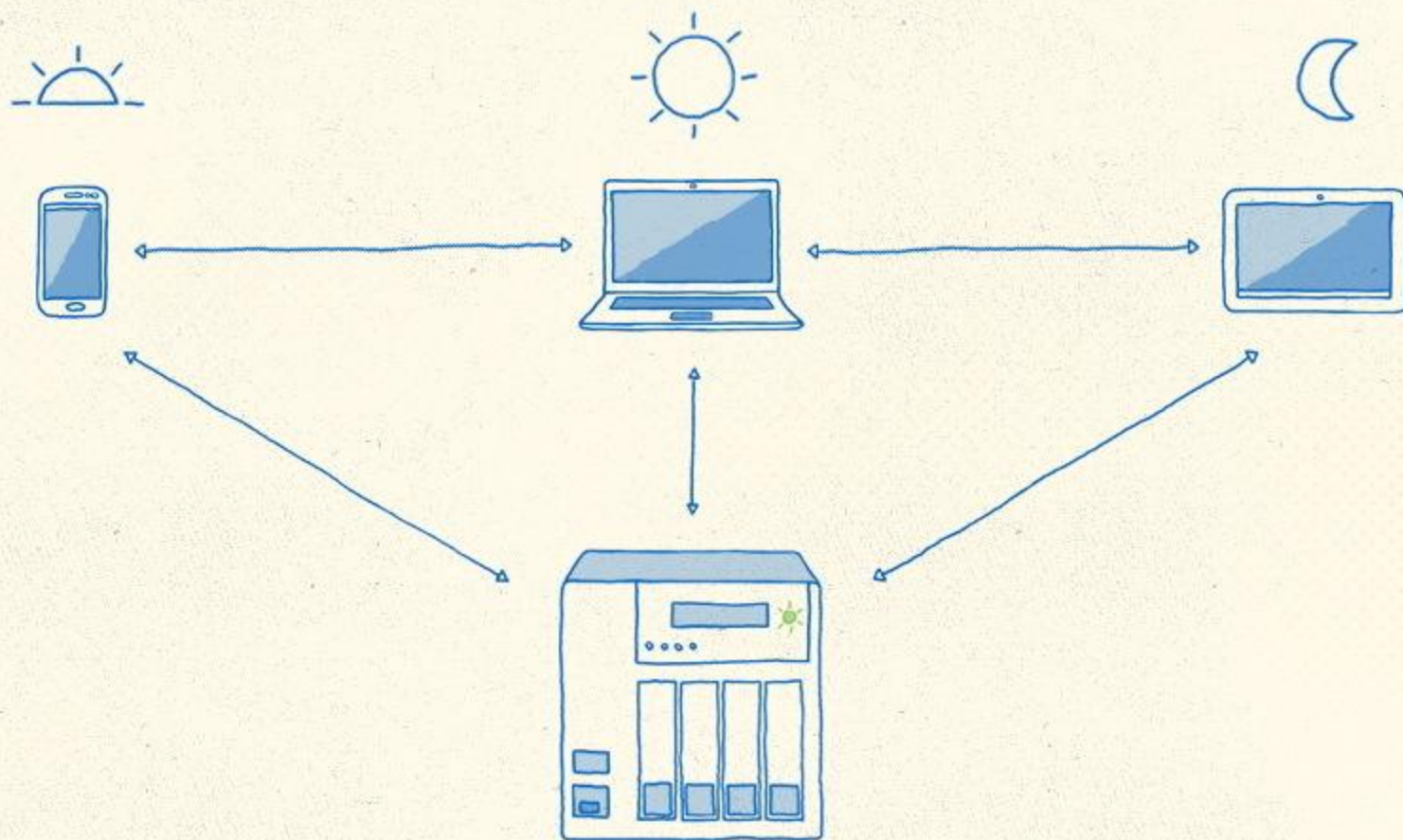
Your data. Always available.

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.4 DNS
- ❑ 2.5 P2P applications
- ❑ 2.6 Video Streaming and Content Distribution Networks
- ❑ 2.7 Socket programming

问题：大家都用过P2P软件吗？

BitTorrent Sync



Your data. Always available.

BitTorrent软件

The screenshot shows the BitTorrent application window. The top bar includes buttons for 'Add', 'Add URL', 'Start', 'Stop', and 'Remove', along with a search bar. The left sidebar shows a list of torrents under 'TORRENTS' and 'LABELS'. The main area displays a table of torrents. The selected torrent, 'Captive by Megan Lisa Jones', is shown in detail at the bottom, including its path, size, done status, percentage, number of pieces, and priority.

Name	#	Size	Done	Status	Seeds	Peers	↓ Speed	↑ Speed	ETA	Upload
Blender_Foundation_-_Big_Bu	2	316.8 M	21%	Downloading	31 (75)	1 (32)	532.4 kB/s	1.5 kB/s	8m 16s	121
LegalTorrents Collections - C	3	756.7 M	2%	Downloading	6 (15)	0 (6)	118.9 kB/s		1h 22m	
Megan Lisa Jones - Captive (BI	1	212.3 M	51%	Downloading	36 (48313)	0 (2493)	466.7 kB/s	1.0 kB/s	5m 6s	
Sick of Sarah - 2205 BitTo	*	49.2 MB		Seeding	0 (54634)	3 (2162)			∞	
Silent Flow Netlabel - Tetarise	4	94.3 MB	64%	Downloading	3 (6)	0 (1)	225.1 kB/s	0.3 kB/s	2m 16s	

Path	Size	Done	%	# Pi...	Pieces	Priority
Captive by Megan Lisa Jones - Bittorrent ...	2.9 MB	1.2 MB	42.6 %	12		normal
Description.txt	1.8 kB	0 B	0.0 %	1		normal
Introduction to Captive by Megan Lisa Jo...	209.3 MB	65.2 MB	31.1 %	839		normal
License.txt	142 B	0 B	0.0 %	1		normal

At the bottom of the window, the overall network status is displayed: ↓ 1.3 MB/s, ↑ 4.4 kB/s, and 0.004.

+ Infospace Search									
#	名称	播放	大小	状态	下载速度	上传速度	剩余...	种子/用户	
1	OBS-Studio-21.0.1-Full-Installer.exe		99.7 MB	正在下载 90.7 %	713.5 KB/s	2.2 KB/s	14 秒	1.486	

文件 信息 用户 Trackers 速度									
IP	客户端	标识	完成率	下载速度	上传速度	请求	已上传	已下载	用户下载...
thisis.feralhostin...	libTorrent 0.13.6	D E	100.0	599.6 KB/s	0.8 KB/s	123 ...		73.4 MB	
2001:0:9d38:6ab...	µTorrent 3.5.1	D IX...	100.0	22.6 KB/s		16 0		4.78 MB	
2001:0:4137:9e7...	µTorrent 3.5.1	D IX...	100.0	18.8 KB/s		7 0		3.67 MB	
2001:0:9d38:953...	µTorrent 3.5.1	D IX...	100.0	21.8 KB/s	0.2 KB/s	9 0		2.96 MB	
2001:0:4137:9e7...	BitTorrent 7.10	D IX...	100.0	37.0 KB/s	0.2 KB/s	14 0		1.81 MB	
2001:0:9d38:6ab...	µTorrent 3.5.1	D IE	100.0	11.4 KB/s	0.1 KB/s	7 0		688 KB	
0540534e.skybr...	µTorrent 3.5.1	DS E	100.0			4 0		160 KB	
176.126.177.235	µTorrent 3.5.1	DS E	100.0			2 0		128 KB	
174-134-239-91...	µTorrent 3.5.1	DS E	100.0			4 0		96.0 KB	
2001:0:4137:9e7...	µTorrent 3.5.1	D IE	100.0	2.0 KB/s		5 0		80.0 KB	
109-61-241-246...	µTorrent 3.5	DS E	100.0			1 0		32.0 KB	
45.251.36.118 [u...	BitTorrent 7.10	d P	100.0						
71.203.209.236	qBittorrent/4.0.3	DS E	100.0		0.5 KB/s	1 0			
112.209.38.233.p...	BitTorrent 7.10	d	100.0						

#	名称	播放	大小	状态	下载速度	上传速度	剩余...	种子/用户
1	OBS-Studio-21.0.1-Full-Installer.exe		99.7 MB	正在下载 54.9 %	551.8 KB/s	2.0 KB/s	1 分...	1.478

文件

信息

用户

Trackers

速度

已经下载:

健康程度:

54.9 %

16.499

传输

已用时间: 3 分钟 41 秒

已经下载: 54.8 MB

下载速度: 551.8 KB/s (平均 253.9 KB/s)

下载限制: ∞

传输状态: Downloading

剩余时间: 1 分钟 38 秒

已经上传: 0 B

上传速度: 2.0 KB/s (平均 0 B/s)

上传限制: 2.0 KB/s

丢弃数据: 54.6 KB (0个散列错误)

种子数量: 16/27 已连接 (缓存: 281)

用户数量: 0/190 已连接 (缓存: 1)

分享率: 0.000

常规

文件位置: d:\TorrentDownloads\OBS-Studio-21.0.1-Full-Installer.exe

总计大小: 99.7 MB (54.8 MB 完成)

创建时间: 2018/1/23 9:57:19

添加日期: 2018/2/12 16:55:13

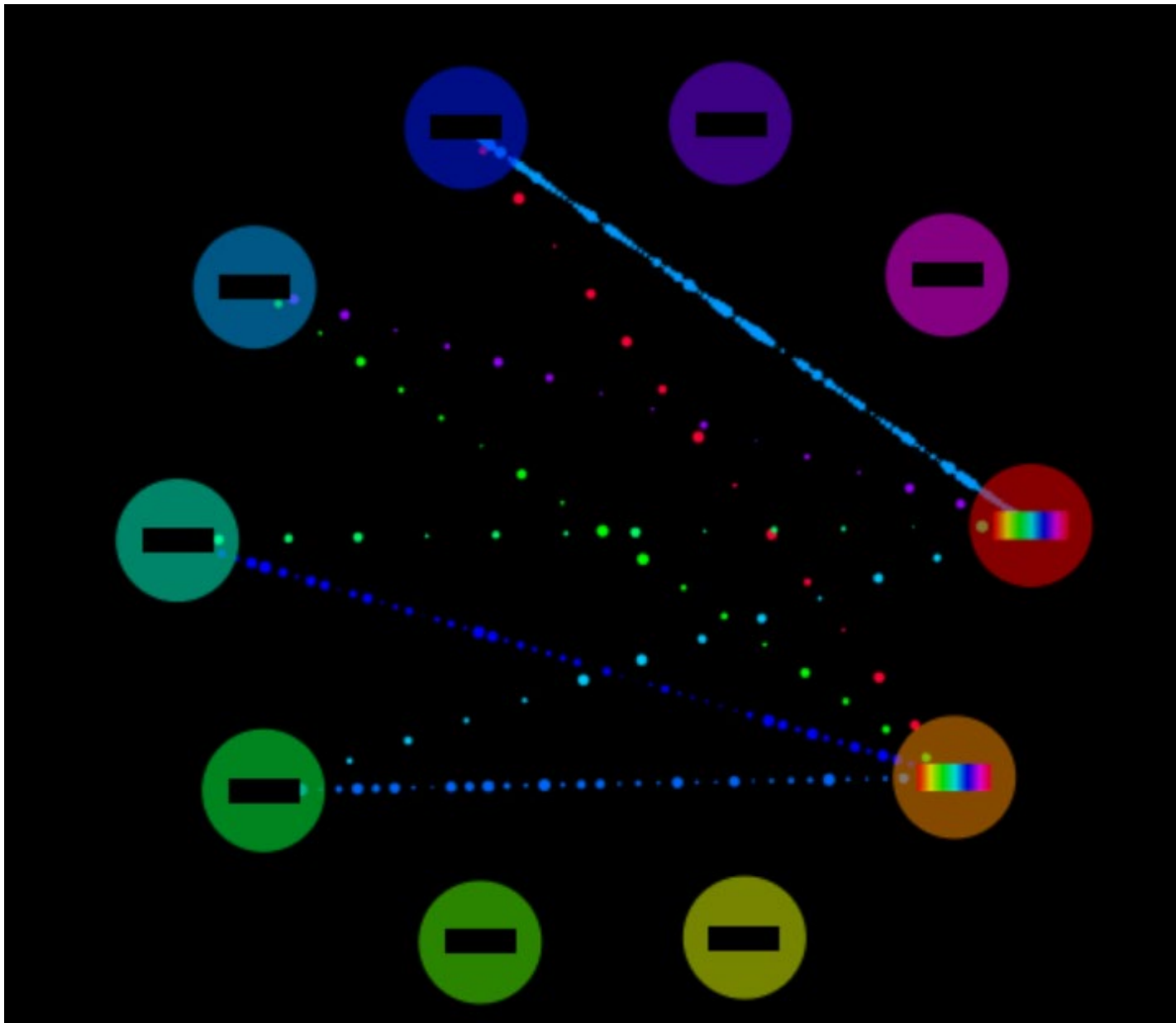
校验码: CED8FF64D7183C78176325C268138687A1390E45

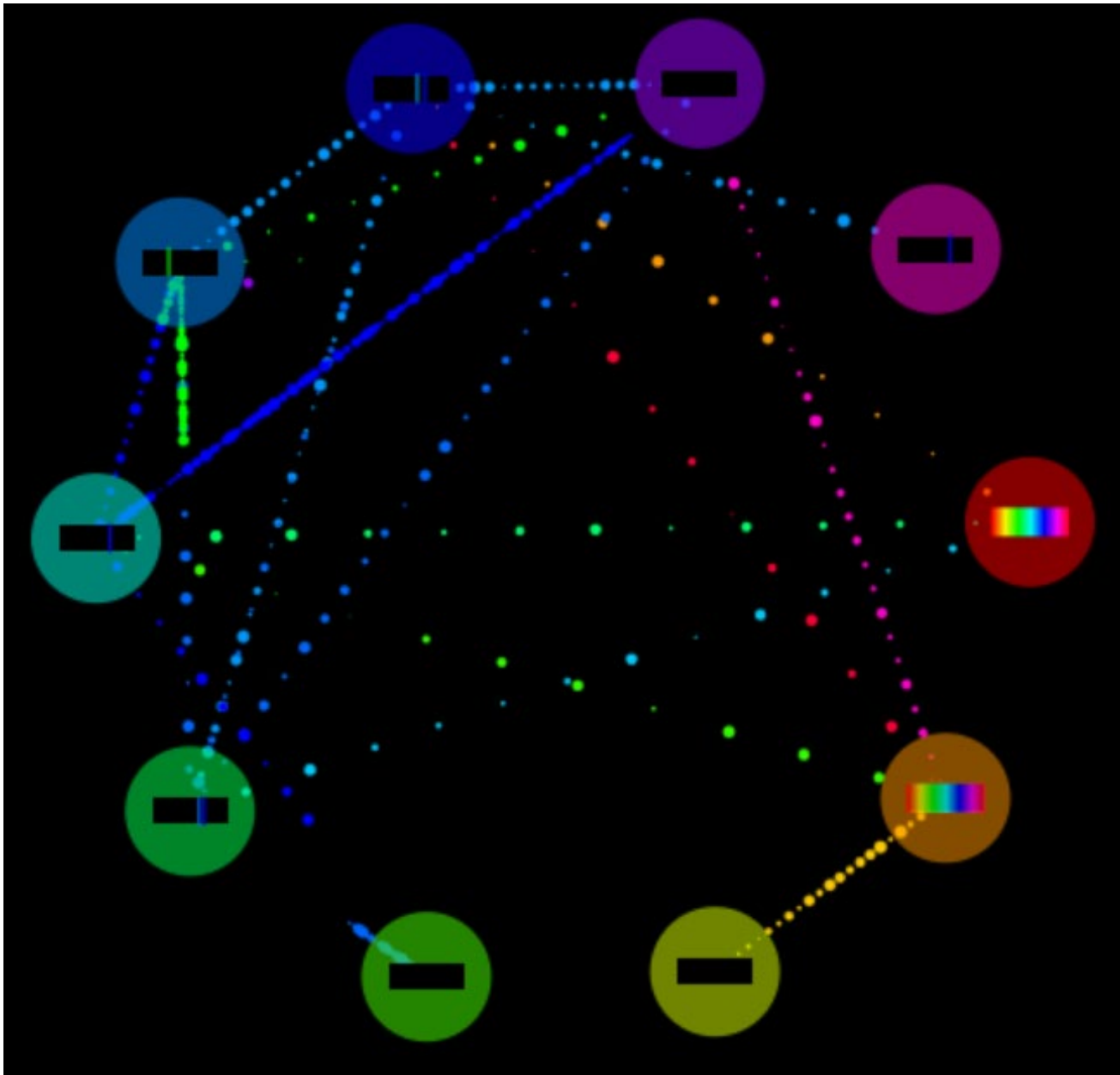
说明: Open Broadcaster Software, <https://obsproject.com/>

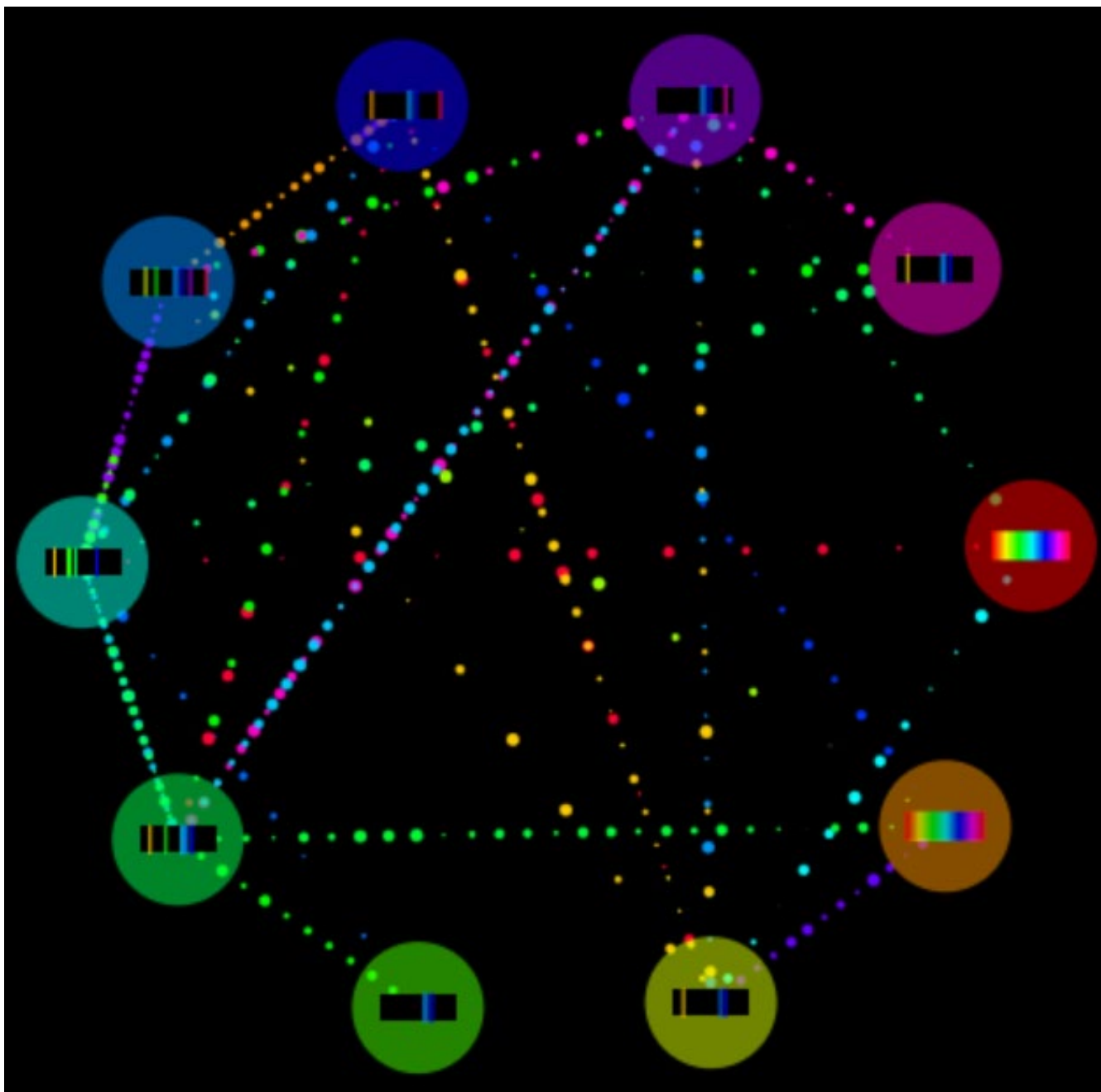
区块数量: 1597 x 64.0 KB (存在 871)

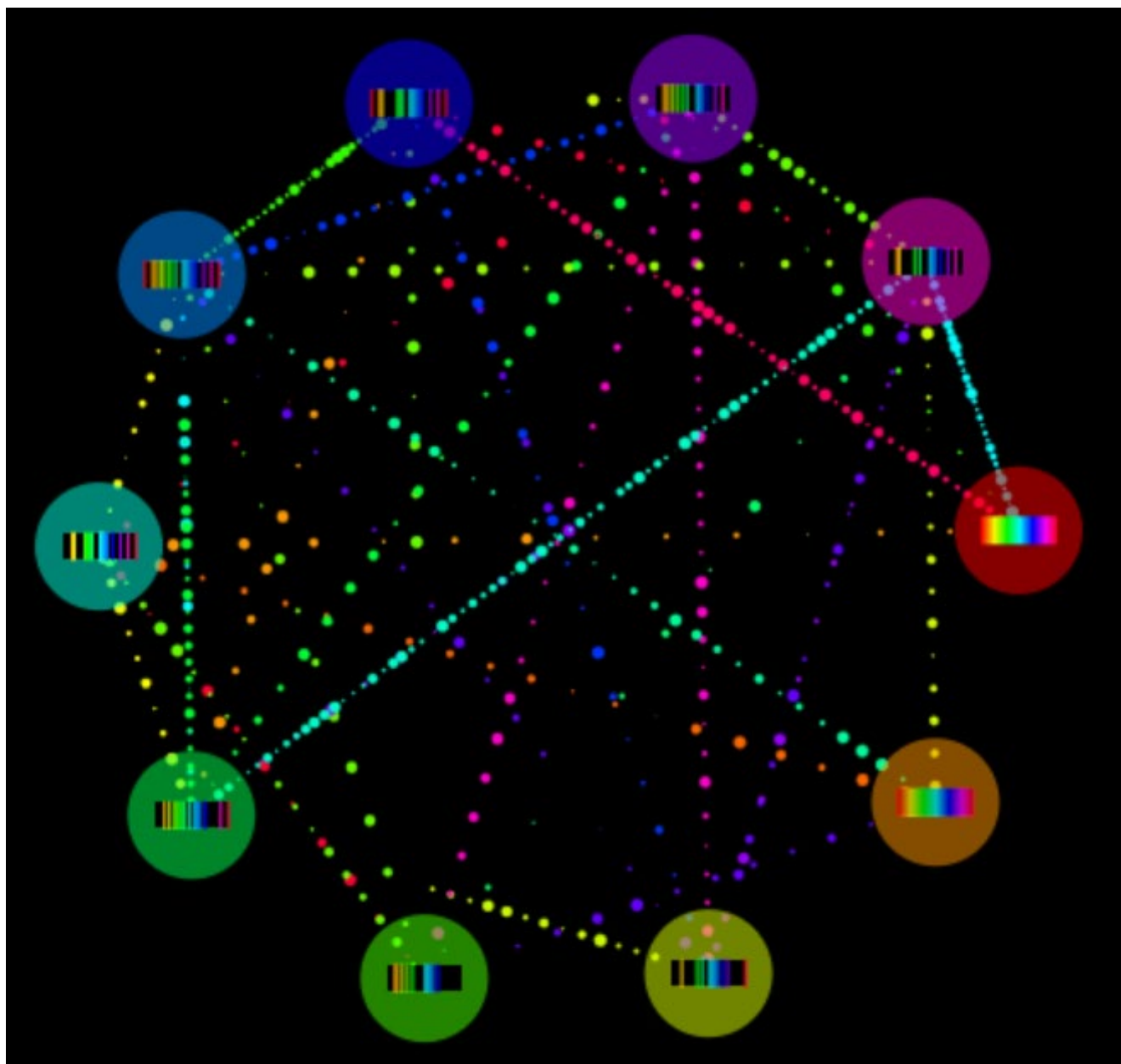
创建于: Transmission/2.92 (14714)

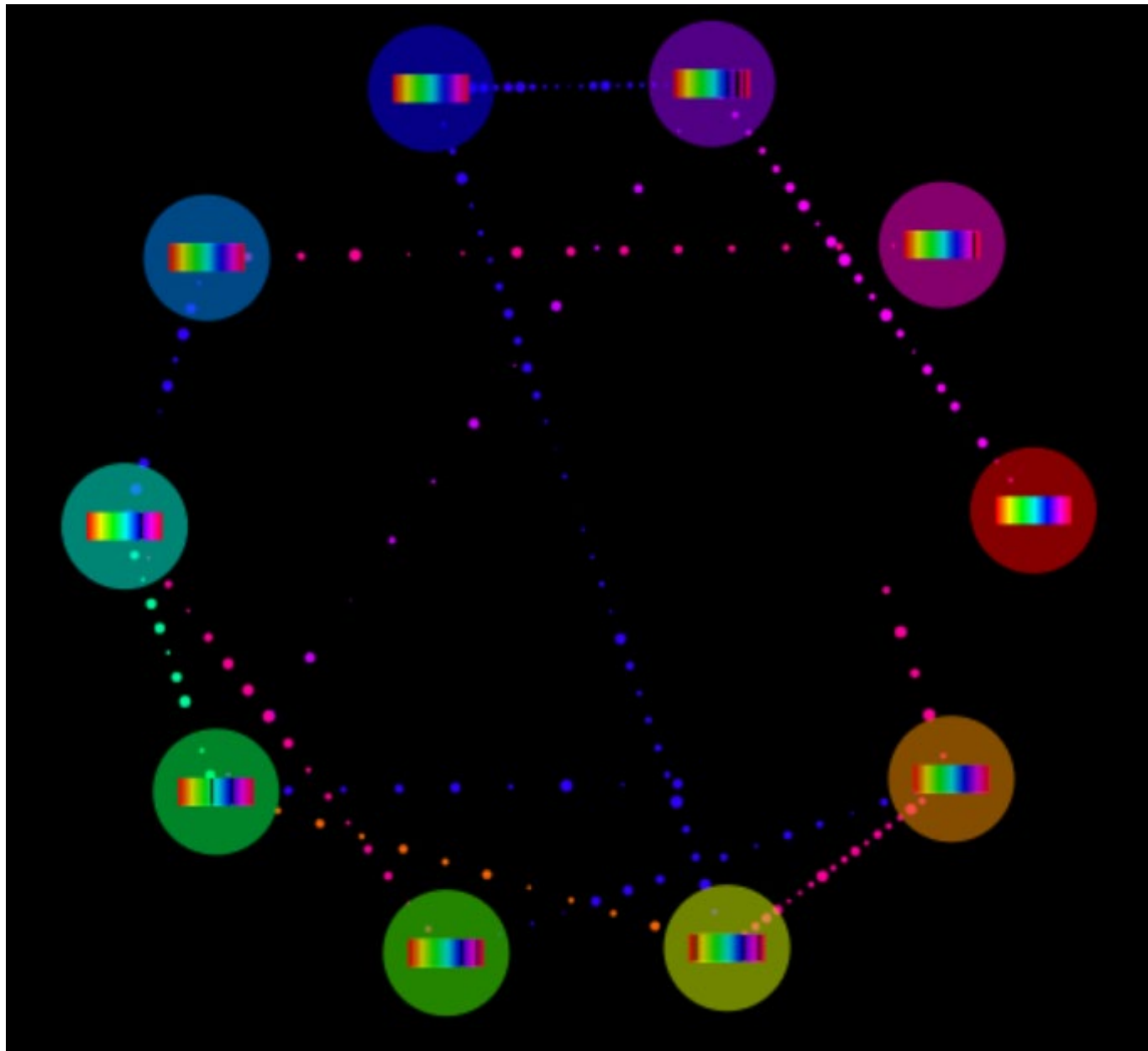
完成日期: 不完整







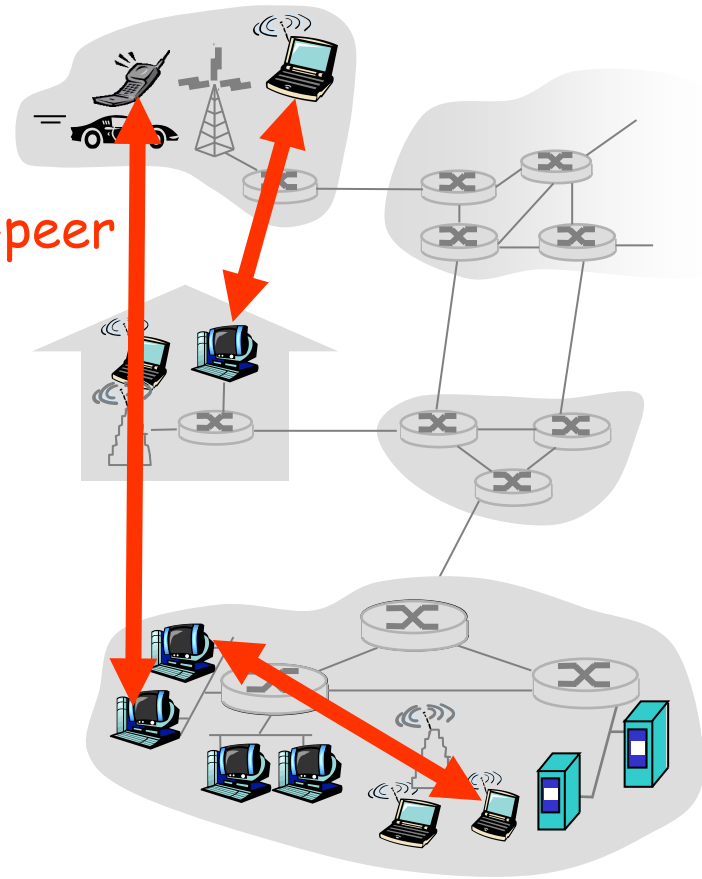




Pure P2P architecture

- ❑ no always-on server
- ❑ arbitrary end systems directly communicate
- ❑ peers are intermittently connected and change IP addresses

peer-peer



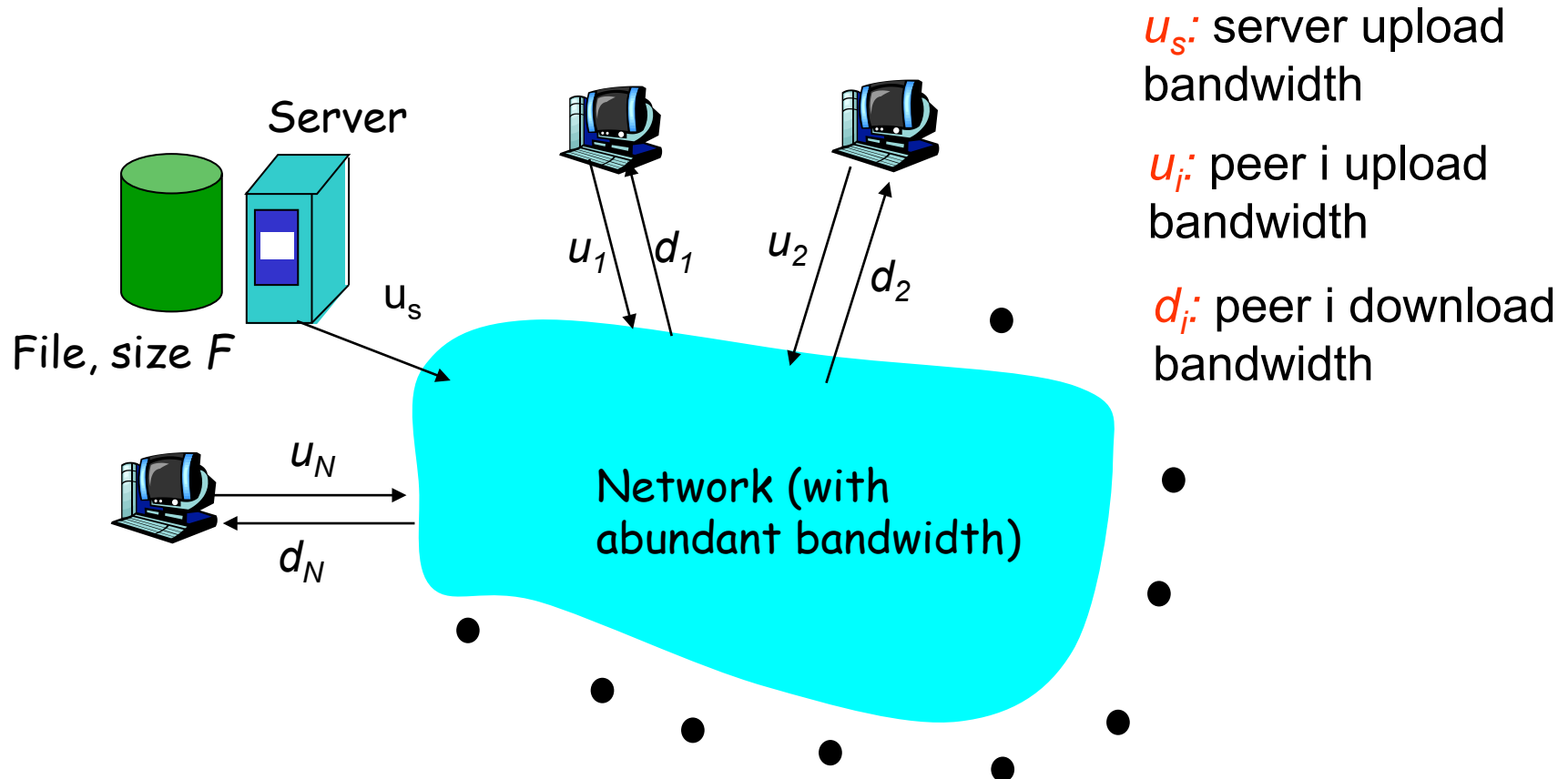
例，用户Alice使用P2P文件共享应用程序下载MP3。

过程

- ❑ 在PC机上运行一个P2P文件共享应用软件(对等方)
- ❑ 通过ADSL间歇地接入因特网（无固定IP地址）
- ❑ 使用该应用程序搜索一首MP3歌曲
- ❑ 显示一张有该首歌曲的对等方列表
 - ✓ 所有在线对等方，愿意共享该首音乐的MP3拷贝
 - ✓ 对等方都是普通的PC，是因特网用户
 - ✓ 列表中提供一些附加信息，如接入带宽和下载时间
- ❑ 选择一个对等方（Bob）并向其请求该MP3文件
 - ✓ 两个用户之间建立一个直接的TCP连接
 - ✓ MP3文件从Bob的PC机向Alice的PC发送
 - ✓ 下载期间若偶然断开，可从其他对等方继续下载

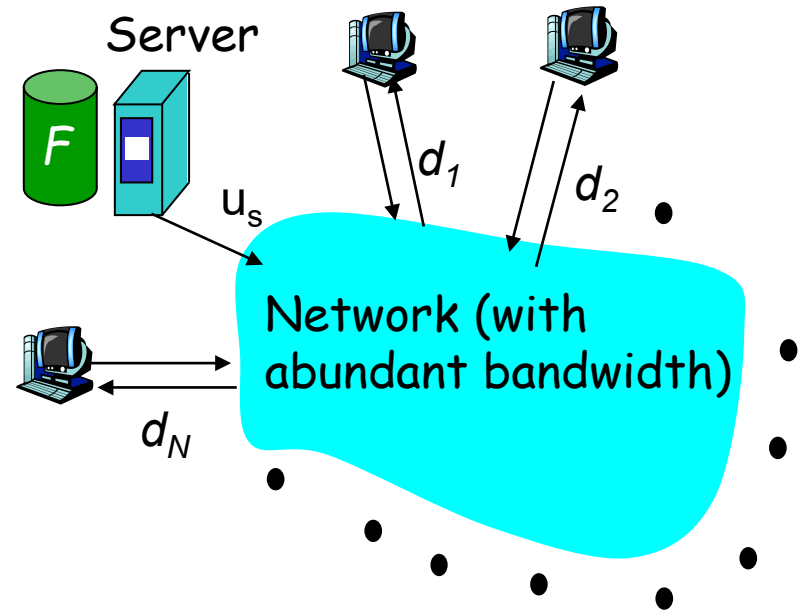
File Distribution: Server-Client vs P2P

Question : How much time to distribute file from one server to N peers?



File distribution time: server-client

- **server** sequentially sends N copies:
 - ❖ NF/u_s time
- **client** i takes F/d_i time to download

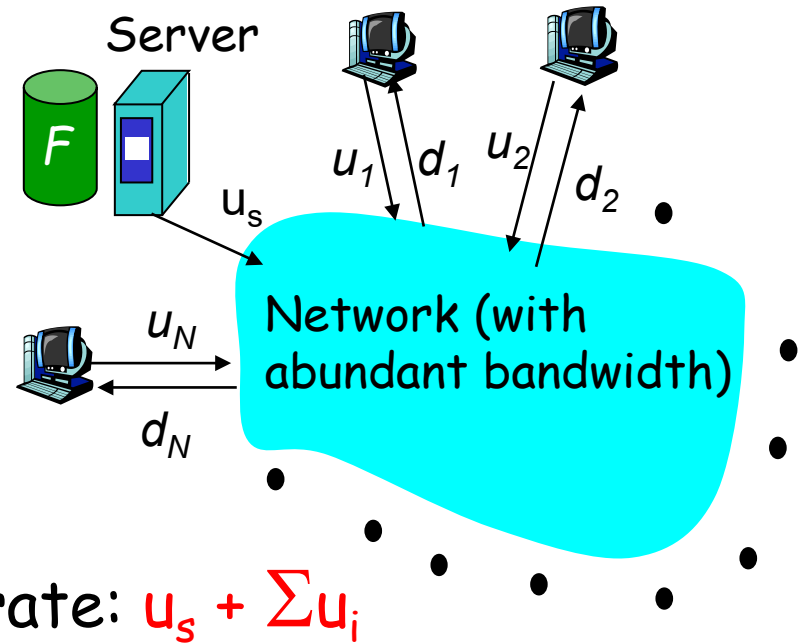


$$\text{Time to distribute } F \text{ to } N \text{ clients} = d_{cs} = \max \{ NF/u_s, F/\min_i(d_i) \}$$

increases linearly in N
(for large N)

File distribution time: P2P

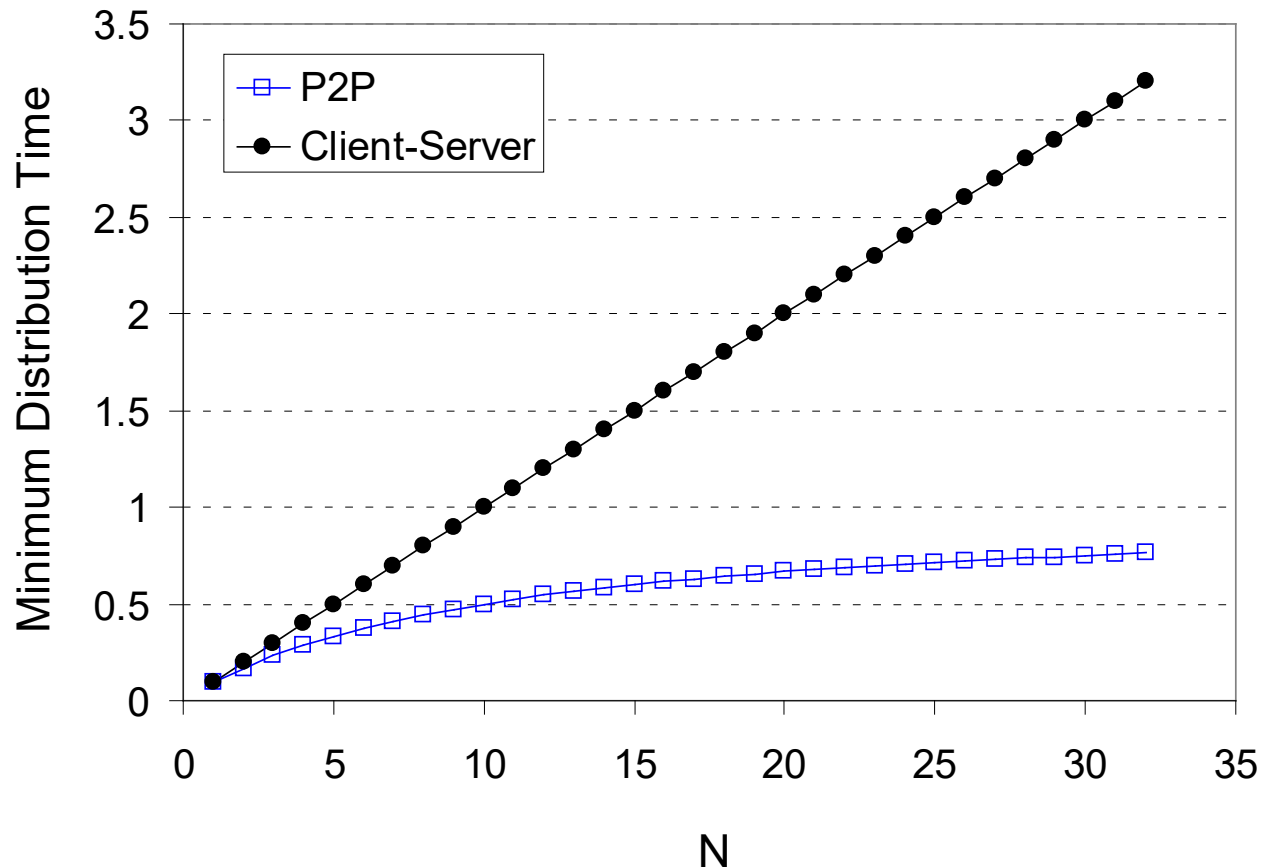
- ❑ **server** must send one copy: F/u_s time
- ❑ **client** i takes F/d_i time to download
- ❑ NF bits must be uploaded (aggregate)
 - ❑ fastest possible **upload** rate: $u_s + \sum u_i$



$$d_{\text{P2P}} = \max \left\{ F/u_s, F/\min(d_i)_i, NF/(u_s + \sum u_i) \right\}$$

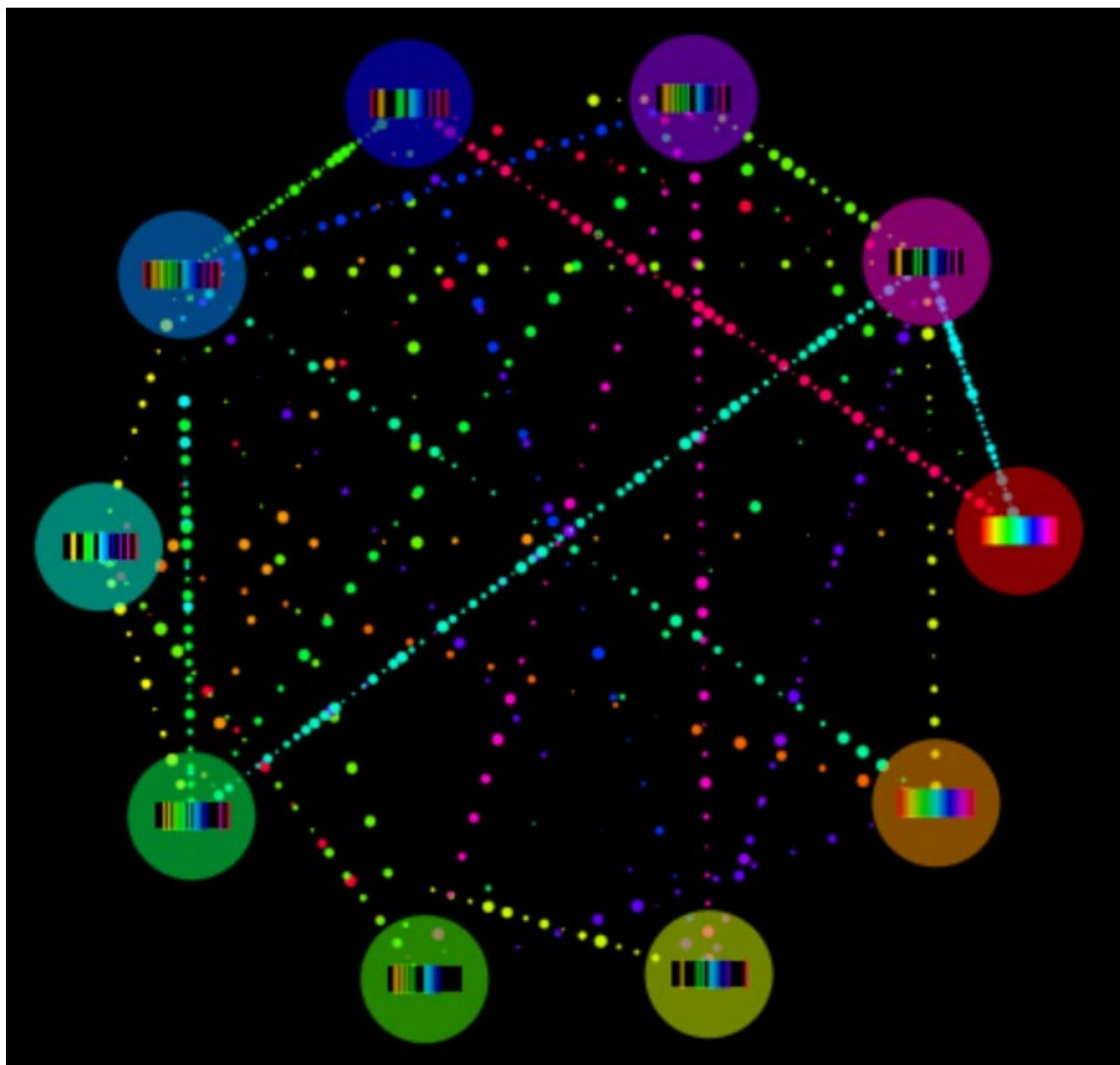
Server-client vs. P2P: example

Client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{\min} \geq u_s$



P2P文件共享特点

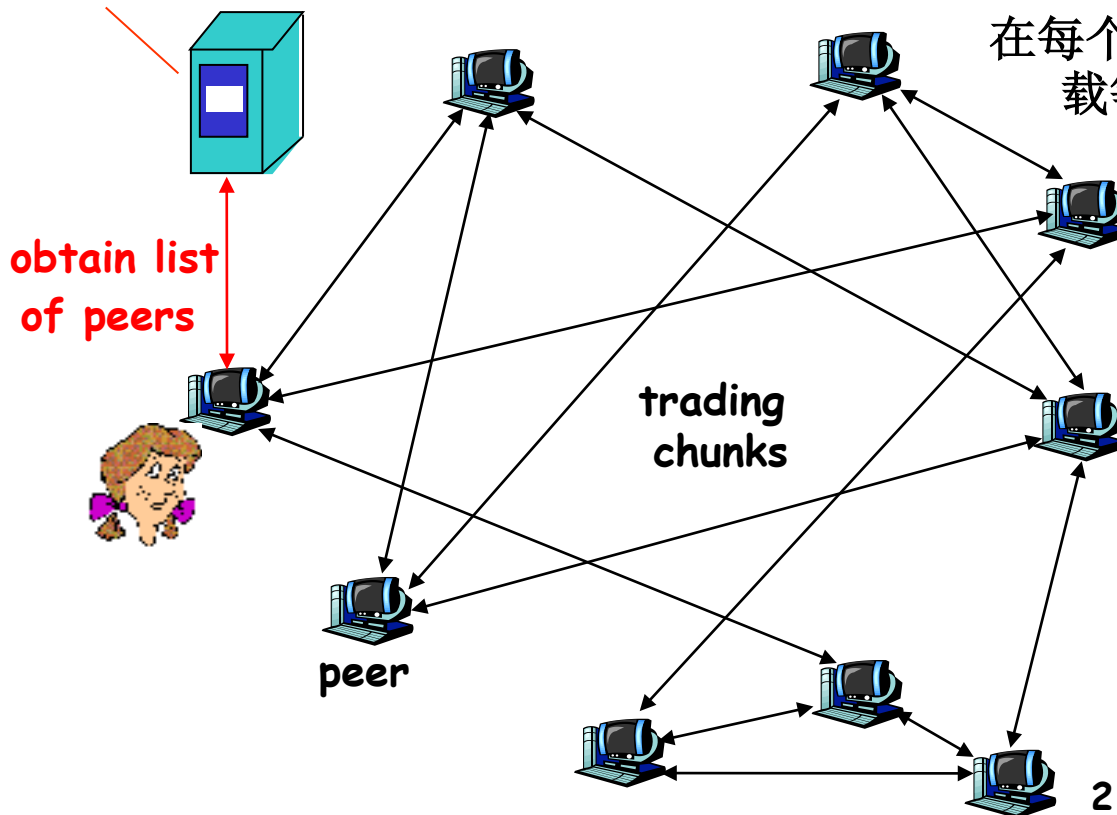
- ❑ **直接在对等方间传输：**所有内容不经过第三方服务器
- ❑ **高度的可扩展能力：**利用众多对等方集合中的资源去分发内容
- ❑ **使用客户机/服务器模式：**请求的对等方是客户机，被选中的对等方是服务器
 - ✓ 服务器对等方使用文件传输协议向客户机对等方传送
 - ◆ 通过传送“HTTP请求”和“HTTP响应”报文进行
 - ✓ 所有的对等方必须运行文件传输协议的客户机端程序，又同时运行服务器端程序
 - ◆ 对等方既是一个客户机，又是一个服务器



File distribution: BitTorrent

□ P2P file distribution

追踪器**tracker**: tracks peers participating in torrent



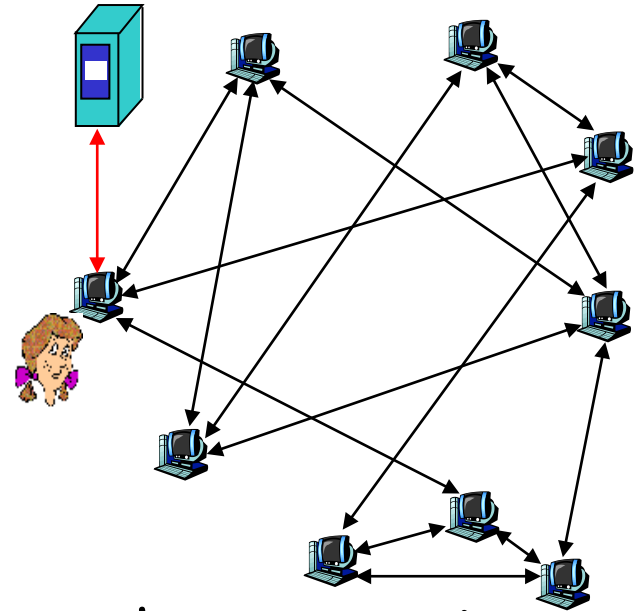
洪流**torrent**: group of peers exchanging **chunks** of a file

参与一个特定文件分发的所有对等方的集合

在每个洪流中，对等方彼此下载等长的文件**块**

BitTorrent (1)

- ❑ file divided into 256KB *chunks*
- ❑ peer joining torrent
 - ❖ has no chunks, but will *accumulate* them over time
 - ❖ registers with tracker to get list of peers, *connects* to subset of peers ("neighbors")
- ❑ while *downloading*, peer *uploads* chunks to other peers
- ❑ peers may come and go
- ❑ once peer has entire file, it may (selfishly) leave or (altruistically) remain



BitTorrent (2)

Pulling Chunks

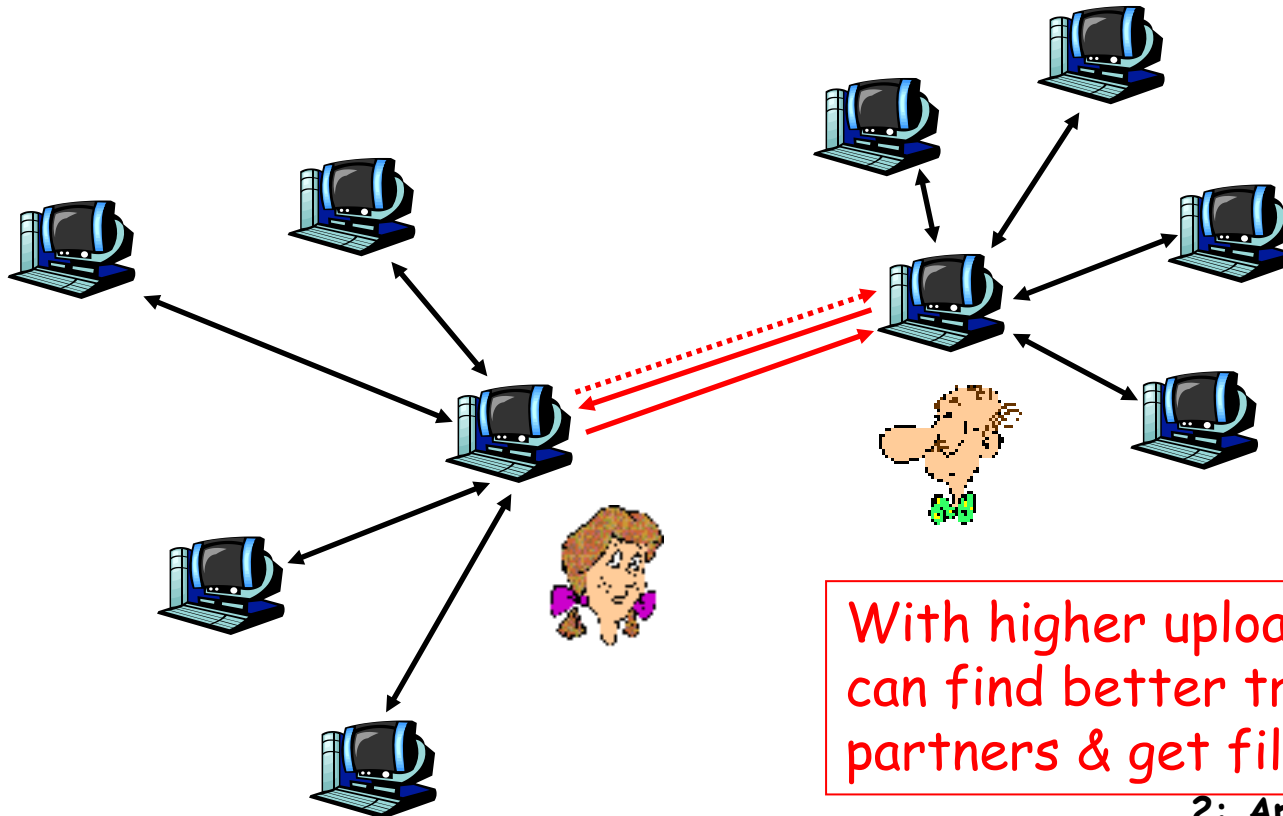
- at any given time, different peers have different subsets of file chunks
- periodically, a peer (Alice) asks each neighbor for list of chunks that they have.
- Alice sends requests for her missing chunks
 - ❖ **rarest first**

Sending Chunks: tit-for-tat

- Alice sends chunks to four neighbors currently sending her chunks at the *highest rate*
 - ❖ re-evaluate top 4 every 10 secs
- every 30 secs: randomly select another peer, starts sending chunks
 - ❖ newly chosen peer may join top 4
 - ❖ "optimistically unchoke"

BitTorrent: Tit-for-tat对换算法

- (1) Alice "optimistically unchokes" Bob
- (2) Alice becomes one of Bob's top-four providers; Bob reciprocates
- (3) Bob becomes one of Alice's top-four providers



With higher upload rate,
can find better trading
partners & get file faster!

内容定位体系结构

一个对等方如何确定哪个对等方有所需要的内容。

(1) 集中式目录

(2) 查询洪泛

(3) 层次覆盖网络

(4) 分布式散列表 (DHT)



(1) 集中式目录

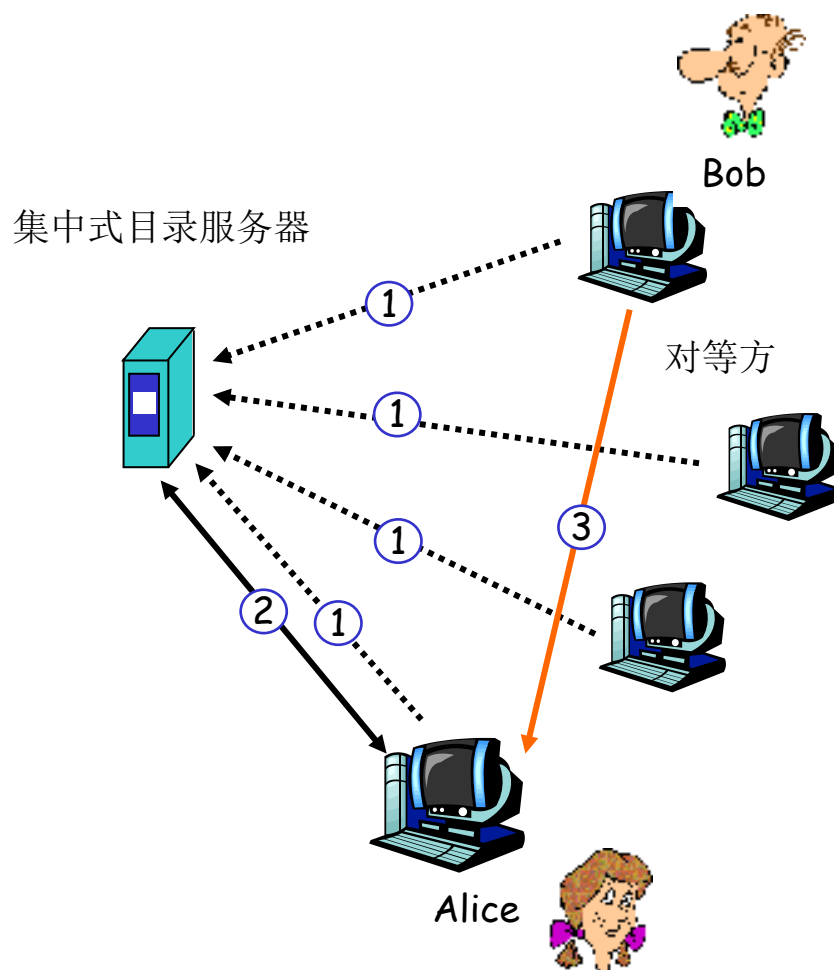
源于Napster公司（第一家世界范围内从事MP3对等应用程序）设计。

- ❑ **目录服务器**(大型服务器或服务器场)：提供目录服务。

收集可共享的对象，建立集中式的动态数据库（对象**名称到IP地址**的映射）。

- ❑ **问题**

- ✓ 单点故障
- ✓ 性能瓶颈
- ✓ 侵犯版权



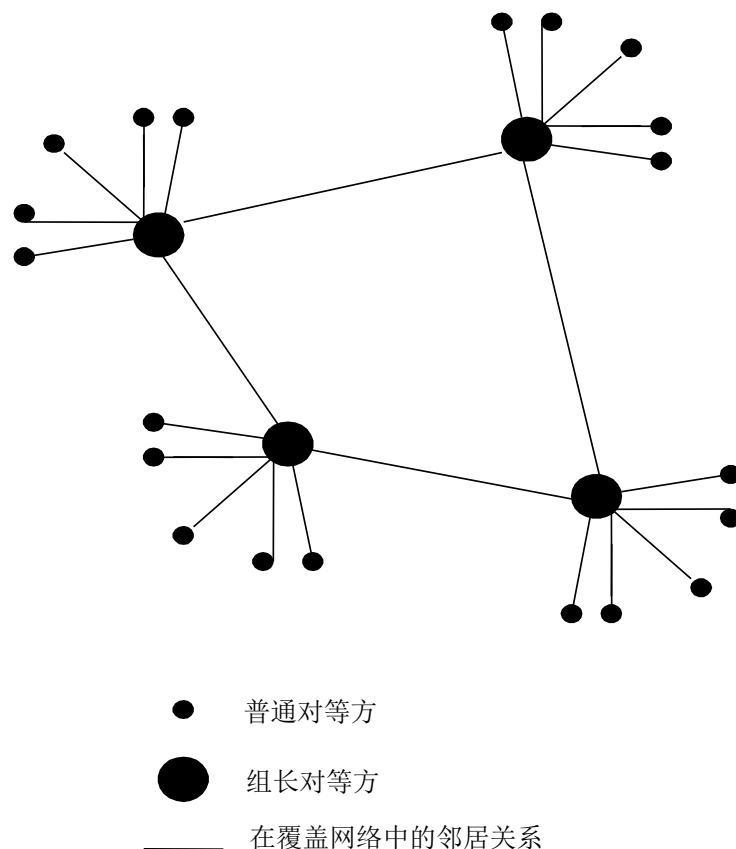
(2) 查询洪泛: Gnutella

- ✓ 是一个公共域文件共享应用程序。
- ✓ 全分布方式: 无中心服务器
- ✓ 许多Gnutella客户机实现协议: 运行在对等方。
- 实现:
 - ✓ 对等方先形成一个抽象的逻辑网络 (覆盖网络)
 - ✓ 通过洪泛查询, 找到所需对象
- 缺点:
 - ✓ 设计简单。
 - ✓ 扩展性差。
 - ✓ “查询报文” 在网络中产生很大的流量。

(3) 层次覆盖网络KaZaA

与Gnutella类似，**无专用服务器，但对等方地位不平等。对等方根据通信关系划分若干组，组成层次结构。**

- 每组包括若干个组员，一个组长
 - ❖ 组员与其组长有一个TCP连接，**将共享内容告诉组长。**
 - ❖ 组长维护一个数据库，该组的共享内容及相关对等方的IP地址。
 - ❖ **相关组长之间建立TCP连接**
- 组长追踪其所有子节点上的内容



(4) 分布式散列表DHT

- ❑ 资源用(键, 值)存储, 如(The Shawshank Redemption, 128.187.123.38) (肖申克救赎, $IP_{张三}$) 和 (肖申克救赎, $IP_{李四}$)
- ❑ 在数以百万计的对等方上存储 (键, 值)
- ❑ 如何知道哪些对等方负责哪部电影的所有 (键, 值) 呢?
- ❑ 如果对等方非常多, 不可能一个一个的询问, 一个可行的方法是采用距离判断, 哪个对等方离资源近就
- ❑ 哈希表 (散列表), 哈希函数形成一个值, 每个对等方对应一个值。
- ❑ 分布式数据库则将定位拥有该相应 (键, 值) 的对等方, 然后向查询的对等方返回该 (键, 值)

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.4 DNS
- ❑ 2.5 P2P applications
- ❑ 2.6 Video Streaming and Content Distribution Networks 视频流与内容分发网
- ❑ 2.7 Socket programming

问题:

- 1) 如何做到为不同用户提供不同质量的视频（高清、清晰、一般）
- 2) 网上的视频平台，如腾讯视频，如何为M以上的用户服务？

Video Streaming and CDNs: context

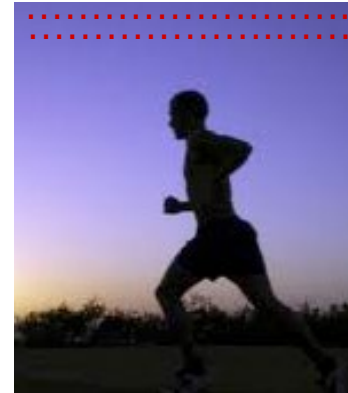
- video traffic: major consumer of Internet bandwidth
 - Netflix, YouTube: 37%, 16% of downstream residential ISP traffic (As of 2015)
 - ~1B YouTube users, ~75M Netflix users
- challenge: heterogeneity (异质性)
 - different users have different capabilities (e.g., wired versus mobile; bandwidth rich versus bandwidth poor)
- challenge: scale (庞大的规模) - how to reach ~1B users?
 - single mega-video server won't work
- *solution*: distributed, application-level infrastructure



Multimedia: video

- video: sequence of images displayed at constant rate
 - e.g., 24 images/sec 像素阵列
- digital image: array of pixels
 - each pixel represented by bits 视频的重要特征: 能被压缩
- coding: use redundancy *within* and *between* images to decrease # bits used to encode image
 - spatial (within image)
 - temporal (from one image to next)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i



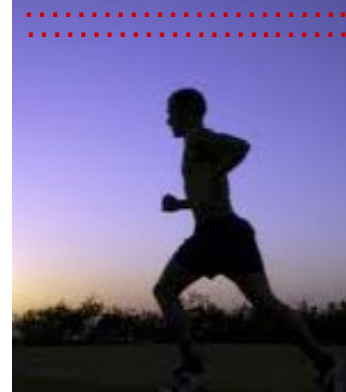
frame $i+1$

Multimedia: video

■ examples:

- MPEG I (CD-ROM) 1.5 Mbps
- MPEG2 (DVD) 3-6 Mbps
- MPEG4 (often used in Internet, < 1 Mbps)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

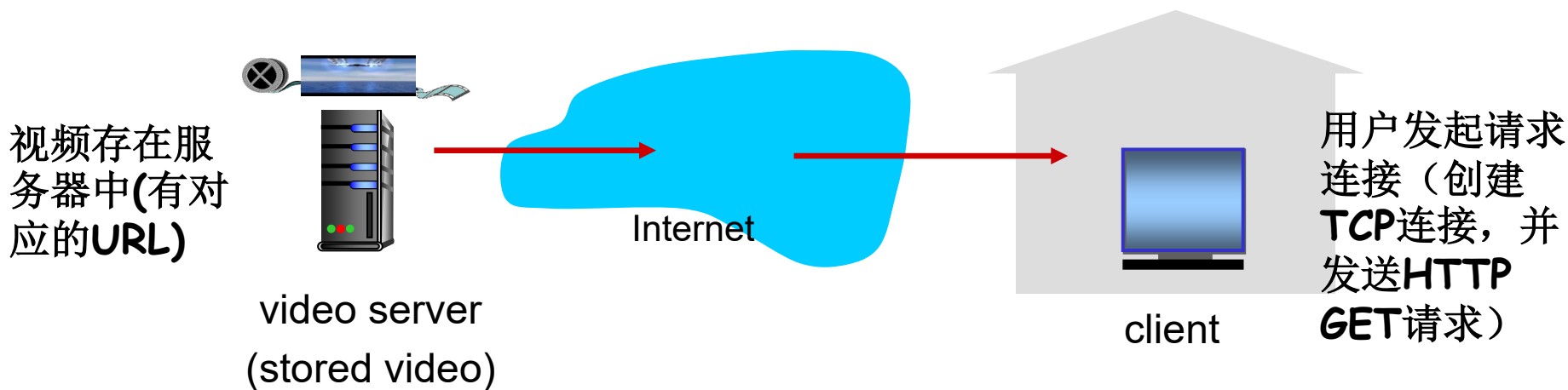
temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i



frame $i+1$

Streaming stored video:

simple scenario:



- 服务器以尽可能快的速率，在**HTTP**响应报文中发送这个视频文件

- 客户端，字节被收集在应用缓存中
- 一旦缓存中的字节数量超过设定的阈值，应用程序就开始播放
- 应用程序周期性地从缓存中抓取帧，解压并播放；同时缓存后面的帧

存在问题：所有客户收到相同编码的视频

Streaming multimedia: DASH

- *DASH*: *D*ynamic, *A*daptive *S*treaming over *H*TTP
经HTTP的动态适应性流
- *server*:
 - divides video file into multiple chunks
 - each chunk stored, encoded **at different rates**
 - *manifest file*: provides URLs for different chunks
- *client*:
 - periodically measures server-to-client bandwidth
 - consulting manifest, requests one chunk at a time
 - chooses **maximum coding rate** sustainable given current bandwidth
根据当前带宽选择最大的编码率
 - can choose different coding rates **at different points** in time
(depending on available bandwidth at time)
在不同时间点选择不同的编码率

Streaming multimedia: DASH

- *DASH: Dynamic, Adaptive Streaming over HTTP*
- “intelligence” at client: client determines
 - *when* to request chunk (so that buffer starvation, or overflow does not occur)
 - *what encoding rate* to request (higher quality when more bandwidth available)
 - *where* to request chunk (can request from URL server that is “close” to client or has high available bandwidth)

Content distribution networks

- *challenge*: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?
- *option 1*: single, large “mega-server”
 - single point of failure
 - point of network congestion
 - long path to distant clients
 - multiple copies of video sent over outgoing link

....quite simply: this solution *doesn't scale*

Content distribution networks

- *challenge*: how to stream content (selected from millions of videos) to hundreds of thousands of simultaneous users?

在不同地理位置的服务器存储视频多个备份

- *option 2*: store/serve multiple copies of videos at multiple geographically distributed sites (*CDN*)

- *enter deep*: push CDN servers deep into many access networks

深入到很多接入网ISP

- close to users
- used by Akamai, 1700 locations

- *bring home*: smaller number (10's) of larger clusters in POPs near (but not within) access networks

- used by Limelight

在关键位置建造大集群，通常放在因特网交换点IXP，邀请ISP来做客

Akamai 阿卡迈

- 是总部位于美国麻省的内容分发网络和云服务提供商，是世界上最大的分布式计算平台之一，承担了全球15-30%的网络流量。
- 该公司主要业务是出租服务器的容量给客户，使得他们能通过从靠近用户的位置来分发内容来提高访问速度。
- 客户包括苹果公司、Facebook、微软、Twitter 等。
- 当用户浏览到Akamai的客户的URL，他的浏览器将被重指向到Akamai的副本之一的网站，此操作对大部分客户而言完全透明。

Limelight Networks

- 总部美国亚利桑那坦佩
- Limelight通过腾讯云为其客户提供在中国的内容分发网络(CDN)服务。而腾讯云通过Limelight为北美和其他全球市场的腾讯云客户提供CDN服务。

Content Distribution Networks (CDNs)

- 具体操作

- 用户请求检索一个特定的视频(一个特定URL)

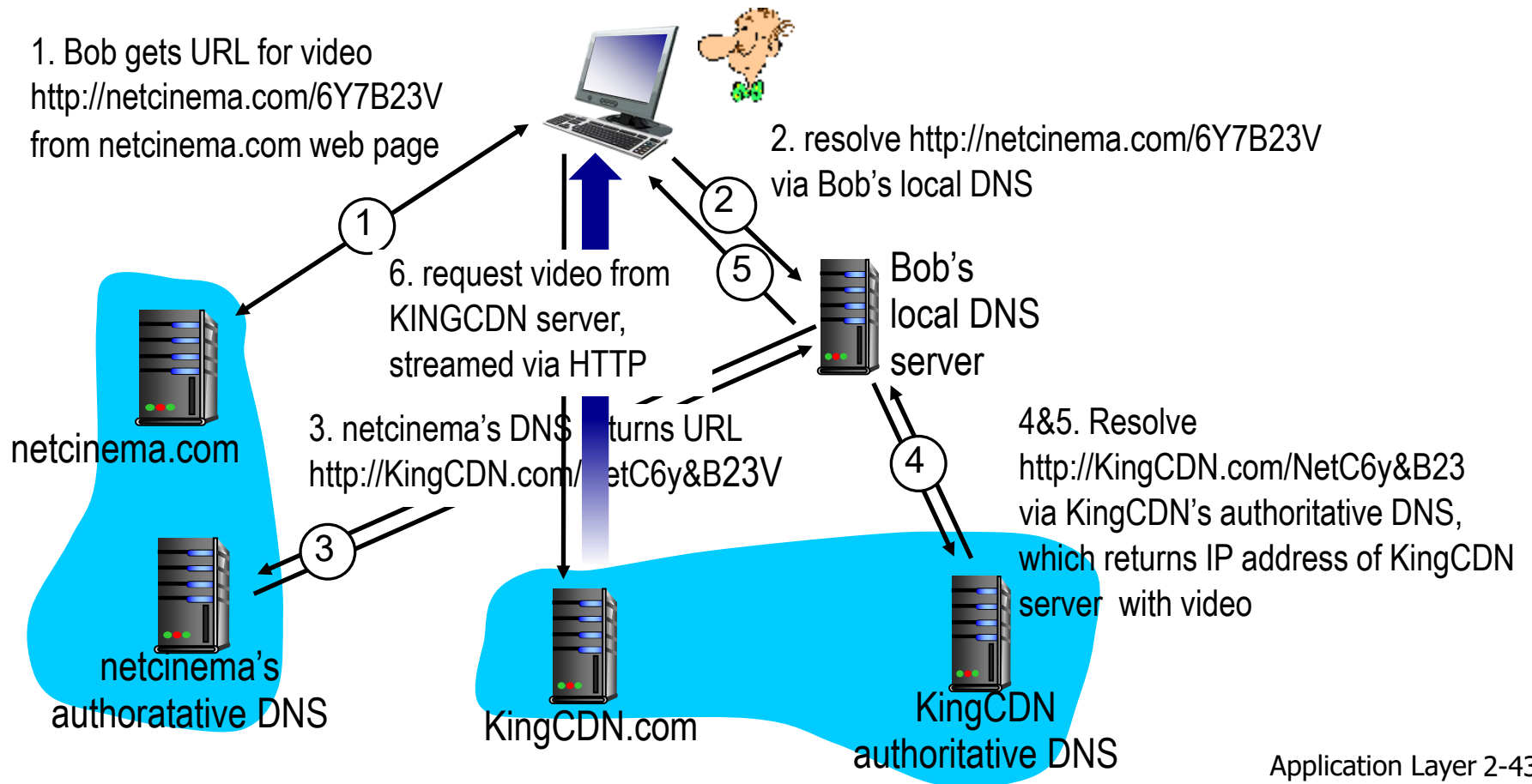
- CDN

- 确定此时适合用于该用户的CDN服务器集群
 - 将客户的请求重定向到该集群的某台服务器
 - 任何CDN部署, 核心是集群选择策略, 如地理位置最近
 - 大多数CDN利用DNS截获和重定向请求

CDN content access: a closer look

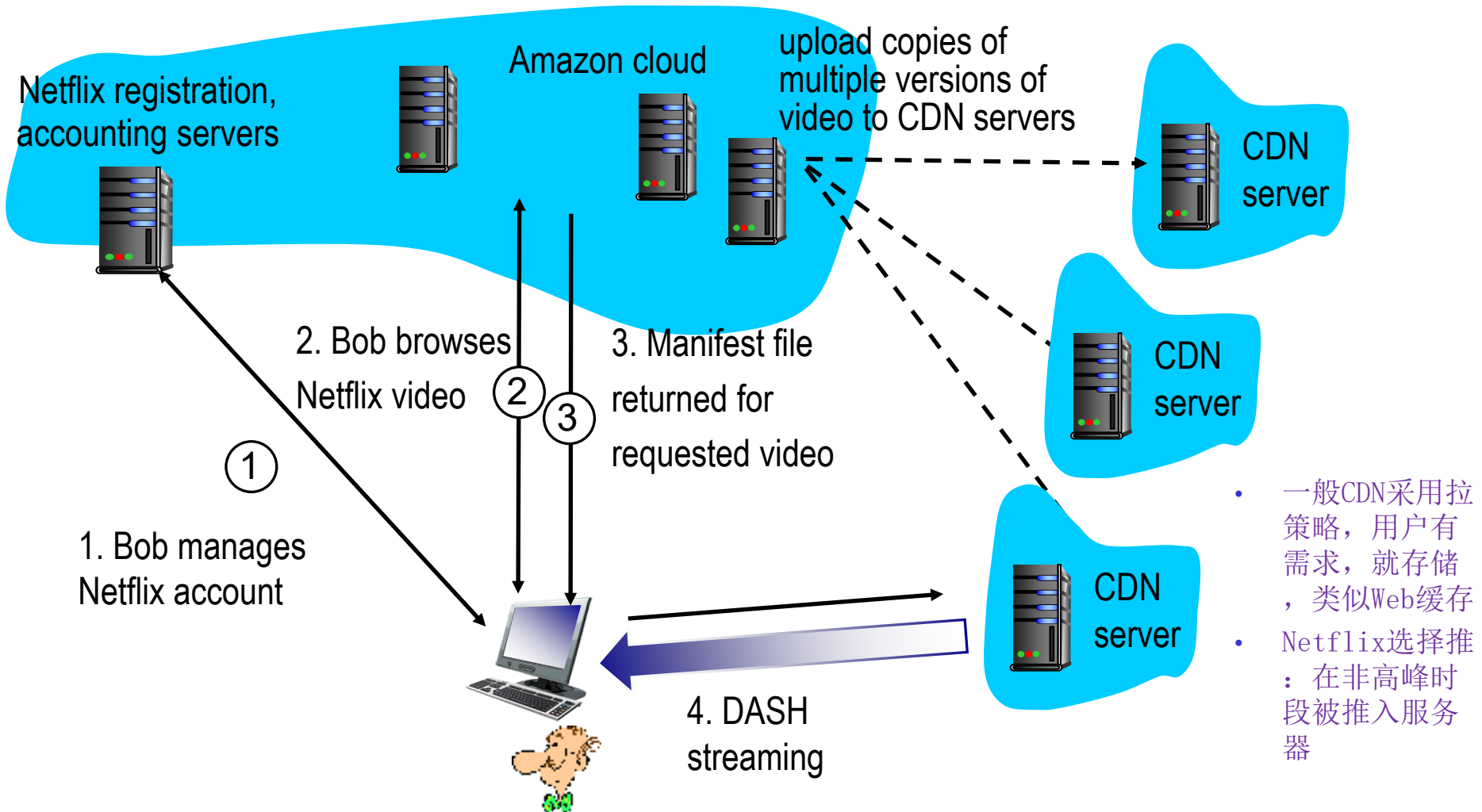
Bob (client) requests video `http://netcinema.com/6Y7B23V`

- video stored in CDN at `http://KingCDN.com/NetC6y&B23V`



Case study: Netflix

- 内容摄取: Netflix得到母带, 上载云端
- 内容处理: 云中机器处理不同格式和比特率
- 向CDN上载版本: 上载到哪些服务器集群?



- 一般CDN采用拉策略, 用户有需求, 就存储, 类似Web缓存
- Netflix选择推: 在非高峰时段被推入服务器

Netflix

- ❖ Netflix成立于1997年，是在线影片租赁提供商，总部美国加利福尼亚州
- ❖ Netflix通过PC、TV及iPad、iPhone收看电影、电视节目，可通过Wii，Xbox360，PS3等设备连接
- ❖ **Netflix百万美元大奖赛**:预测Netflix的客户分别喜欢什么影片，要把预测的性能提高10%以上（2006年10月份开始）
- ❖ 2017年6月6日，《2017年BrandZ最具价值全球品牌100强》公布，Netflix名列第92位
- ❖ 2018年10月，《财富》未来公司50强排行榜发布，Netflix排名第8
- ❖ 2018年12月，世界品牌实验室编制的《2018世界品牌500强》揭晓，排名第88
- ❖ 2019年10月，位列2019福布斯全球数字经济100强榜第46
- ❖ 2019年10月，Interbrand发布的全球品牌百强榜排名65
- ❖ 2020年1月22日，名列2020年《财富》全球最受赞赏公司榜单第16

2.5-2.6小结

□ P2P

- ❖ C/S和P2P的区别
- ❖ 多个节点下载大文件的时间对比
- ❖ BT 洪流、几种机制

□ Video Streaming and Content Distribution Networks

- Internet视频
- HTTP流和DASH
- 内容分发网

提前预习

□ 套接字编程

- ❖ TCP套接字编程
- ❖ UDP套接字编程
- ❖ 语言不限
- ❖ 功能不限



谢谢大家！

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 Electronic Mail
 - SMTP, POP3, IMAP
- ❑ 2.4 DNS
- ❑ 2.5 P2P applications
- ❑ 2.6 Video Streaming and Content Distribution Networks
- ❑ 2.7 Socket programming

2.7 套接字编程

- ❑ **网络应用程序的核心：** 客户机程序和服务器程序
运行时，分别创建一个客户机进程和一个服务器进程，
相互之间通过**套接字读写数据**进行通信
- ❑ **网络应用程序类型**
 - **通用应用程序：** 通过RFC文档所定义的标准协议来实现
程序必须满足该RFC所规定的规则；使用与协议相关的
端口号；如Web应用
 - **专用的应用程序：** 程序不必符合RFC规则；开发者根据
实际应用设计；**不能使用RFC中定义的周知端口号**

套接字编程实例

客户机和服务器经TCP连接进行通信

- 客户机从**键盘读**一行字符，通过套接字向服务器发送
- **服务器**从套接字读取数据
- **将该行字符转换成大写**
- 将修改的行通过其连接套接字再回发给客户机
- **客户机**从其套接字中读取修改的行，并将该行在**显示器上显示**

2.7.1 UDP套接字编程

- UDP是一种无连接的服务，即在两个进程之间没有创建管道时所需的初始握手阶段
- 进程之间的数据传递以分组为单位进行
- 分组中含目的进程地址（主机IP地址和端口号）
- 提供不可靠的传输服务

编程说明

- 通信进程之间没有初始握手，不需要欢迎套接字
- 发送进程将信息字节封装生成分组，再发送
- 接收进程解封收到的分组，获得信息字节

Client/server socket interaction: UDP

server (running on serverIP)

create socket, port= x:
**serverSocket =
socket(AF_INET,SOCK_DGRAM)**

↓
read datagram from
serverSocket

↓
write reply to
serverSocket
specifying
client address,
port number

client

create socket:

**clientSocket =
socket(AF_INET,SOCK_DGRAM)**

↓
Create datagram with server IP and
port=x; send datagram via
clientSocket

↓
read datagram from
clientSocket

↓
close
clientSocket

Example app: UDP client

Python UDPClient

include Python's socket library	→	from socket import *
可以为localhost或 127.0.0.1或服务器IP地址	→	serverName = 'hostname'
端口号, 1024-65535之间	→	serverPort = 12000
create UDP socket for server	→	clientSocket = socket(socket.AF_INET, socket.SOCK_DGRAM)
get user keyboard input	→	message = raw_input('Input lowercase sentence:')
Attach server name, port to message; send into socket	→	clientSocket.sendto(message,(serverName, serverPort))
read reply characters from socket into string	→	modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print out received string and close socket	→	print modifiedMessage clientSocket.close()

Example app: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000

create UDP socket → serverSocket = socket(AF_INET, SOCK_DGRAM)
bind socket to local port number 12000 → serverSocket.bind(("", serverPort))

print "The server is ready to receive"

loop → while 1:
  forever
  Read from UDP socket into message, getting client's address (client IP and port) → message, clientAddress = serverSocket.recvfrom(2048)
  send upper case string back to this client → modifiedMessage = message.upper()
  serverSocket.sendto(modifiedMessage, clientAddress)
```

2.7.2 TCP套接字编程

先建立TCP连接，再进行数据传输

- 客户机程序是连接的发起方
- 服务器必须先准备好，对客户机程序发起的连接做出响应
 - 服务器程序事先已经在系统中运行
 - 服务器程序的一个套接字（欢迎套接字）已经打开，准备接收客户机程序发起的连接（敲门）

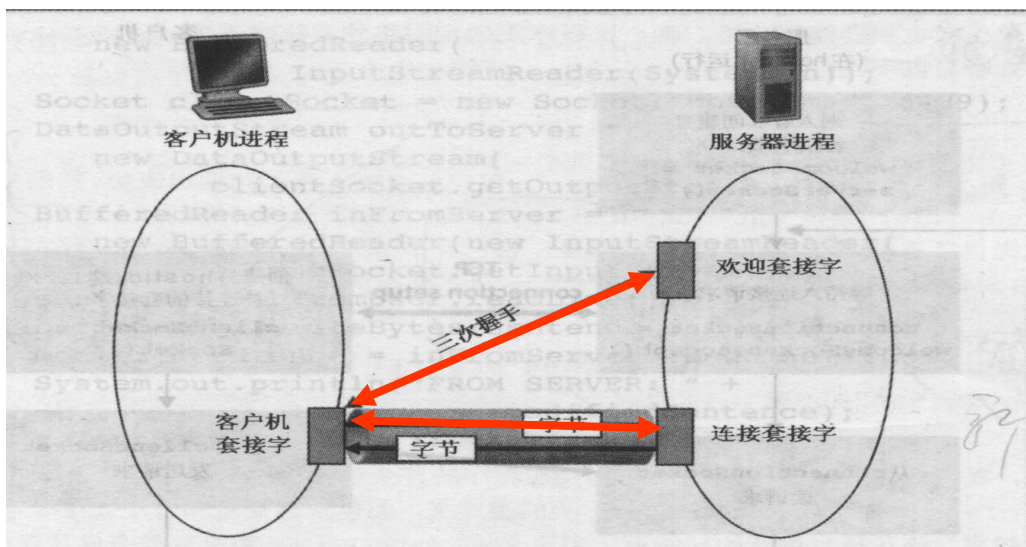
(1) 建立TCP连接

□ 客户机进程向服务器发起一个TCP连接

创建一个本地套接字，指定相应服务器进程的地址（IP地址和端口号）

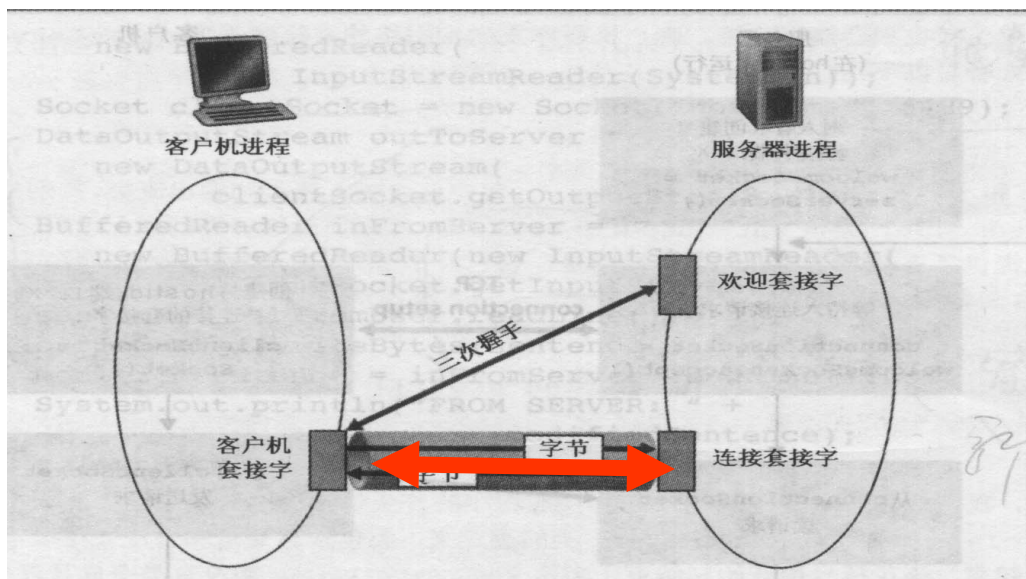
□ 建立一个TCP连接

当服务器听到客户机的连接请求（敲门）时，创建一个新套接字，经过“三次握手”，客户机套接字和服务器套接字之间建立一个TCP连接（直接的虚拟管道）



(2) 传送数据

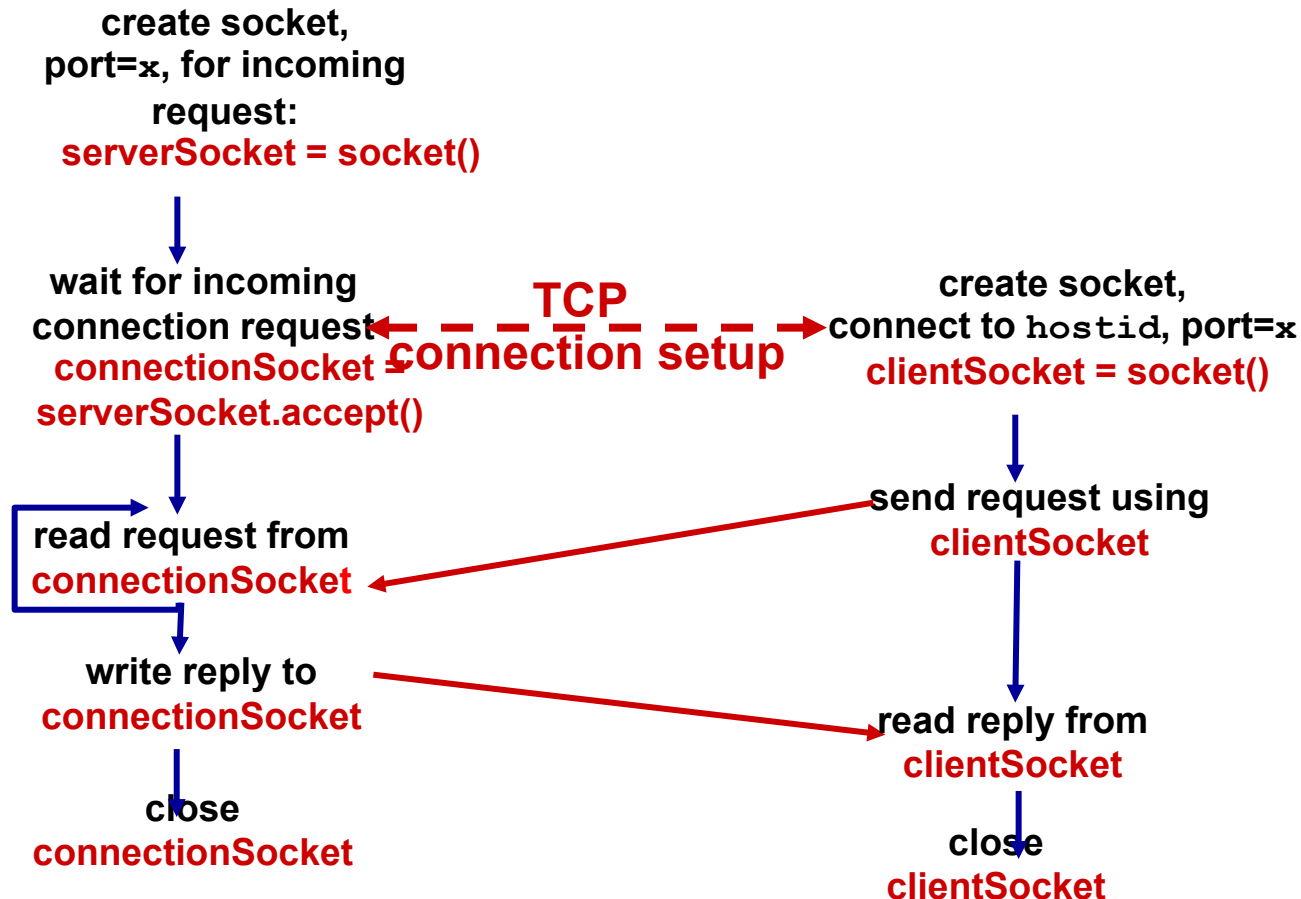
- ❑ TCP连接为客户机和服务器了一个直接的传输管道
- ❑ 可靠的，顺序的，字节流的传输



Client/server socket interaction: TCP

server (running on `hostid`)

client



Example app: TCP client

Python TCPClient

```
from socket import *
```

```
serverName = 'servername'
```

```
serverPort = 12000
```

create TCP socket,
remote port 12000

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

```
clientSocket.connect((serverName, serverPort))
```

```
sentence = raw_input('Input lowercase sentence:')
```

No need to attach server
name, port

```
clientSocket.send(sentence)
```

```
modifiedSentence = clientSocket.recv(1024)
```

```
print 'From Server:', modifiedSentence
```

```
clientSocket.close()
```

Example app: TCP server

Python TCPServer

```
from socket import *
serverPort = 12000

create TCP welcoming socket → serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
server begins listening for incoming TCP requests → print 'The server is ready to receive'
loop forever → while 1:
server waits on accept() → connectionSocket, addr = serverSocket.accept()
for incoming requests, new socket created on return
read bytes from socket (but not address as in UDP) → sentence = connectionSocket.recv(1024)
close connection to this client (but not welcoming socket) → capitalizedSentence = sentence.upper()
connectionSocket.send(capitalizedSentence)
connectionSocket.close()
```

客户机 (UDPClient.java)

产生输入流、套接字→输入→封装→发送

```
import java.io.*;
import java.net.*;
```

```
class UDPClient {
    public static void main(String args[]) throws Exception
    {
```

生成输入流



```
        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));
```

生成客户机

套接字



```
        DatagramSocket ClientSocket = new DatagramSocket();
```

使用DNS将主机名转换为IP地址



```
        InetAddress IPAddress = InetAddress.getByName("hostname");
```

```
        byte[ ] sendData = new byte[1024];
        byte[ ] receiveData = new byte[1024];
```

从输入流读取数据

输入

```
        String sentence = inFromUser.readLine();
        sendData = sentence.getBytes();
```

字符流-字节流 (字符串-字节数组)

生成数据报
指定要发送的数据、
长度、IP地址、端口

```
DatagramPacket sendPacket =
    new DatagramPacket(sendData, sendData.length, IPAddress, 9876);
```

向服务器
发送数据报

```
ClientSocket.send(sendPacket);

DatagramPacket receivePacket =
    new DatagramPacket(receiveData, receiveData.length);
```

从服务器读数据报

```
ClientSocket.receive(receivePacket);

String modifiedSentence =
    new String(receivePacket.getData());
```

显示

```
System.out.println("FROM Server:" + modifiedSentence);
ServerSocket.close();
}

}
```

服务器(*UDPServer.java*)

产生套接字→接收→解封→发送

```
import java.io.*;  
import java.net.*;
```

```
class UDPServer {  
    public static void main(String args[]) throws Exception  
    {
```

在端口**9876**生成
数据报套接字

```
        DatagramSocket ServerSocket = new DatagramSocket(9876);
```

```
        byte[] receiveData = new byte[1024];  
        byte[] sendData = new byte[1024];
```

```
        while(true)  
        {
```

为接收的数据报
生成空间

```
            DatagramPacket receivePacket =  
                new DatagramPacket(receiveData, receiveData.length);
```

接收数据报

```
            ServerSocket.receive(receivePacket);
```

```
String sentence = new String(receivePacket.getData());
```

获得发送方的 IP
地址，端口#

```
InetAddress IPAddress = receivePacket.getAddress();
```

```
int port = receivePacket.getPort();
```

```
String capitalizedSentence = sentence.toUpperCase();
```

```
sendData = capitalizedSentence.getBytes();
```

产生数据报发
送给客户机

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length, IPAddress,  
        port);
```

向套接字写
出数据报

```
ServerSocket.send(sendPacket);
```

```
}  
}  
}
```

循环结束，返回等待另
一个数据报

客户机 : (TCPClient.java)

产生输入流、输出流、套接字→输入→发送→接收→显示

```
import java.io.*;  
import java.net.*;  
class TCP Client {
```

```
    public static void main(String argv[ ]) throws Exception  
    {
```

```
        String sentence;  
        String modifiedSentence;
```

指定从哪里输入

产生输入流

```
        BufferedReader inFromUser =  
            new BufferedReader(new InputStreamReader(System.in));
```

产生客户机套接字,
与服务器连接

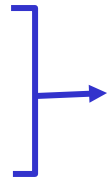
```
        Socket ClientSocket = new Socket("hostname", 6789);
```

生成输出流与
套接字联系

```
        DataOutputStream outToServer =  
            new DataOutputStream(ClientSocket.getOutputStream());
```

指定往哪里输出

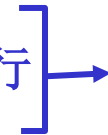
产生与套接字
联系的输入流



```
BufferedReader inFromServer =  
    new BufferedReader(new  
        InputStreamReader(ClientSocket.getInputStream()));
```

指定从哪里得到输入

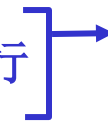
向服务器发送行



```
sentence = inFromUser.readLine();  
  
outToServer.writeBytes(sentence + '\n');
```

键盘输入

从服务器读行



```
modifiedSentence = inFromServer.readLine();
```

显示内容

```
System.out.println("FROM Server: " + modifiedSentence);
```

```
ClientSocket.close();
```

```
    }  
}
```

服务器: (TCPServer.java)

产生输入流、输出流、套接字→接收→发送

```
import java.io.*;  
import java.net.*;
```

```
class TCP Server {
```

```
    public static void main(String argv[]) throws Exception  
    {
```

```
        String ClientSentence;  
        String capitalizedSentence;
```

```
        ServerSocket welcomeSocket = new ServerSocket(6789);
```

```
        while(true) {
```

```
            Socket connectionSocket = welcomeSocket.accept();
```

```
            BufferedReader inFromClient =  
                new BufferedReader(new  
                    InputStreamReader(connectionSocket.getInputStream()));
```

在端口**6789** 生成
欢迎套接字, 监听

创建一个连
接套接字

生成输入流,
与套接字联系

指定从哪里得到输入

指定往哪里输出

生成输出流，
与套接字联系

```
DataOutputStream outToClient =  
    new DataOutputStream(connectionSocket.getOutputStream());
```

从套接字读入客
户机来的数据

```
ClientSentence = inFromClient.readLine();
```

```
capitalizedSentence = ClientSentence.toUpperCase() + '\n';
```

向套接字输出
数据到客户机

```
outToClient.writeBytes(capitalizedSentence);
```

```
    }  
    }  
}
```

小结

□ 应用程序体系结构

- 客户机-服务器
- P2P
- 混合

□ 应用程序服务要求:

- 可靠, 带宽, 时延

□ 因特网传输服务模型

- 面向连接, 可靠: TCP
- 不可靠, 数据报: UDP

□ 特定协议

- HTTP
- SMTP, POP, IMAP
- DNS

□ P2P文件共享

□ 视频流与内容分发

□ 套接字编程

实验内容一：Wireshark的使用

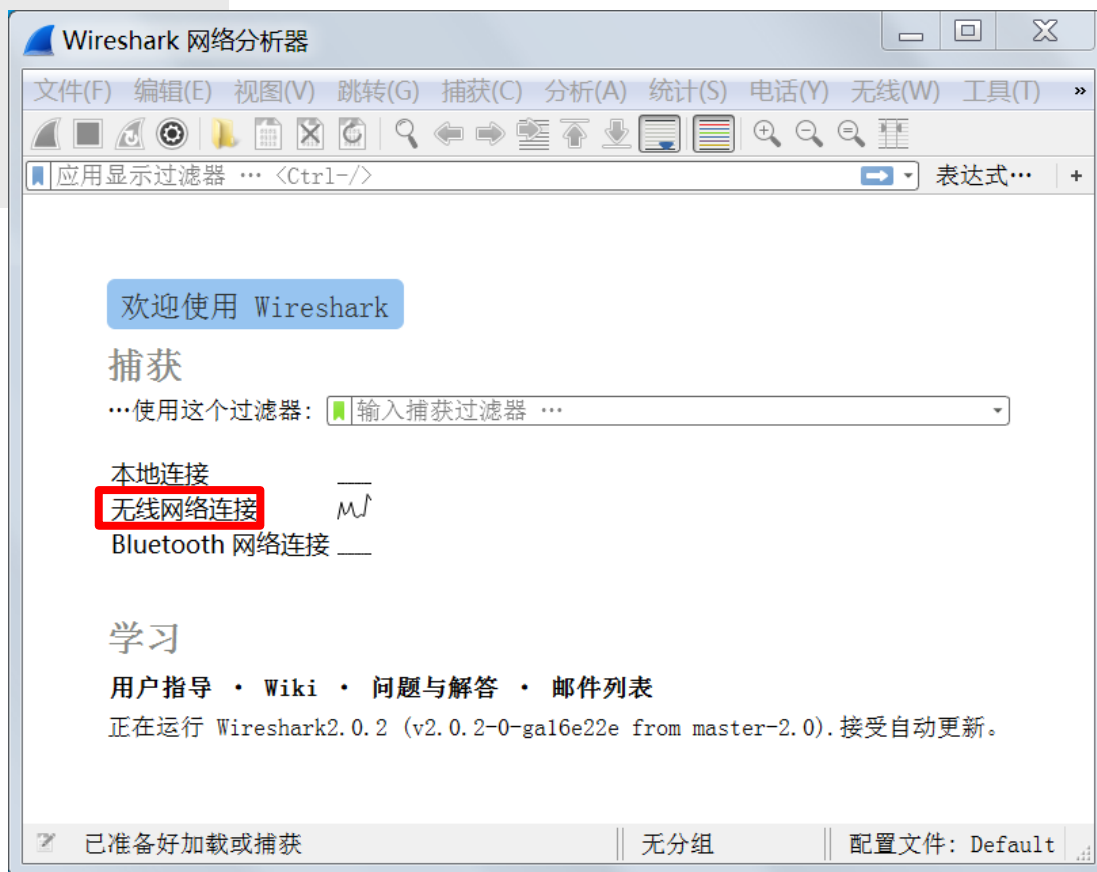
1. Wireshark 下载与安装

Download Wireshark

The current stable release of Wireshark is 2.2.5. It supersedes all previous releases.

Stable Release (2.2.5)

- Windows Installer (64-bit)
- Windows Installer (32-bit)
- Windows PortableApps® (32-bit)
- macOS 10.6 and later Intel 64-bit .dmg
- Source Code



欢迎使用 Wireshark

捕获

...使用这个过滤器:

以太网

WLAN

wireshark出现无法抓包的解决方法

- ❑ <https://jingyan.baidu.com/article/215817f7b313ed1eda142393.html>
- ❑ 以管理员身份运行命令行
- ❑ 在命令行中，输入命令“**net start npf**”
- ❑ 按回车键确定，DOS界面中就会出现“WinPcap Packet Driver (NPF) 服务已经启动成功”的提示。
- ❑ 重启wireshark就可以了

Wireshark使用界面

命令和菜单

协议筛选框

捕获分组列表

选定分组首部明细

分组内容

左：十六进制

右：ASCII 码

No.	Time	Source	Destination	Protocol	Info
8	2.430014	FujianSt_f6:d0:07	Broadcast	ARP	who has 202.202.210.28? Tell 202.202.210.1
9	3.134697	FujianSt_f6:d0:07	Broadcast	ARP	who has 202.202.210.25? Tell 202.202.210.1
10	3.136981	FujianSt_f6:d0:07	Broadcast	ARP	who has 202.202.210.21? Tell 202.202.210.1
11	3.525347	58.60.14.35	202.202.210.105	ICQ	
12	3.678765	202.202.210.105	58.218.149.76	TCP	2104 > 3030 [SYN] Seq=0 Len=0 MSS=1460
13	3.842179	58.218.149.76	202.202.210.105	TCP	3030 > 2104 [RST, ACK] Seq=0 Ack=1 Win=0 Len=0
14	4.135423	FujianSt_f6:d0:07	Broadcast	ARP	who has 202.202.210.26? Tell 202.202.210.1
15	4.136578	FujianSt_f6:d0:07	Broadcast	ARP	who has 202.202.210.7? Tell 202.202.210.1
16	4.137521	FujianSt_f6:d0:07	Broadcast	ARP	who has 202.202.210.28? Tell 202.202.210.1
17	4.287747	202.202.210.105	58.218.149.76	TCP	2104 > 3030 [SYN] Seq=0 Len=0 MSS=1460
18	4.446452	58.218.149.76	202.202.210.105	TCP	3030 > 2104 [RST, ACK] Seq=0 Ack=1 Win=0 Len=0
19	4.891383	202.202.210.105	58.218.149.76	TCP	2104 > 3030 [SYN] Seq=0 Len=0 MSS=1460
20	5.053352	58.218.149.76	202.202.210.105	TCP	3030 > 2104 [RST, ACK] Seq=0 Ack=1 Win=0 Len=0
21	5.135483	FujianSt_f6:d0:07	Broadcast	ARP	who has 202.202.210.21? Tell 202.202.210.1
22	5.137131	FujianSt_f6:d0:07	Broadcast	ARP	who has 202.202.210.25? Tell 202.202.210.1
23	5.311748	202.202.210.105	192.168.5.250	UDP	Source port: 61440 Destination port: 61440
24	5.312097	192.168.5.250	202.202.210.105	UDP	Source port: 61440 Destination port: 61440

Frame 1 (60 bytes on wire, 60 bytes captured)

Ethernet II, Src: FujianSt_f6:d0:07 (00:d0:f8:f6:d0:07), Dst: Broadcast (ff:ff:ff:ff:ff:ff)

- Destination: Broadcast (ff:ff:ff:ff:ff:ff)
- Source: FujianSt_f6:d0:07 (00:d0:f8:f6:d0:07)
 - Type: ARP (0x0806)
 - Trailer: 00000000000000000000000000000000
- Address Resolution Protocol (request)
 - Hardware type: Ethernet (0x0001)
 - Protocol type: IP (0x0800)
 - Hardware size: 6
 - Protocol size: 4
 - opcode: request (0x0001)
 - Sender MAC address: FujianSt_f6:d0:07 (00:d0:f8:f6:d0:07)
 - Sender IP address: 202.202.210.1 (202.202.210.1)

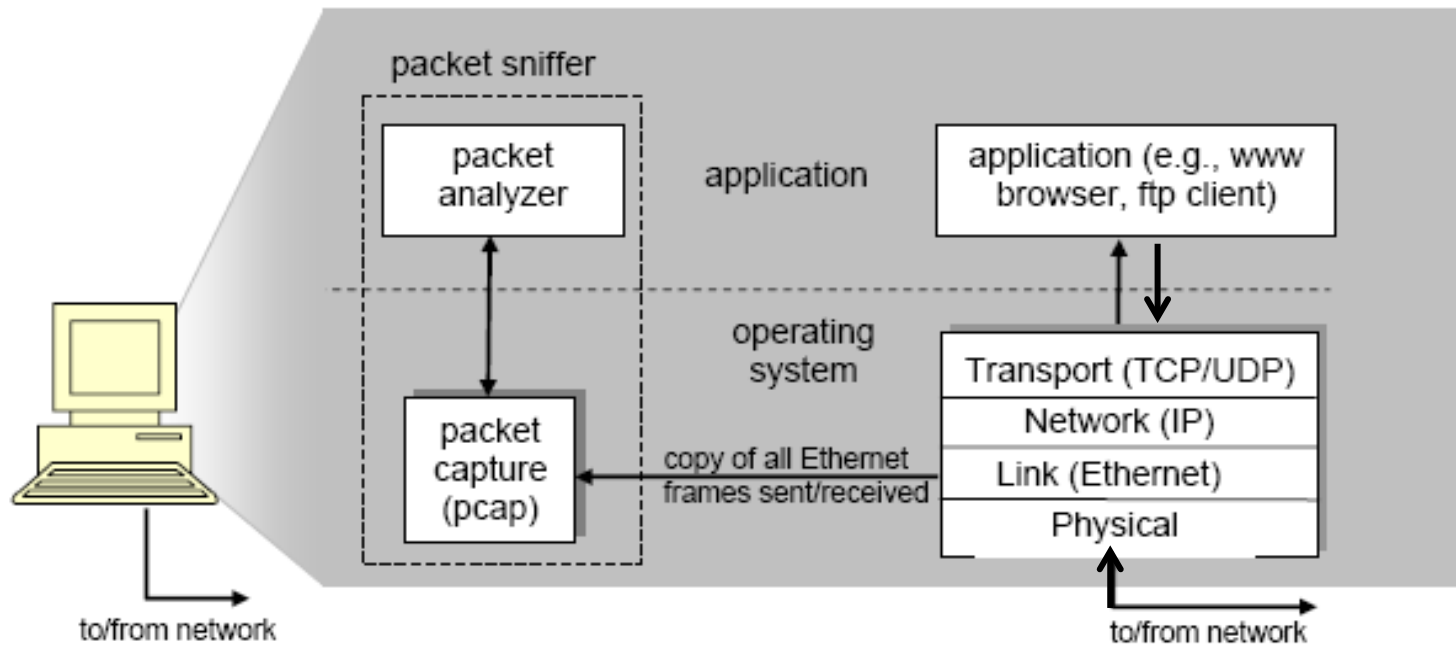
0000 ff ff ff ff ff 00 d0 f8 f6 d0 07 08 06 00 01

0010 08 00 06 04 00 01 00 d0 f8 f6 d0 07 ca ca d2 01

0020 00 00 00 00 00 00 ca ca d2 19 00 00 00 00 00 00

0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

工作原理



□ 左边分组嗅探器

- ❖ 第一是分组捕获器，其功能是捕获计算机发送和接收的每一个链路层帧的拷贝；
- ❖ 第二是分组分析器，其作用是分析并显示协议报文所有字段的内容（它能识别目前正在使用的各种网络协议）。

- 右边是计算机上正常运行的协议和应用程序（如：Web浏览器和FTP客户端）

实验内容二：HTTP协议分析

第一部分 利用 Wireshark 俘获 HTTP 分组

□ (1) 在进行跟踪之前，我们首先清空 Web 浏览器的高速缓存，并清空 DNS 缓存

□ (2) 启动 Wireshark 分组俘获器。

□ (3) 在 Web 浏览器中输入：
<http://www.cqu.edu.cn/> (重庆大学网站)。

□ (4) 停止分组俘获。

回答6个问题

- ❑ Q1.你的浏览器运行的是HTTP1.0，还是HTTP1.1？你所访问的服务器所运行的HTTP版本号是多少？
- ❑ Q2.你的浏览器向服务器指出它能接收何种语言版本的对象？
- ❑ Q3.你的计算机的IP地址是多少？服务器的IP地址是多少？
- ❑ Q4.从服务器向你的浏览器返回的状态代码是多少？
- ❑ Q5.你从服务器上所获取的HTML文件的最后修改时间是多少？
- ❑ Q6.返回到你的浏览器的内容一共多少字节？

第二部分 HTTP条件 GET/response交互

- (1) 启动浏览器，清空浏览器的缓存。
- (2) 启动Wireshark分组俘获器，开始Wireshark分组俘获。
- (3) 在浏览器地址栏中如下网址：
- <http://gaia.cs.umass.edu/wire-shark-labs/HTTP-wireshark-file2.html>
- 你的浏览器中将显示一个的非常简单的HTML文件。
- (4) 在你的浏览器中重新输入相同的URL或单击浏览器中的“刷新”按钮。
- (5) 停止Wireshark分组俘获，在显示过滤筛选说明处输入“http”,分组列表子窗口中将只显示所俘获到的HTTP报文。

- ❑ Q7.分析你的浏览器向服务器发出的第一个HTTP GET请求的内容，在该请求报文中，是否有一行是：IF-MODIFIED-SINCE？
- ❑ Q8.分析服务器响应报文的内容，服务器是否明确返回了文件的内容？如何获知？
- ❑ Q9.分析你的浏览器向服务器发出的第二个“HTTP GET”请求，在该请求报文中是否有一行是：IF-MODIFIED-SINCE？如果有，在该首部行后面跟着的信息是什么？
- ❑ Q10.服务器对第二个HTTP GET请求的响应中的HTTP状态代码是多少？服务器是否明确返回了文件的内容？请解释。

协议分析注意事项

*以太网

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

应用显示过滤器 ... <Ctrl-F> 表达式...

No.	Time	Source	Destination	Protocol	Length	Info
404	5.958635	10.250.110.14	183.61.51.120	TCP	54	53025 → 36688 [ACK] Seq=226 Ack=21 Win=65516 Len=0
405	6.268149	HuaweiTe_ef:c3:ac	Broadcast	0x0607	60	Ethernet II
406	6.277940	HuaweiTe_ef:c3:ac	Broadcast	0x0607	60	Ethernet II
407	6.666319	10.250.110.14	125.39.247.226	UDP	292	54359 → 8000 Len=250
408	6.667182	10.250.110.14	125.39.247.226	UDP	420	54359 → 8000 Len=378
409	6.692565	10.250.110.14	202.202.0.33	DNS	74	Standard query 0xea66 A www.cqu.edu.cn
410	6.694265	202.202.0.33	10.250.110.14	DNS	164	Standard query response 0xea66 A www.cqu.edu.cn A 202.202.1.3 NS pridns.cqu.edu.cn NS secdns...
411	6.695265	10.250.110.14	202.202.1.3	TCP	66	53026 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=8 SACK_PERM=1
412	6.695270	10.250.110.14	202.202.1.3	TCP	66	53027 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=8 SACK_PERM=1
413	6.696205	202.202.1.3	10.250.110.14	TCP	66	80 → 53027 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1448 SACK_PERM=1 WS=128
414	6.696281	10.250.110.14	202.202.1.3	TCP	54	53027 → 80 [ACK] Seq=1 Ack=1 Win=262144 Len=0
415	6.696619	10.250.110.14	202.202.1.3	HTTP	334	GET / HTTP/1.1
416	6.697550	202.202.1.3	10.250.110.14	TCP	66	80 → 53026 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1448 SACK_PERM=1 WS=128
417	6.697551	202.202.1.3	10.250.110.14	TCP	60	80 → 53027 [ACK] Seq=1 Ack=281 Win=30336 Len=0
418	6.697656	10.250.110.14	202.202.1.3	TCP	54	53026 → 80 [ACK] Seq=1 Ack=1 Win=262144 Len=0
419	6.698140	202.202.1.3	10.250.110.14	HTTP	601	HTTP/1.1 200 Found (text/html)

> Frame 409: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 0

> Ethernet II, Src: AsustekC_b1:52:76 (40:16:7e:b1:52:76), Dst: HuaweiTe_67:28:99 (28:6e:d4:67:28:99)

> Internet Protocol Version 4, Src: 10.250.110.14, Dst: 202.202.0.33

> User Datagram Protocol, Src Port: 65522, Dst Port: 53

▼ Domain Name System (query)

[Response In: 410]

Transaction ID: 0xea66

> Flags: 0x0100 Standard query

Questions: 1

Answer RRs: 0

Authority RRs: 0

Additional RRs: 0

▼ Queries

> www.cqu.edu.cn: type A, class IN

条件GET请求

609	10.803995	10.250.110.14	128.119.245.12	HTTP	581 GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1
612	11.095127	128.119.245.12	10.250.110.14	HTTP	293 HTTP/1.1 304 Not Modified

Frame 609: 581 bytes on wire (4648 bits), 581 bytes captured (4648 bits) on interface 0

Ethernet II, Src: AsustekC_b1:52:76 (40:16:7e:b1:52:76), Dst: HuaweiTe_67:28:99 (28:6e:d4:67:28:99)

Internet Protocol Version 4, Src: 10.250.110.14, Dst: 128.119.245.12

Transmission Control Protocol, Src Port: 53332, Dst Port: 80, Seq: 777, Ack: 1215, Len: 527

Hypertext Transfer Protocol

> GET /wireshark-labs/HTTP-wireshark-file2.html HTTP/1.1\r\n

Host: gaia.cs.umass.edu\r\n

Connection: keep-alive\r\n

Cache-Control: max-age=0\r\n

Upgrade-Insecure-Requests: 1\r\n

User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/54.0.2840.99 Safari/537.36\r\n

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8\r\n

Accept-Encoding: gzip, deflate, sdch\r\n

Accept-Language: zh-CN,zh;q=0.8\r\n

If-None-Match: "173-54a46c7a5faf6"\r\n

If-Modified-Since: Thu, 09 Mar 2017 06:59:01 GMT\r\n

\r\n

[Full request URI: <http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html>]

[HTTP request 3/3]

[Prev request in frame: 569]

[Response in frame: 612]

- ❑ 实验命名规则:
- ❑ 学号+姓名+第一次实验报告

谢谢大家！